

Slovníkové metody (Dictionary methods)

Zdeněk VAŠÍČEK

Fakulta informačních technologií, Vysoké učení technické v Brně

Brno, Czech Republic

vasicek@fit.vutbr.cz

Obsah

■ Metoda LZ77

- Princip
- Vyhledávací binární strom (BST)
- Varianty LZ77 (LZSS, LZOP, LZ4)

■ Metoda LZ78

- Princip
- Vyhledávací n-ární strom (datová struktura Trie)
- Varianty LZ78 (LZFG, LZRW, LZW, LZMW, LZAP, LZY)

■ Využití slovníkových metod

- Diferencování souborů s využitím LZ77
- GZIP, Deflate

Komprese textu s využitím statického slovníku

- Nejjednodušším příkladem **statického slovníku** je slovník jazyka anglického, použitého ke kompresi anglického textu.
- **Slovník**
 - Předpokládejme slovník obsahující cca 500 tisíc anglických slov (bez jejich definice).
 - Slovem nechť je řetězec symbolů zakončený mezerou nebo interpunkčním znakem.
- **Princip komprese (logická komprese)**
 - Slovo se načte ze vstupního toku a následně se prohledá slovník. Když se nalezne shoda, tak se ukazatel do slovníku zapíše do výstupního toku. Jinak se zapíše samotné nekomprimované slovo.
 - Výstupní tok obsahuje **dva typy tokenů** – indexy a prostá slova, a důležité je umět mezi nimi rozlišit (můžeme např. v každé zapisované položce vyhradit jeden extra bit)
 - Výstupem bude token **<0, index>** pokud se nalezne shoda (index je 19-bitové číslo, 2^{19} odpovídá 524 288) a token **<1, délka, 'řetězec'>** pokud shoda neexistuje (délka nechť je 7-bitové číslo)

b	e	t		x	e	t	
---	---	---	--	---	---	---	--

<0, 1025>

0|0000000010000000001

20 bitů

<1, 4, 'xet '>

1|0000100|01111000|01100101
|01110100|00100000

40 bitů

Analýza efektivity metody se statickým slovníkem

- Předpokládáme-li, že velikost se zapíše jako 7-bitové číslo, a že střední velikost slova je pět znaků, pak nekomprimované slovo zabírá průměrně 6 bytů (48 bitů) výstupního toku. Komprese 40 bitů na 20 je vynikající, za předpokladu, že taková situace nastává dostatečně často.
- Ke kolika shodám musí dojít, aby se dosáhlo komprese?
 - Po přečtení a kompresi N slov o průměrné velikosti vstupního toku $40N$ (5 znaků na slovo) bude velikost výstupního toku $N[20P + 48(1 - P)] = N[48 - 28P]$ bitů, přičemž P nechť označuje pravděpodobnost shody (případu, kdy se slovo ve slovníku najde).
 - Komprese se dosáhne, když $N[48 - 28P] < 40N$, což znamená $P > 0,29$, tzn. abychom dosáhli komprese, potřebujeme frekvenci shod 29% nebo lepší.
- Jaký bude kompresní faktor pro $P=0,9$?
 - Velikost výstupního toku je $N[48 - 28P] = N[48 - 25,2] = 22,8N$. Velikost vstupního toku je, jako dříve, $40N$. Kompresní faktor je tedy $40/22,8 \approx 1,75$.
- Pokud je tvořen vstupní tok anglickým textem, většina slov se v takto sestaveném slovníku najde. Jiný typ dat se však nebude chovat tak dobře a může snadno dojít i k expanzi (např. zdrojový kód v C, binární data, apod.)
- Statický slovník **není** pro univerzální kompresor **vhodná volba**. Pro jednoúčelový kompresor to však může být vhodná koncepce (viz databáze prodejny železářských potřeb – šroub MX).

Komprese textu s využitím dynamického slovníku

- Adaptivním slovníkovým metodám se dává přednost z důvodu větší obecnosti.
- Princip adaptivní slovníkové metody:
 - **počáteční slovník:** typicky se začíná s prázdným, nebo malým slovníkem a postupně se přidávají další slova tak, jak se vyskytují ve vstupním toku a odmazávají stará slova (veliký slovník znamená pomalé vyhledávání)
 - **algoritmus:** typicky obsahuje smyčku, ve které každá iterace provede
 1. čtení vstupního toku a jeho rozklad (parsování) do slov a frází.
 2. vyhledání slova ve slovníku a vygenerování tokenu. Pokud dojde ke shodě, zapíše do výstupního toku příslušný token. V opačném případě se musí vyslat nekomprimované slovo a přidat do slovníku.
 3. promazání slovníku, tj. test, který zjišťuje, které slovo ze slovníku odstranit.
- **Výhody:**
 - Neobsahuje numerické výpočty jako statistické metody, pouze operace prohledávání řetězce a srovnávání.
 - Asymetrická kompresní metoda s jednoduchým dekodérem. Dekodér nemusí vstupní tok složitě rozkládat a nemusí k vyhledávání shody prohledávat slovník, pouze čte a interpretuje tokeny.

Huffmanův kód

- Mějme stejný vstupní řetězec, který zkomprimujeme statistickou kompresní metodou – Huffmanovým kódem a budeme sledovat kompresní poměr. Huffmanův kód přiřazuje častěji se vyskytujícím symbolům kratší kód.

a a c a a c a a c a a c a a c a a c a a c a a a b c a b a a a c d a d a f a f

$L(x)=40, H(x)=1.56$

Symbol	Výskytů	Kód
a	25	1
c	9	01
f	2	000
b	2	0010
d	2	0011

Výstup (hlavička)

<5>

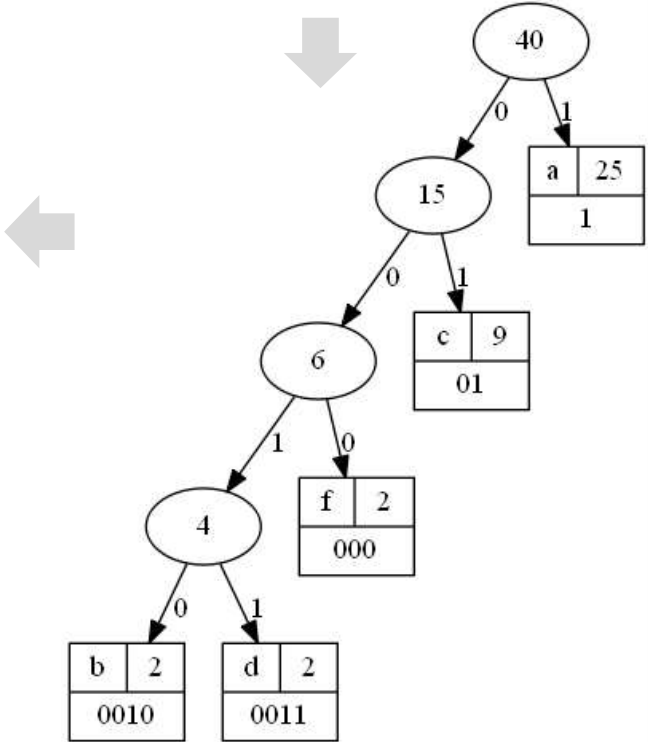
<'a',1,'1'>

<'c',2,'01'>

<'f',3,'000'>

<'b',4,'0010'>

<'d',4,'0011'>



0000010101100001000110110001100100101100110001100001100010010000100110010001000011

$L_{out}=82$ bits prvních 8 bitů počet položek, položka: 8 bitů ASCII + 4 bity délka kódu + bity kódu

Huffmanův kód

- Vstupní řetězec lze zakódovat pomocí 147 bitů (3,675 bitů na symbol). Dosažen kompresní faktor 0.459 (147 / 40*8). Vlastní data (tj. bez hlavičky) jsou zakódována pomocí 65 bitů (1,625 bitů na symbol), hlavička (tabulka symbolů) je zakódována pomocí 82 bitů.

a a c a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=1.56



hlavička

00000101011000010001101100011001001011001
10001100001100010010000100110010001000011

data

110111011101110111011101110111011100100110
0101110100111001110001000

Symbol	Výskytů	Kód
a	25	1
c	9	01
f	2	000
b	2	0010
d	2	0011

Huffmanův kód – kanonická verze

- Mějme stejný vstupní řetězec, který zkomprimujeme statistickou kompresní metodou – Huffmanovým kódem a budeme sledovat kompresní poměr. Huffmanův kód přiřazuje častěji se vyskytujícím symbolům kratší kód.

a a c a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f

$L(x)=40, H(x)=1.56$

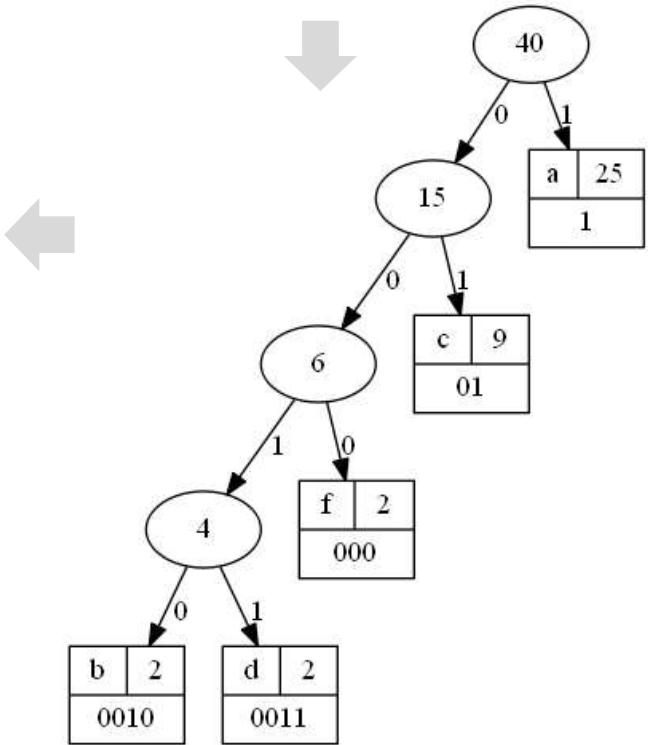
Symbol	Výskytů	Kód
a	25	1
c	9	01
f	2	000
b	2	0010
d	2	0011

Výstup (hlavička)

<5>

<1, 1, 1, 2>

<'a','c','f','b','d'>



00000101 00000001000000010000000100000010 01100001 01100011 01100110 01100010 01100100

$L_{out}=80$ bits prvních 8 bitů počet položek, následují 4 8bitové položky počtu kódových slov dané délky, následuje 5 8bitových položek s délkami

Kompresce řetězců

- Obecně platí, že kompresní metody pracující s řetězcí symbolů mohou být účinnější, než metody, které komprimují jednotlivé symboly.

- Demonstrační příklad:

- Mějme abecedu obsahující dva symboly a a b s pravděpodobnostmi výskytu $P(a)=0,8$ a $P(b)=0,2$
- Jakýkoliv kód s proměnnou délkou pracující na úrovni symbolů přidělí těmto symbolům 1-bitové kódy, takže průměrná délka kódu bude **1 bit na symbol** (nezávisle na délce řetězce).

Symbol x	$P(x)$	Kód
a	0.8	1
b	0.2	0

- Nagenerrujeme všechny řetězce délky 2 společně s jejich pravděpodobnostmi a Huffmanovými kódy. Střední délka Huffmanova kódu je $1 \times 0,64 + 2 \times 0,16 + 3 \times 0,16 + 3 \times 0,04 = 1,56$ bitů na řetězec, což je **0,78 bitů na symbol**.

Řetězec w	$P(w)$	Kód
aa	$0.8 \times 0.8 = 0.64$	1
ba, ab	$0.2 \times 0.8 = 0.16$	00, 011
bb	$0.2 \times 0.2 = 0.04$	010

- V případě řetězců délky 3 je efektivní střední délka Huff. kódu rovna $1 \times 0,512 + 3 \times 3 \times 0,128 + 3 \times 5 \times 0,032 + 5 \times 0,008 = 2,184$ bitů na řetězec, což dává **0,728 bitů na symbol**.

Řetězec w	$P(w)$	Kód
aaa	$0.8 \times 0.8 \times 0.8 = 0.512$	1
aab, aba, baa	$0.8 \times 0.8 \times 0.2 = 0.128$	011, 001, 010
abb, bab, bba	$0.8 \times 0.2 \times 0.2 = 0.032$	00001, 00010, 00011
bbb	$0.2 \times 0.2 \times 0.2 = 0.008$	00000

Slovníková metoda LZ77 a LZ78

- První metody pro slovníkovou kompresi (LZ77, LZ78) navrženy v sedmdesátých letech dvojicí Jacob Ziv a Abraham Lempel.
 - Jejich myšlenky byly zdrojem inspirace pro mnoho výzkumníků, kteří je zobecnili, vylepšili a zkombinovali s metodami RLE a statistickými, čímž vzniklo mnoho všeobecně užívaných bezztrátových kompresních metod pro text, obraz a zvuk, které umožnily významný rozvoj mobilních technologií.



Jacob Ziv
(†25.3.2023, 91 let, Israel)

1997 ocenění [Claude E. Shannon Award](#) udělené [IEEE Information Theory Society](#)

2021: ocenění [IEEE Medal of Honor](#), nejvyšší uznání IEEE, za "fundamental contributions to information theory and data compression technology, and for distinguished research leadership"



Abraham Lempel (†4.2.2023, 86 let, Israel)

1998: ocenění [Golden Jubilee Award](#) "for Technological Innovation" udělené [IEEE Information Theory Society](#)

2007: [IEEE Richard W. Hamming Medal](#) za "pioneering work in data compression, especially the Lempel-Ziv algorithm".

Slovníková metoda LZ77

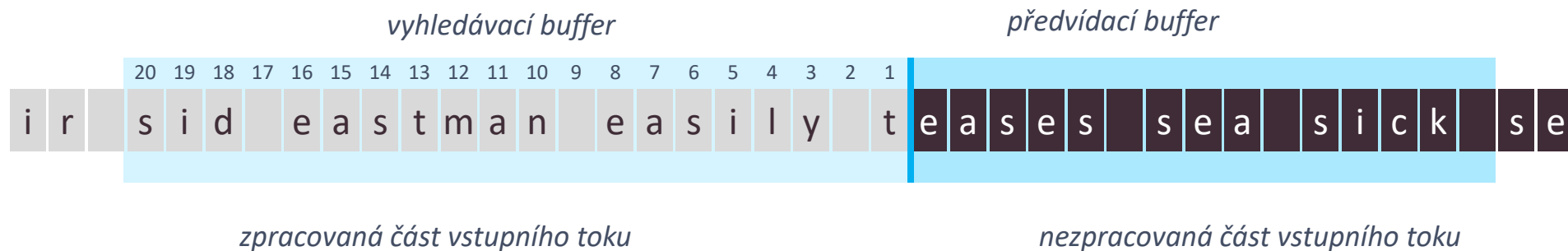
- První metody pro slovníkovou kompresi (LZ77, LZ78) navrženy v sedmdesátých letech dvojicí Jacob Ziv a Abraham Lempel.
 - Jejich myšlenky byly zdrojem inspirace pro mnoho výzkumníků, kteří je zobecnili, vylepšili a zkombinovali s metodami RLE a statistickými, čímž vzniklo mnoho všeobecně užívaných bezztrátových kompresních metod pro text, obraz a zvuk, které umožnily významný rozvoj mobilních technologií.
- Princip LZ77:
 - Základní myšlenkou této metody (někdy označované LZ1) je použít části dříve načteného vstupního toku jako „slovníku“.
 - Kodér udržuje nad vstupním tokem okno (tzv. sliding window), do kterého nasouvá zprava data (lze interpretovat také tak, že okno se posouvá nad daty zleva doprava).



Slovníková metoda LZ77

■ Okno

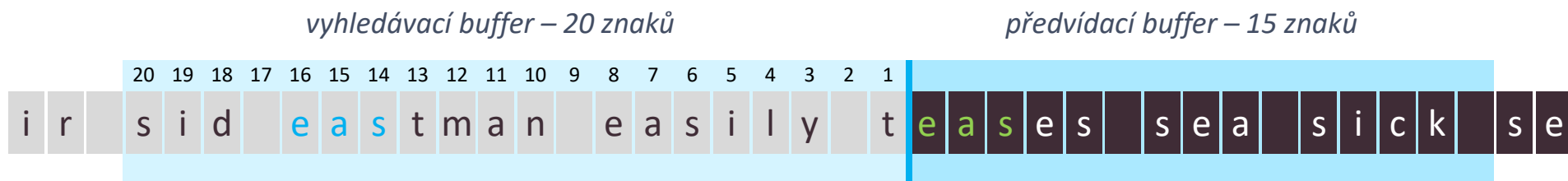
- rozděleno na levou část označovanou jako *vyhledávací buffer* (search buffer) a pravou část známou jako *předvídací buffer* (look-ahead buffer).
- V praktických implementacích je vyhledávací buffer dlouhý několik tisíc bytů, zatímco předvídací buffer je dlouhý pouze několik desítek bytů.
- Vyhledávací buffer obsahuje již zpracovaná data, předvídací buffer data, která právě jsou nebo teprve budou komprimována.
- Kodér prohlíží vyhledávací buffer zprava doleva (od konce, odzadu) a vyhledává nejdelší shodu řetězce ve vyhledávacím a předvídacím bufferu.



Slovníková metoda LZ77

■ Princip hledání nejdelší shody:

- Kodér prohlíží vyhledávací buffer zprava doleva (od konce, odzadu) a vyhledává shodu s prvním symbolem v předvídacím bufferu. Následně hledá shodu v co nejvíce symbolech. Poté kodér pokračuje ve zpětném prohlížení a snaží se najít delší shodu. Kodér vybere nejdelší shodu, anebo když jsou všechny stejně dlouhé, pak tu poslední nalezenou (zjednodušuje kodér).



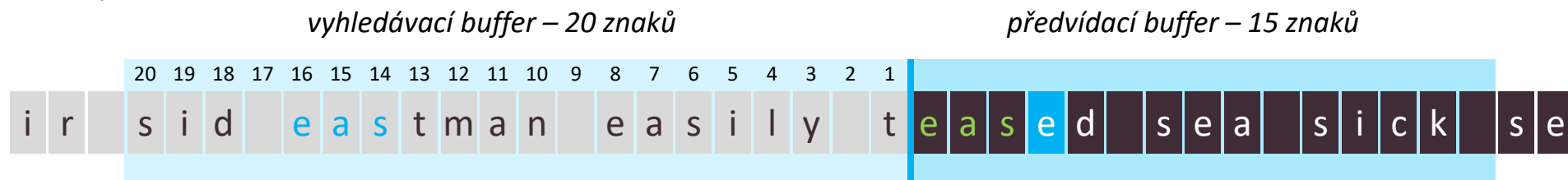
Příklad:

- První shoda s 'e' nalezena ve vzdálenosti 8 (slovo easily) o délce shody 3 symboly.
- Kodér pokračuje ve zpětném prohlížení a snaží se najít delší shodu.
- Další shoda nalezena ve vzdálenosti 16 (slovo eastman) o délce shody 3 symboly.
- Kodér vybere tuto shodu a pokračuje ve zpětném prohlížení. Další shoda se však již ve vyhledávacím bufferu nenachází.

Slovníková metoda LZ77

■ Formát tokenu LZ77:

- token má tři části a sice **<offset, délka, symbol>**, kde "offset" kóduje pozici shody ve vyhledávacím bufferu, "délka" délku shody, tj. počet symbolů ke zkopírování a "symbol" odpovídá symbolu nacházejícímu se v předvídacím bufferu bezprostředně za nalezenou shodou.
- V případě, že nebyla nalezena shoda, pak kodér zapíše do výstupu token <0,0,'x'> (shoda na pozici 0 délky 0), kde 'x' je symbol v předvídacím bufferu.
Takové tokeny se budou vyskytovat často na začátku, kdy vyhledávací buffer je prázdný, nebo téměř prázdný.



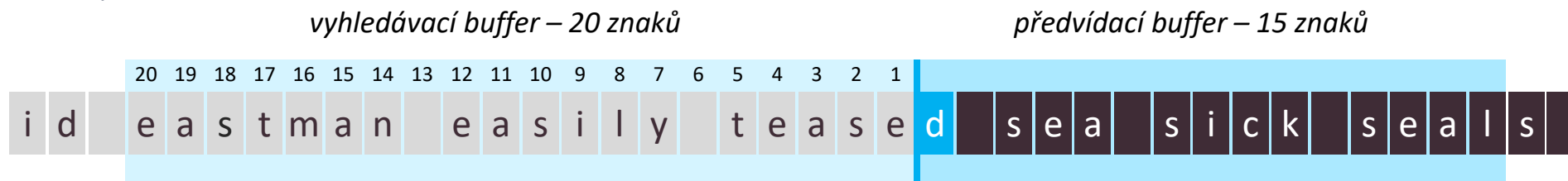
Příklad (krok n):

- Kodér našel shodu ve vzdálenosti 16 o délce 3 symboly. Na výstup proto zapíše token <16,3,'e'> neboť znak **e** následuje bezprostředně za shodou v předvídacím bufferu.
- Následně kodér posune pohyblivé okno o čtyři pozice doprava a pokračuje v hledání shody.

Slovníková metoda LZ77

■ Formát tokenu LZ77:

- token má tři části a sice **<offset, délka, symbol>**, kde "offset" kóduje pozici shody ve vyhledávacím bufferu, "délka" délku shody, tj. počet symbolů ke zkopírování a "symbol" odpovídá symbolu nacházejícímu se v předvídacím bufferu bezprostředně za nalezenou shodu.
- V případě, že nebyla nalezena shoda, pak kodér zapíše do výstupu token <0,0,'x'> (shoda na pozici 0 délky 0), kde 'x' je symbol v předvídacím bufferu.
Takové tokeny se budou vyskytovat často na začátku, kdy vyhledávací buffer je prázdný, nebo téměř prázdný.



Příklad (krok n+1):

- Kodér shodu ve vyhledávacím bufferu nenalezl (symbol 'd' se již v bufferu nenachází), na výstup proto zapíše token <0,0,'d'>.
- Následně kodér posune pohyblivé okno o jednu pozici doprava a pokračuje v hledání shody.

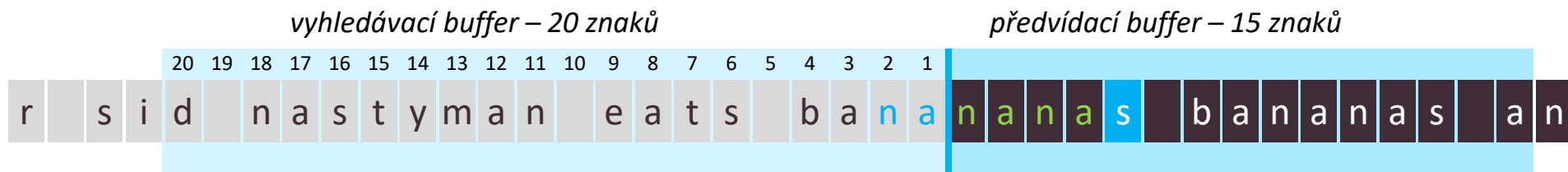
Slovníková metoda LZ77

- **Kódování tokenu $\langle \text{offset}, \text{délka}, \text{symbol} \rangle$:**
 - Nechť S je délka vyhledávacího bufferu a L délka předvídacího bufferu, pak pro pole “offset” je nutné vyhradit $\lceil \log_2 S \rceil$ bitů a pro pole „délka“ je nutné vyhradit $\lceil \log_2 (L - 1) \rceil$ bitů. Symbol je typicky kódován pomocí 8 bitů, ale obecně $\lceil \log_2 A \rceil$ bitů, kde A je velikost abecedy.
- **Značka ve tvaru $\langle 0, 0, 'x' \rangle$, která kóduje jediný symbol, neposkytuje dobrou kompresi. Počet výskytů by proto měl být minimální (dostatečně velký vyhledávací buffer).**
 - V praxi může být vyhledávací buffer dlouhý několik tisíc bytů, takže délka offsetu je typicky 10-12 bitů.
 - Předvídací buffer je typicky krátký, ne více než 32 znaků.
 - Celková délka značky pro jeden symbol $\langle 0, 0, 'x' \rangle$ může být typicky $11 + 5 + 8 = 24$ bitů, tedy mnohem delší, než základní 8-bitová velikost jednoho symbolu, který kóduje.

Slovníková metoda LZ77

■ Efektivní kódování opakujících se sekvencí

- Aby bylo možné efektivně kódovat opakující se sekvence, LZ77 umožňuje, aby pole “délka” nabývalo hodnoty větší, než pole “offset”.
- Výhodou je, že dekodér může zpracovat takovou značku přirozeně, bez jakýchkoliv úprav. Začne v pozici k svého vyhledávacího bufferu a okopíruje následujících $k + n$ znaků, jeden po druhém, a rozšiřuje přitom buffer doprava. Prvních k symbolů jsou kopie starého obsahu bufferu, a dalších n je kopií prvních n z těch k .



Příklad:

- Kodér našel shodu ve vyhledávacím bufferu na pozici 2 délky 4 symboly neboť řetězec “nana” se shoduje s prvními dvěma symboly ve vyhledávacím bufferu a následujícími dvěma symboly v předvídacím bufferu.
- Dále kodér nalezne shodu na pozici 18, avšak ta má délku pouze 2 symboly a na výstup se proto zapíše token $\langle 2, 4, 's' \rangle$.

Slovníková metoda LZ77

■ Kodér (pseudokód)

```
naplň okno vstupním tokem (předvídací buffer začíná prvním znakem)
while (okno není prázdné) do
  begin
    najdi nejdelší prefix předvídacího bufferu p ve vyhledávacím bufferu
    i := pozice p v bufferu
    j := délka p
    X := první znak v předvídacím bufferu za p
    ulož značku <i,j,'X'>
    posuň okno o j+1 znaků
  end
```

■ Dekodér je mnohem jednodušší, než kodér (asymetrická kompresní metoda).

- Dekodér musí udržovat buffer s velikostí rovnou oknu kodéru. Načte značku, zkopíruje z bufferu shodu a třetí položku do výsledného toku a nasune zapsaná data do bufferu.

Slovníková metoda LZ77

■ Implementace posuvného okénka:

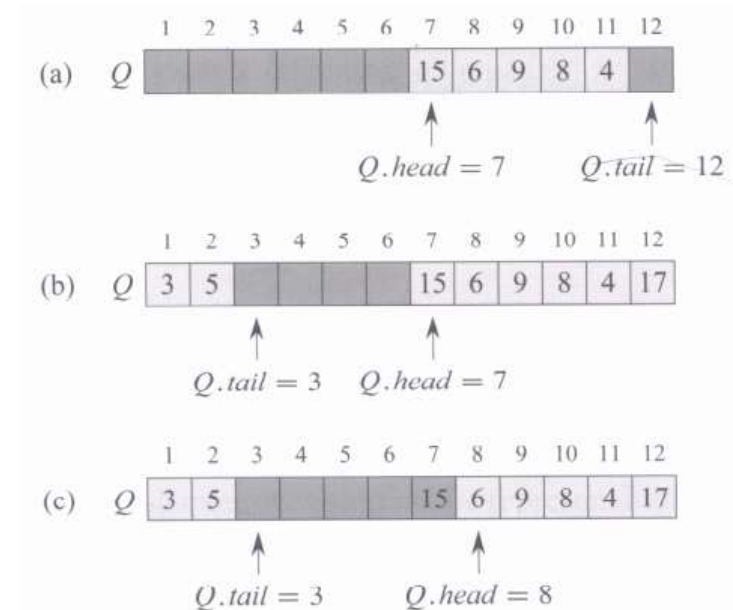
- používá se struktura známá jako kruhová fronta (circular queue), neimplementuje se jako fyzický posun položek v paměti (drahé)
- kruhová fronta Q se typicky realizuje pomocí pole pevné délky ($Q.length$) a dvou ukazatelů – pozice počátku ($Q.head$) dat a pozice konce ($Q.tail$) dat
- nad strukturou jsou definovány dvě operace – přidání položky na konec (enqueue) a přečtení položky ze začátku (dequeue), my potřebujeme kombinaci obou (tzv. shift)
- fronta je plná pokud $Q.head = Q.tail + 1$,
- počáteční stav je $Q.head = Q.tail = 0$

SHIFT(Q, x)

$Q[Q.tail] = x$

$Q.head = (Q.head + 1) \% Q.length$

$Q.tail = (Q.tail + 1) \% Q.length$



Slovníková metoda LZ77

■ Analýza efektivity

- Metoda nepředpokládá nic o četnosti výskytu jednotlivých symbolů nebo frází, ale implicitně předpokládá, že se vzorky ve vstupních datech vyskytují blízko u sebe.
- Data, která splňují předpoklad se komprimují dobře, avšak pokud se data neustále vysouvají z bufferu, může snadno dojít k expanzi (token $\langle 0,0,'x' \rangle$ je drahý oproti $'x'$).

■ Možná řešení, jak zvýšit efektivitu

- efektivnější kódování tokenů
 - offset proměnné délky
 - zavedení dvou typů tokenů (token s jednou položkou a token s třemi položkami)
- zvětšit velikost obou bufferů (výhoda: podpoří nalézání shody, nevýhoda: nárůst režie z důvodu složitějšího vyhledávání => potřeba promyšlenější datové struktury, která umožňuje rychlejší vyhledávání)

- Na počátku nemáme žádnou historii (vyhledávací buffer je prázdný) a tak jedinou možností je zapsat na výstup token s nulovou shodou.
- Kodér zapíše token a posouvá okno o jeden znak doprava.



Výstup

<0, 0, 'a'>

00000000001100001

$L_{OUT}=17$ bits TOKEN: 5b offset, 4b délka, 8b znak

- Vyhledávací buffer obsahuje jeden symbol, který se shoduje se symbolem v předvídacím bufferu.
- Kodér zapíše na výstup shodu následovanou znakem 'c' a posune okno o dva znaky doprava.



Výstup

<0, 0, 'a'>

<1, 1, 'c'>

- V třetím kroku obsahuje vyhledávací buffer tři symboly, které se shodují s prvními třemi symboly předvídacího bufferu. Shoda však nastává i v dalších 14 symbolech (opakující se sekvence aac).
- Protože však máme omezenou velikost předvídacího bufferu, kodér zapíše shodu délky 15 symbolů a posune okno o 16 znaků doprava.



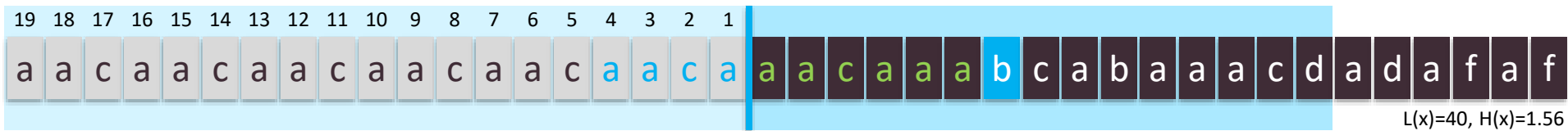
Výstup

<0, 0, 'a'>

<1, 1, 'c'>

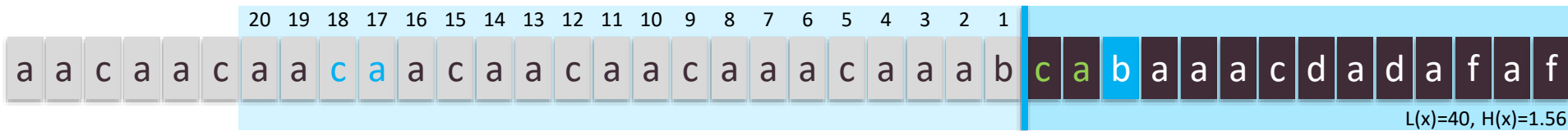
<3, 15, 'a'>

- Kodér postupně prochází prohledávací buffer a nachází shodu na pozici 1, 3, 4, 6, 7, 9, 10, 12, 13, 15, 16, 18 a 19.
- Nejdelší shoda (délky 6) je však na pozici 4. Kodér zapíše token a posune okno o 7 znaků doprava.



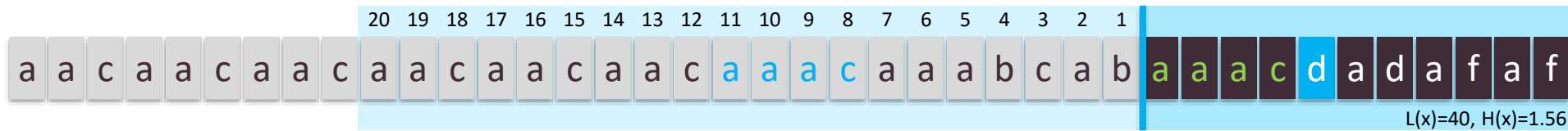
Výstup
<0, 0, 'a'>
<1, 1, 'c'>
<3, 15, 'a'>
<4, 6, 'b'>

- Podobně jako v předchozím případě existuje více shod a sice na pozici 5, 9, 12, 15 a 18. Ve všech případech se jedná o shodu délky 2.
- Kodér zapíše na výstup nejlevější shodu (zjednodušuje implementaci).



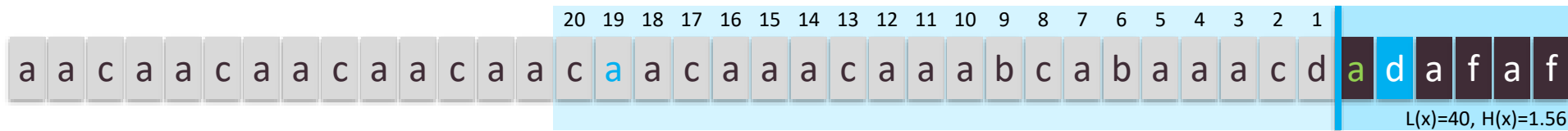
Výstup
<0, 0, 'a'>
<1, 1, 'c'>
<3, 15, 'a'>
<4, 6, 'b'>
<18, 2, 'b'>

- Shoda byla nalezena na všech pozicích prohledávacího bufferu obsahující znak a, avšak nejdelší shoda délky 4 se nachází na pozici 8.
- Kodér zapíše token na výstup a posune okno o 5 znaků doprava.



Výstup
 <0, 0, 'a'>
 <1, 1, 'c'>
 <3, 15, 'a'>
 <4, 6, 'b'>
 <18, 2, 'b'>
 <11, 4, 'd'>

- Symbol 'd' se v prohledávacím bufferu nenachází a tak nejdelší možná shoda je jeden znak.
- Kodér zapisuje nejlevější shodu (pozice 19) a posouvá okno o dva znaky.



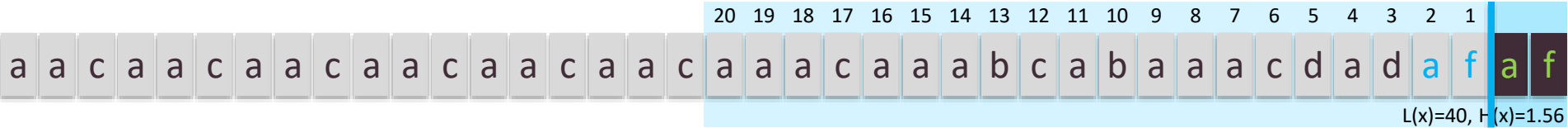
Výstup
 <0, 0, 'a'>
 <1, 1, 'c'>
 <3, 15, 'a'>
 <4, 6, 'b'>
 <18, 2, 'b'>
 <11, 4, 'd'>
 <19, 1, 'd'>

000000000011000010000100010110001100011111101100001001000110011000101001000100110001001011010001100100100100101

100100

L_{OUT}=119 bits

- Kodér končí zpracováním posledních dvou symbolů, které se shodou okolností nachází v prohledávacím bufferu. Třetí pole tokenu je ponecháno prázdné a nezapisuje se na výstup.



Výstup

- <0, 0, 'a'>
- <1, 1, 'c'>
- <3, 15, 'a'>
- <4, 6, 'b'>
- <18, 2, 'b'>
- <11, 4, 'd'>
- <19, 1, 'd'>
- <20, 1, 'f'>

00000000001100001000010001011000110001111110110000100100011001100010100100010011000100101101000110010010011000101
10010010100000101100110000100010

$L_{out}=145$ bits

- Vstupní řetězec byl zakódován pomocí 145 bitů (*3,625 bitů na symbol*). Dosažen *kompresní faktor 0,453* ($145 / 40 \cdot 8$), tj. lepší než v případě Huffmanova kódu (0,459).
- Poznámka: výstup neobsahuje žádnou hlavičku, parametry kodéru (délka prohledávacího bufferu a okna) jsou předány externě.
- Token má pevnou délku: 5b offset, 4b délka, 8b znak

a

a

c

a

a

c

a

a

c

a

a

c

a

a

c

a

a

c

a

a

a

c

a

a

a

b

c

a

b

a

a

a

c

d

a

d

a

f

a

f

L(x)=40, H(x)=1.56



data

00000000001100001000010001011000110001111110110000100100
01100110001010010001001100010010110100011001001001100010
110010010100000101100110000100010

Slovníková metoda LZSS

- LZSS (Storer a Szimanski, 1982) vylepšuje základní variantu LZ77 ve třech směrech
 1. vytváří **značky se dvěma poli** namísto tří
 2. udržuje předvídací buffer v **kruhové frontě**
 3. udržuje vyhledávací buffer v **binárním vyhledávacím stromu** (vytváří slovník)

- Značka LZSS
 - obsahuje pouze **offset a délku**. Nenažde-li se žádná shoda, kodér vyšle nekomprimovaný kód následujícího textového symbolu.
 - aby bylo možné značky odlišit od nekomprimovaných kódů, předchází před nimi jeden **bit** (coded/uncoded flag).
 - v případě, kdy je nekomprimovaný kód kratší než-li token (shoda 1 znak) se zapíše na výstup nekomprimovaný kód

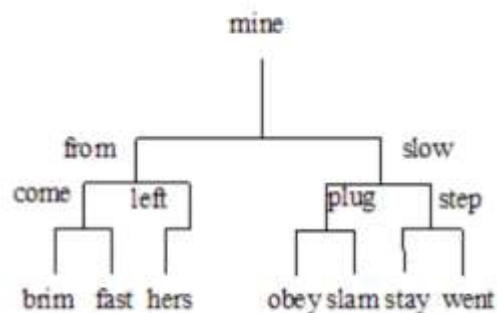
Slovníková metoda LZSS – značka

- **Velikost předvídacího bufferu se volí tak, aby velikost značky byla 16 bitů.**
 - Např. je-li velikost vyhledávacího bufferu 2 KB, pak musí být předvídací buffer dlouhý 32 bajtů. Pole offsetu bude dlouhé 11 bitů, a pole délky 5 bitů (odpovídá velikosti předvídacího bufferu).
 - Kodér pak bude vysílat buď 2-bajtové značky, nebo nekomprimované 1-bajtové kódy ASCII.

- **Možnost vypořádání se s jednobitovými flagy**
 - shromáždit osm vstupních položek (značek nebo kódů ASCII) v malém vstupním bufferu, následně vyslat jeden bajt obsahující osm flagů, následovaný osmi položkami (kdy je každá dlouhá 1 nebo 2 bajty).

Vyhledávání pomocí binárního stromu

- K zrychlení procesu vyhledávání je možné využít binární vyhledávací strom (BST)
 - BST je binární strom, ve kterém levý podstrom každého uzlu A obsahuje uzly lexikograficky menší než A, a pravý podstrom obsahuje uzly lexikograficky větší než A.
 - V našem případě uzly musí obsahovat řetězce. Využijeme toho, že řetězce se nacházejí ve slovníku, kde jsou seřazeny abecedně podle prvního písmena, při konstrukci a prohledávání stromu tedy lze využívat lexikografické pořadí (seřazení řetězců lexikograficky dle ASCII hodnot)
- Příklad relací:
 - “rote” < “said” (r je v abecedě před s), „□abc“ < “ab” (mezera je před a)
- V binárním stromu lze najít jakýkoliv uzel během maximálně k kroků, kde k je výška stromu. Ve vyváženém stromu platí, že $k = \lceil \log_2 n \rceil$, kde n je počet uzlů.



vyvážený BST

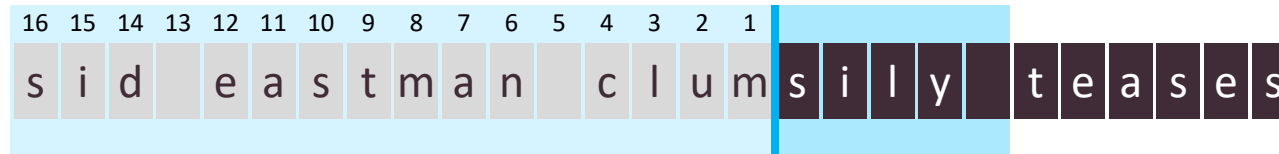


nevyvážený BST

Vyhledávání pomocí binárního stromu

■ Princip využití BST k urychlení hledání ve slovníku:

- Pro jednoduchost předpokládejme vyhledávací buffer o velikosti $S = 16$ symbolů a předvídací buffer o velikosti $L = 5$ symbolů a následující výchozí situaci



řetězce 5 znaků

sid□e	16
id□ea	15
d□eas	14
□east	13
eastm	12
astma	11
stman	10
tman□	9
man□c	8
an□cl	7
n□clu	6
□clum	5

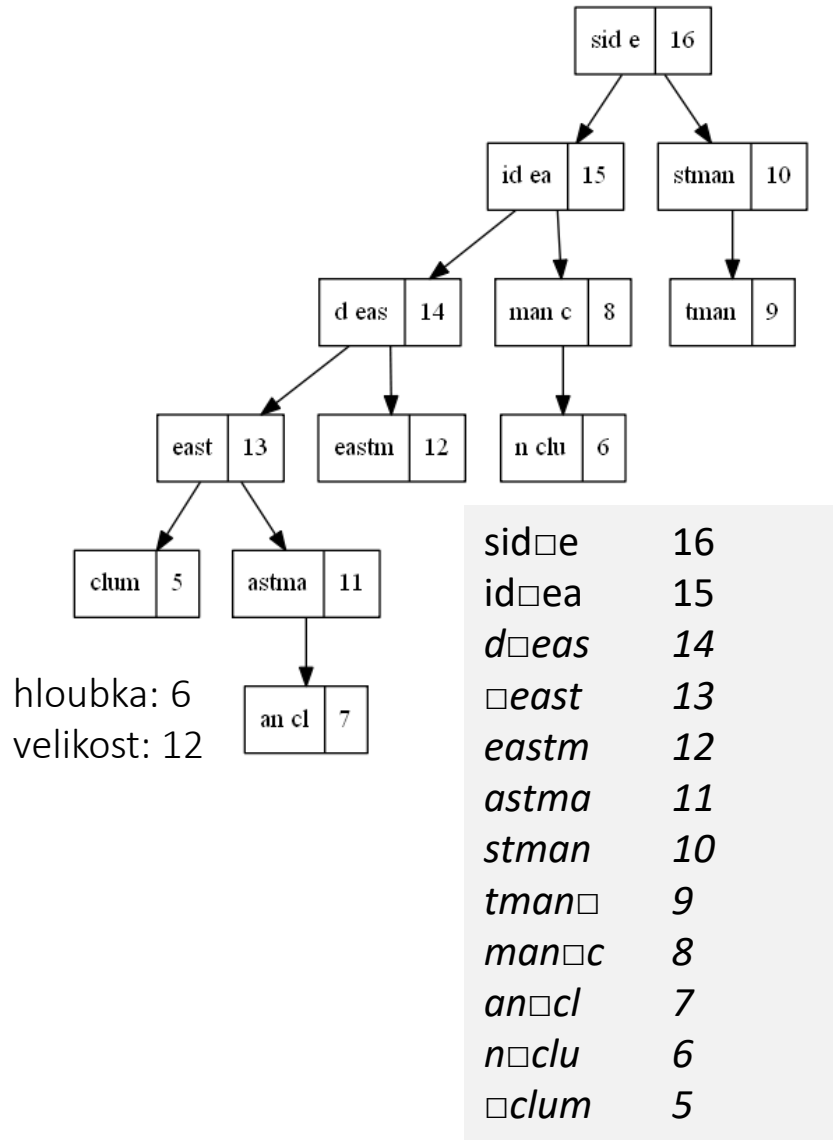
- Kodér projde vyhledávací buffer a vytvoří všechny možné L-znakové řetězce nacházející se ve vyhledávacím bufferu a vloží je do BST
- První symbol v předvídacím bufferu je **s**, tak kodér vyhledává ve stromu řetězce, které začínají na **s**. Najdou se dva, s offsetem 16 a 10, a první z nich dává delší shodu. Shoda má délku $k = 2$ a vyše se značka (16, 2).
- Kodér musí okno posunout o dvě místa doprava a aktualizovat strom.
- Strom se aktualizuje odstraněním řetězců sid□e a id□ea, a vložením nových řetězců clums a lumsi.

Vyhledávání pomocí binárního stromu

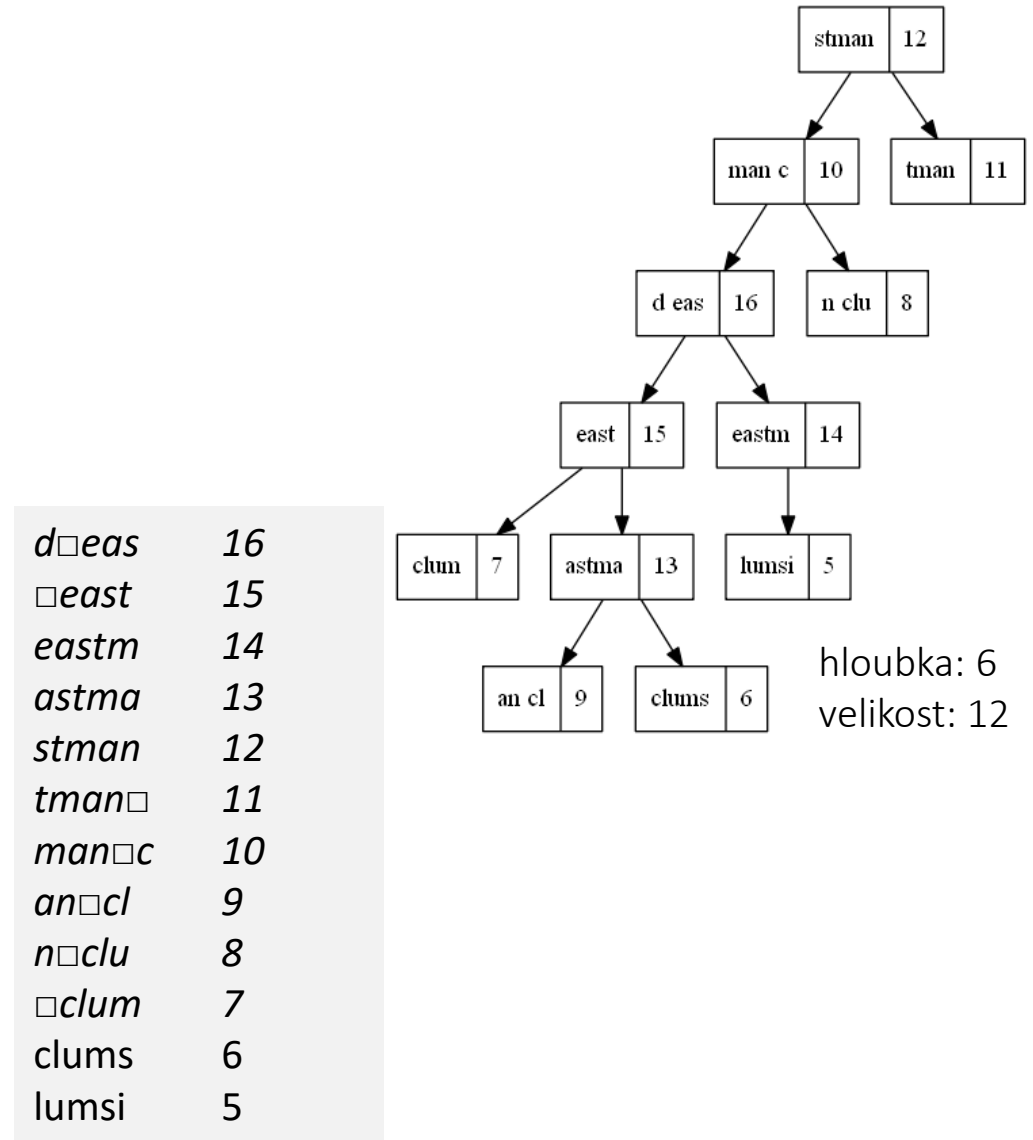
- Pokud se najde shoda délky k symbolů, musí se okno posunout o k míst, a strom se musí aktualizovat vymazáním k řetězců a přidáním k nových řetězců.
 - k řetězců, které se musí vymazat, jsou ty první ve vyhledávacím bufferu, ale ještě před posuvem,
 - k řetězců, které se mají přidat, jsou ty poslední v bufferu, ale až po posuvu.
- Princip aktualizace stromu
 - aktualizovat indexy (přičíst hodnotu k ke všem uzlům)
 - připravit řetězec, sestávající z prvních L symbolů vyhledávacího bufferu, najít jej ve stromě a vymazat ho
 - posunout buffer o jednu pozici doprava (nebo posunout data doleva), připravit řetězec, tvořený posledními L symboly vyhledávacího bufferu, a připojit ho ke stromu se správným indexem
 - poslední dva uvedené kroky k krát opakovat
- Vlastnosti
 - Velikost stromu se nikdy nemění (odmazává se a přidává stejný počet řetězců), vždy obsahuje $S - L + 1$ řetězců.
 - Tvar stromu se však může výrazně měnit – od plně nevyváženého (pro vyhledávání nejhorší případ) až po plně vyvážený (pro vyhledávání ideální).

Vyhledávání pomocí binárního stromu – příklad

Tvar stromu před aktualizací



Tvar stromu po aktualizaci



LZSS - krok 1

- Na počátku nemáme žádnou historii (vyhledávací buffer je prázdný) a tak jedinou možností je zapsat na výstup **nekomprimovaný znak**.
- Kodér zapíše token $\langle 0, 'a' \rangle$ (flag 0 následovaný znakem 'a') a posouvá okno o jeden znak doprava.



Výstup

$\langle 0, 'a' \rangle$

001100001

$L_{OUT}=9$ bits TOKEN: 1b flag 8b znak nebo 1b flag, 5b offset, 4b délka

- Symbol 'c' se v prohledávacím bufferu nenachází, na výstup se tedy zapíše nekomprimovaný znak
- Kodér zapíše token <0,'c'> (flag 0 následovaný znakem 'c') a posouvá okno o jeden znak doprava.



$L(x)=40$, $H(x)=1.56$

Výstup

<0, 'a'>

<0, 'a'>

<0, 'c'>

001100001001100001001100011

$L_{OUT}=27$ bits

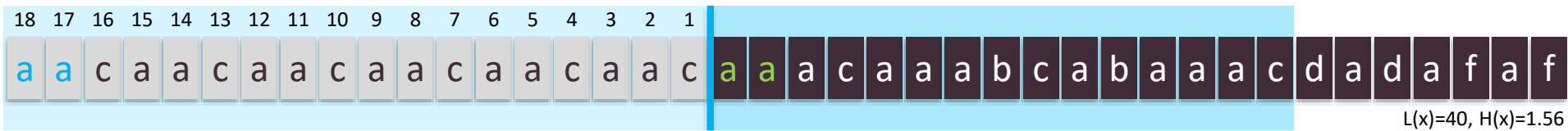
- Ve čtvrtém kroku obsahuje prohledávací buffer tři symboly, které se shodují s prvními třemi symboly předvídacího bufferu. Shoda však nastává i v dalších 17 symbolech (opakující se sekvence aac). Protože však máme omezenou velikost předvídacího bufferu, kodér zapíše shodu délky 15 symbolů (flag 1 následovaný dvojicí 3, 15) a posune okno o 15 znaků doprava.



Výstup
 <0, 'a'>
 <0, 'a'>
 <0, 'c'>
 <1, 3, 15>

LZSS - krok 5

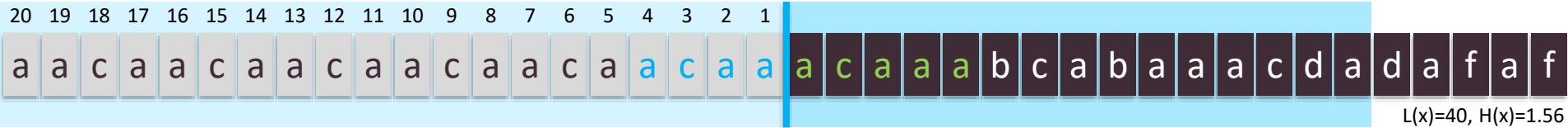
- Kodér prochází binární vyhledávací strom a nachází shody na pozici 2, 3, 5, 6, ..., 17 a 18. Ve několika případech se jedná o shodu délky 2.
- Kodér zapíše na výstup nejlevější shodu (zjednodušuje implementaci).



Výstup
<0, 'a'>
<0, 'a'>
<0, 'c'>
<1, 3, 15>
<1, 18, 2>

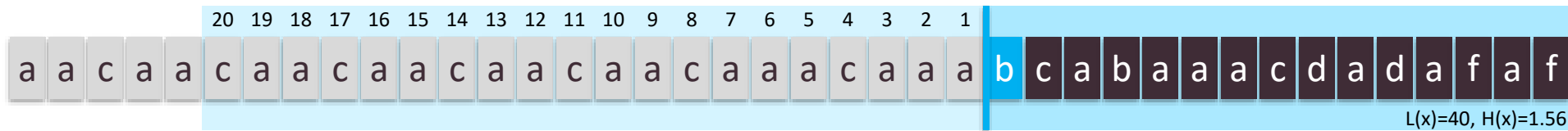
LZSS - krok 6

- Opět existuje více shod. Nejdelší shoda je však na pozici 1, neboť lze využít přesahu do předvídacího bufferu.
- Kodér zapíše shodu na pozici 4 délky 5 a posouvá okno o 5 pozic doprava.



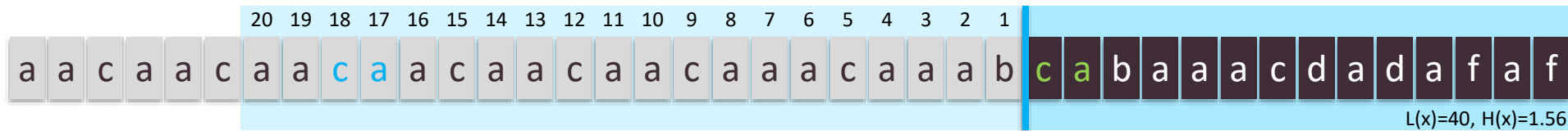
Výstup
<0, 'a'>
<0, 'a'>
<0, 'c'>
<1, 3, 15>
<1, 18, 2>
<1, 4, 5>

- Symbol 'b' se v prohledávacím bufferu nenachází a na výstup se tedy zapíše nekomprimovaný znak
- Kodér zapíše token <0,'b'> (flag 0 následovaný znakem 'b') a posouvá okno o jeden znak doprava.



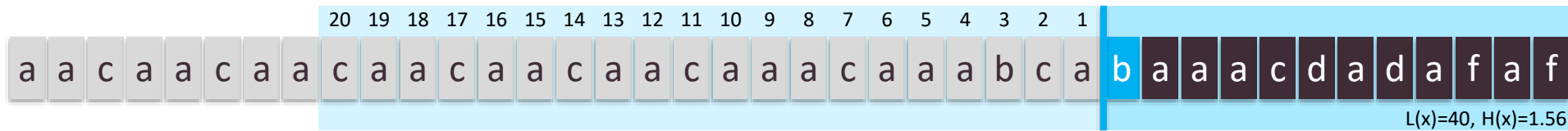
Výstup
<0, 'a'>
<0, 'a'>
<0, 'c'>
<1, 3, 15>
<1, 18, 2>
<1, 4, 5>
<0, 'b'>

- Kodér nalezne v binárním vyhledávacím stromu shodu délky 2 na pozici 5, 9, 12, 15 s 18. Vybere nejlevější shodu a zapíše na výstup.
- Okno se posune o dva znaky doprava.



Výstup
<0, 'a'>
<0, 'a'>
<0, 'c'>
<1, 3, 15>
<1, 18, 2>
<1, 4, 5>
<0, 'b'>
<1, 18, 2>

- Nyní kodér nalezne shodu na pozici 3 délky 1, avšak token by byl delší, než-li jeden byte a tak se z důvodu **optimalizace** zapíše nekomprimovaný znak.
- Kodér zapíše token <0,'b'> (flag 0 následovaný znakov 'b') a posouvá okno o jeden znak doprava.



Výstup

- <0, 'a'>

<0, 'a'>

<0, 'c'>

<1, 3, 15>

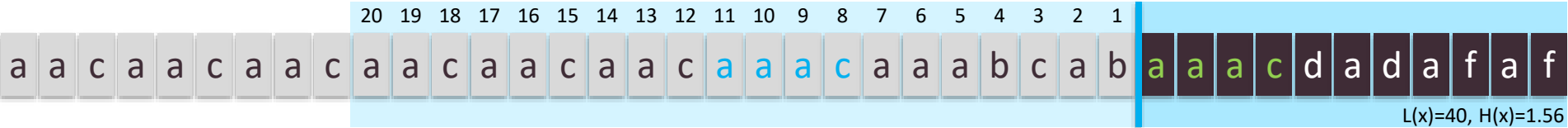
<1, 18, 2>

<1, 4, 5>

<0, 'b'>

<1, 18, 2>
- <0, 'b'>

- Nejdelší shoda je na pozici 11.

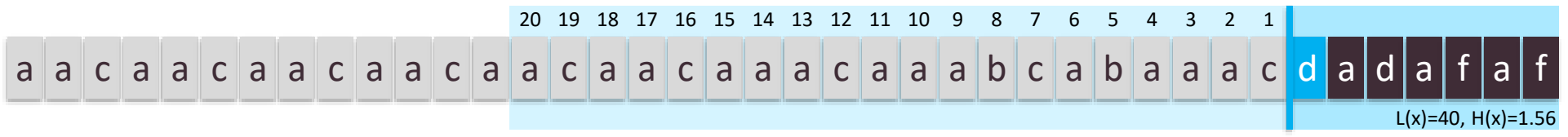


Výstup

- | | |
|------------|------------|
| <0, 'a'> | <0, 'b'> |
| <0, 'a'> | <1, 11, 4> |
| <0, 'c'> | |
| <1, 3, 15> | |
| <1, 18, 2> | |
| <1, 4, 5> | |
| <0, 'b'> | |
| <1, 18, 2> | |

LZSS - krok 11

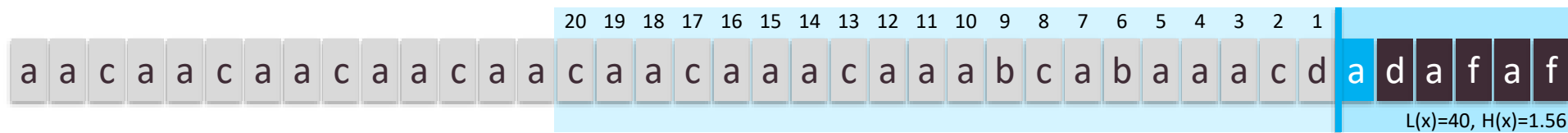
- Symbol 'd' se v prohledávacím bufferu nenachází a na výstup se tedy запиše nekomprimovaný znak
- Kodér запиše token <0,'d'> a posouvá okno o jeden znak doprava.



Výstup

- | | |
|------------|------------|
| <0, 'a'> | <0, 'b'> |
| <0, 'a'> | <1, 11, 4> |
| <0, 'c'> | <0, 'd'> |
| <1, 3, 15> | |
| <1, 18, 2> | |
| <1, 4, 5> | |
| <0, 'b'> | |
| <1, 18, 2> | |

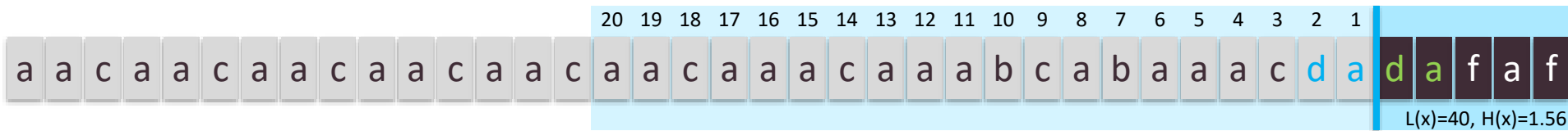
- Nyní kodér nalezne shodu na několik a pozicích, avšak pouze délky 1. Z důvodu **optimalizace** tedy zapíše nekomprimovaný znak.
- Kodér zapíše token $\langle 0, 'a' \rangle$ a posouvá okno o jeden znak doprava.



Výstup

- | | |
|----------------------------|----------------------------|
| $\langle 0, 'a' \rangle$ | $\langle 0, 'b' \rangle$ |
| $\langle 0, 'a' \rangle$ | $\langle 1, 11, 4 \rangle$ |
| $\langle 0, 'c' \rangle$ | $\langle 0, 'd' \rangle$ |
| $\langle 1, 3, 15 \rangle$ | $\langle 0, 'a' \rangle$ |
| $\langle 1, 18, 2 \rangle$ | |
| $\langle 1, 4, 5 \rangle$ | |
| $\langle 0, 'b' \rangle$ | |
| $\langle 1, 18, 2 \rangle$ | |

- V binárním vyhledávacím stromu je jediná shoda a sice shoda na pozici 2 délky 2.
- Kodér zapíše příslušný token a posouvá okno o dvě pozice doprava.



Výstup

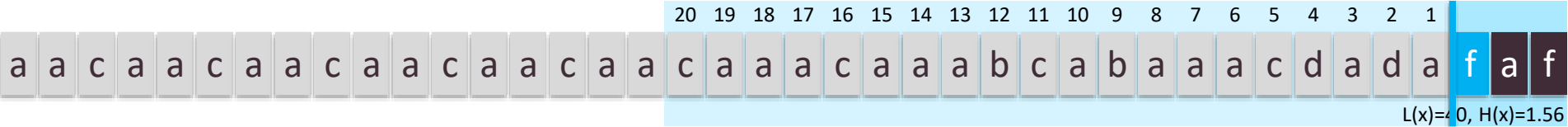
- | | |
|------------|------------|
| <0, 'a'> | <0, 'b'> |
| <0, 'a'> | <1, 11, 4> |
| <0, 'c'> | <0, 'd'> |
| <1, 3, 15> | <0, 'a'> |
| <1, 18, 2> | <1, 2, 2> |
| <1, 4, 5> | |
| <0, 'b'> | |
| <1, 18, 2> | |

00110000100110000100110001110001111111100100010100100010100110001011001000100011000101010110100001100100001100001

1000100010

$L_{out}=123$ bits

- Symbol 'f' se v prohledávacím bufferu nenachází a na výstup se tedy zapíše nekomprimovaný znak
- Kodér zapíše token <0,'f'> a posouvá okno o jeden znak doprava.



Výstup

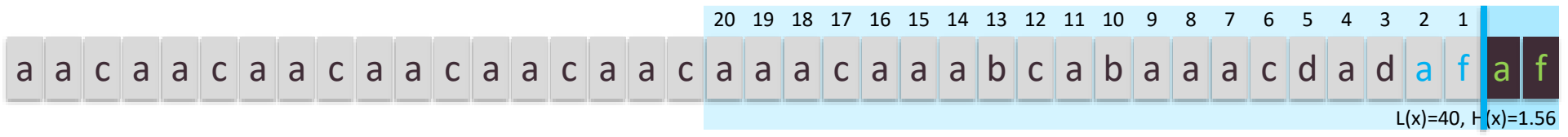
<0, 'a'> <0, 'b'>
<0, 'a'> <1, 11, 4>
<0, 'c'> <0, 'd'>
<1, 3, 15> <0, 'a'>
<1, 18, 2> <1, 2, 2>
<1, 4, 5> <0, 'f'>
<0, 'b'>
<1, 18, 2>

00110000100110000100110001110001111111100100010100100010100110001011001000100011000101010110100001100100001100001

1000100010001100110

L_{OUT}=132 bits

- Kodér končí svou činnost nalezením shody na pozici 2 o délce dva znaky. Zapiše tedy příslušný token na výstup.



Výstup

- | | |
|------------|------------|
| <0, 'a'> | <0, 'b'> |
| <0, 'a'> | <1, 11, 4> |
| <0, 'c'> | <0, 'd'> |
| <1, 3, 15> | <0, 'a'> |
| <1, 18, 2> | <1, 2, 2> |
| <1, 4, 5> | <0, 'f'> |
| <0, 'b'> | <1, 2, 2> |
| <1, 18, 2> | |

00110000100110000100110001110001111111100100010100100010100110001011001000100011000101010110100001100100001100001

10001000100011001101000100010

$L_{out}=142$ bits

- Vstupní řetězec byl zakódován pomocí 142 bitů (3,55 bitů na symbol). Dosažen kompresní faktor 0,443 (142 / 40*8), tj. lepší než v případě Huffmanova kódu (0,459) i původního LZ77 (0,453).
- Poznámka: výstup neobsahuje žádnou hlavičku, parametry kodéru (délka prohledávacího bufferu a okna) jsou předány externě.
- Token má proměnnou délku: 1b flag + 5b offset + 4b délka, nebo 1b flag + 8b znak

a a c a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=1.56



data

```
00110000100110000100110001110001111111100100010100100010
10011000101100100010001100010101011010000110010000110000
110001000100011001101000100010
```

Nedostatky metody LZ77 a jejích variant

- LZ77 využívá vnitřního implicitního předpokladu, že vzorky se ve vstupních datech vyskytují blízko u sebe. Datové toky, které tento předpoklad nesplňují, se komprimují špatně.
 - Př: text, ve kterém se často vyskytuje určité slovo, např. „ekonomika“, ale je v textu rovnoměrně rozmístěno. Když se toto slovo nasune do předvídacího bufferu, jeho předchozí výskyt může být již vysunut z vyhledávacího bufferu ven.
 - Lepší algoritmus by si měl *ukládat často se vyskytující řetězce do slovníku*, a nevysouvat je neustále ven.
- *Velikost srovnávaných řetězců je omezená na $L - 1$* , avšak L se musí udržovat malé, protože proces vyhledávání shody řetězců zahrnuje srovnávání jednotlivých symbolů.
 - Kdyby se velikost L zdvojnásobila, komprese by se zlepšila, protože by se tím umožnily delší shody, ale kodér by byl značně pomalejší, protože by vyhledával dlouhé shody.
- Velký vyhledávací buffer umožňuje lepší kompresi, ale kodér se tím zpomaluje, protože vyhledávání trvá déle (i s binárním vyhledávacím stromem). Zvětšit velikosti obou bufferů rovněž znamená vytvářet delší značky, čímž se snižuje účinnost komprese - viz **zlomový bod** (při jisté délce shody je výhodnější zakódovat nekomprimovaně než-li použít token, který vede na expanzi)

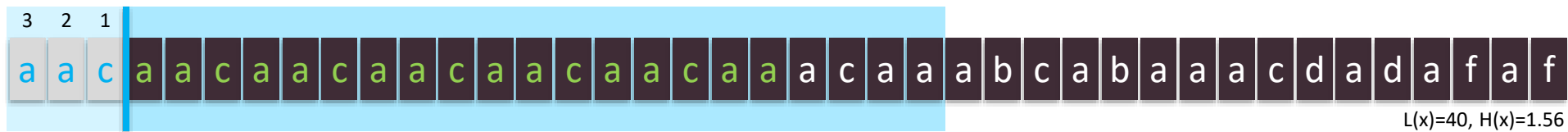
Další varianty LZ77 (informativní)

- V praxi se můžeme setkat s řadou variant LZ77, mezi nejčastěji používané patří
- **LZO** (Lempel–Ziv–Oberhumer, 1996)
 - optimalizace zaměřená na rychlost komprese a dekomprese
 - mírně horší kompresní poměr než zlib
 - kompresní nástroj LZOP
- **LZ4** (Collet, 2011, lz4.github.io/lz4)
 - nástupce LZO, optimalizace zaměřená na rychlost komprese (HASH tabulky pro vyhledávání) a dekomprese
 - výstupní tok sestává z tzv. sekvencí složených z **tokenu** (definuje délku shody minus 4, a počet literálů), **literálů** (nekomprimovaných znaků) a **offsetu** (0-65535)
 - počet literálů není omezen (nárůst pro nekomprimovatelná data pouze 0.4%)
 - použito např. v : ZFS, MySQL, Redis, openVPN, PHP Zend, Hadoop, atd.

Token : ==> 4-high-bits : literal length / 4-low-bits : match length

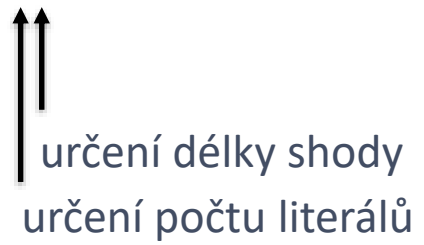
Token	<i>Literal length+ (optional)</i>	Literals	Offset	<i>Match length+ (optional)</i>
1-byte	0-n bytes	0-L bytes	2-bytes	0-n bytes

- V prvním, druhém a třetím kroku metoda LZ4 nevygeneruje žádný token, neboť se nedaří nalézt shodu minimální délky, tj. 4 znaky.
- Až ve čtvrtém kroku se podaří nalézt shodu a to délky 17 znaků.



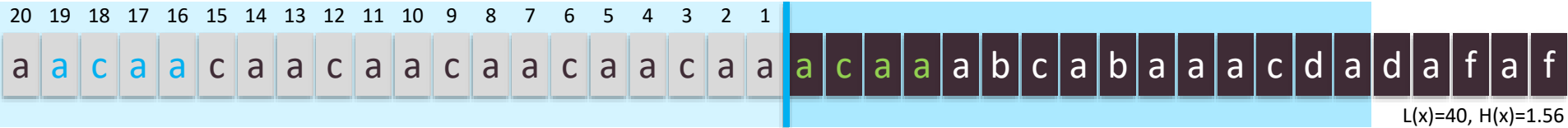
Výstup

<0x3d, 'aac', 0x0003> dekodér nakopíruje aac a poté udělá 17 kopií znaků z offsetu 3



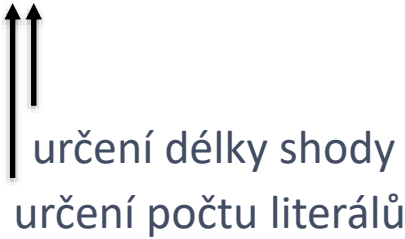
Metoda LZ4

- V dalším kroku postupujeme obdobně jako v případě LZ77. Nejdelší shoda je shoda délky 4 na offsetu 4, 7, 10, 13, 16, 19. Kodér vybere poslední shodu, tj. offset 19.



Výstup

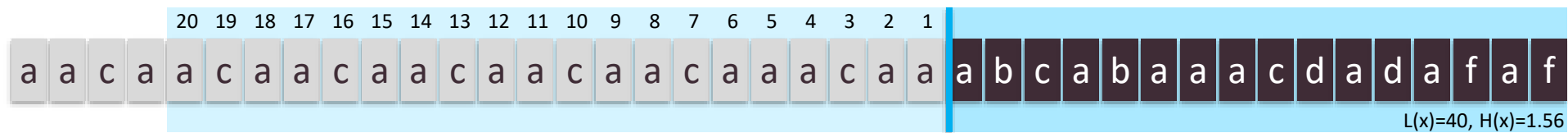
<0x3d, 'aac', 0x0003> dekodér nakopíruje aac a poté udělá 17 kopií znaků z offsetu 3
<0x00, 0x0013> dekodér udělá 4 kopie z offsetu 19



00111101 01100001 01100001 01100011 00000011 00000000 00000000 00010011
L_{OUT}= 64 bits

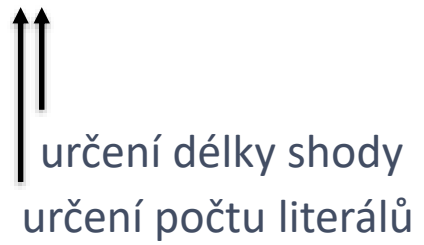
Metoda LZ4

- Máme zpracováno 24 znaků vstupní sekvence. V dalších sedmi krocích se však nedaří nalézt žádnou shodu minimální délky 4.
- Protože narazíme na limit, vygeneruje se proto přímo literál složený z 16ti znaků, neboť by se nevyplatilo kódovat předchozí krátké sekvence.



Výstup

<0x3d, 'aac', 0x0003> dekodér nakopíruje aac a poté udělá 17 kopií znaků z offsetu 3
<0x00, 0x0013> dekodér udělá 4 kopie z offsetu 19
<0xF0, 0x01, 'abcabaaacdadafaf'> dekodér pouze nakopíruje řetězec 16ti znaků



Diferencování souborů (Delta encoding)

- Diferencování souborů označuje jakoukoliv metodu, která lokalizuje difference mezi dvěma soubory a ty následně komprimuje.
 - Představme si soubor A se dvěma kopiemi, které udržují dva uživatelé. Když jeden uživatel kopii upraví, musí ji poslat druhému uživateli, aby se udržela identičnost kopií. Místo posílání kopie A, která může být velká, se může poslat mnohem menší soubor, obsahující v komprimované formě pouze rozdíly, který pak příjemce použije k aktualizaci své kopie A.
- Smyslem je použít oba soubory k zakódování diferencí mezi nimi v komprimované formě tak, aby kdokoliv, kdo má zdrojový soubor a zakódované difference byl schopen rekonstruovat cílový soubor.
- Princip diferencování souborů:
 - Cílový soubor se připojí ke zdrojovému souboru, zkombinovaný soubor se prožene přes LZ77 nebo obdobnou metodu a jakmile se narazí na cílový soubor (začátek cílového souboru dostane na hranici mezi předvídacím bufferem a vyhledávacím bufferem), začne se zapisovat komprimovaný soubor.
- **Komprese je zvláštním případem diferencování souborů.**
 - Neexistuje-li zdrojový soubor, pak se diferencování redukuje na LZ77 kompresi cílového souboru.

Diferencování souborů - VCDIFF

- Jednou z metod používaných pro diferencování je metoda označovaná jako VCDIFF, která se používá pro tzv. delta encoding v HTTP (podporováno v Google Chrome).
- Autoři VCDIFF navrhují variantu původního algoritmu LZ77, který komprimuje difference mezi zdrojovým a cílovým souborem, a vytváří komprimovaný „delta“ soubor pomocí tří typů instrukcí:
 - ADD (adds a new sequence),
 - RUN (adds repeated data),
 - COPY (copies from an old sequence).
- **Kodér je obecně komplikovanější než dekodér.**
 - Úlohou kodéru je prohlížet spojené soubory (buffer U), nacházet shody mezi zdrojovými a cílovými řetězci, a vytvářet delta soubor s delta instrukcemi.
 - Jakýkoliv kodér, který vytváří platný delta soubor se považuje za kodér pracující v souladu (compliant) s definicí VCDIFF.

Diferencování souborů - VCDIFF

- Dekodér VCDIFF používá instrukce pro generování cílového souboru takto:
 - **ADD** má dva argumenty: **délku** x a **posloupnost** x bytů.
Dekodér tyto byty připojí ke generovanému cílovému souboru.
 - **RUN** má dva argumenty: **délku** x a jediný **byte** b .
Dekodér přidá k cílovému souboru x výskytů bytu b .
 - **COPY** má rovněž dva argumenty, **délku** x a **index** p do bufferu U .
Dekodér lokalizuje podřetězec délky x v U na indexu p , a připojí k cílovému souboru kopii toho podřetězce. Index p může ukazovat i do cílového souboru – viz následující příklad. Délka tak může překračovat aktuální velikost bufferu U . Symboly se do bufferu přidávají postupně znak po znaku.

Diferencování souborů - VCDIFF

- Příklad: zdrojový soubor obsahující 16 znaků, cílový soubor 28 znaků

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	a	b	c	d	w	x	y	z	e	f	g	h	e	f	g	h	e	f	g	h	e	f	g	h	z	z	z	z
zdrojový soubor																cílový soubor																											

- Diferenci mezi soubory lze vyjádřit pěti delta instrukcemi:

– COPY 4,0

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	a	b	c	d

– ADD 4,w x y z

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	a	b	c	d	w	x	y	z

– COPY 4,4

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	a	b	c	d	w	x	y	z	e	f	g	h

– COPY 12,24 (instrukce kopíruje data z vytvářeného cílového souboru – offset 24)

– RUN 4,z

Slovníková metoda LZ78

- Metoda LZ78 (Lempel Ziv, 1978) nepoužívá pohyblivé okno (tzn. ani vyhledávací buffer, ani předvídací buffer), ale **udrží si slovník dříve nalezených řetězců**, který obsahuje fragmenty nekomprimovaného vstupního řetězce.
- **Formát tokenu**
 - Kodér vysílá **značky se dvěma poli** tvaru $\langle \text{index}, 'x' \rangle$, kde pole index je ukazatel do slovníku a kóduje shodu a 'x' odpovídá kódu symbolu nenalezeného ve slovníku.
 - Značky neobsahují délku řetězce, protože ta vyplývá ze slovníku.
- **Slovník řetězců**
 - Slovník začíná s jednou položkou na indexu 0 a sice prázdným řetězcem (null) kvůli možnosti zakódovat shodu délky 0.
 - Velikost slovníku je obecně omezena pouze velikostí poskytnuté paměti.
 - Slovník se aktualizuje vždy, když kodér zapíše token do komprimovaného toku.
 - Aktualizace slovníku spočívá v přidání řetězce, který kóduje token, do slovníku.



Slovníková metoda LZ78

■ Princip činnosti kódéru:

- Slovník začíná s prázdným řetězcem v pozici nula. Jak symboly vstupují a kódují se, řetězce se přidávají do slovníku na pozice 1, 2, a tak dále.
- Kóder čte ze vstupního toku postupně symboly a přidává je k řetězci, který doposud kódér viděl a to tak dlouho, než se řetězec nenalezne ve slovníku. V tu chvíli kódér pošle na výstup značku kódující index nejdelší shody a symbolu, který způsobil neshodu. Současně se přidá řetězec do slovníku na první volnou pozici.

■ Příklad

- Předpokládejme, že se ze vstupního toku přečte symbol 'x'. Tzn. vyhledá se ve slovníku položka s jednosymbolovým řetězcem "x".
- Když se řetězec "x" nenajde, přidá se "x" na první volnou pozici slovníku a na výstup se pošle značka (0, 'x') indikující spojení prázdného řetězce a 'x'.
- Pokud se řetězec "x" najde (např. na pozici 41), načte se následující symbol 'y', a ve slovníku se vyhledává dvousymbolový řetězec "xy". Když se takový nenajde, tak se řetězec "xy" přidá na první volnou pozici slovníku a vyšle se značka (41, 'y'). Pokud se takový řetězec najde, pokračuje se dalším symbolem.



Slovníková metoda LZ78

■ Kodér (pseudokód)

```
inicializace slovníku D = [null], F = "", i = 0
while (nepřečten celý vstupní tok) do
  begin
    načti symbol X
    if F+X ∈ D then
      přidej X k řetězci se shodou F = F + X a poznač si pozici i = pozice F v D
    else
      ulož značku <i, X>
      přidej F+X do slovníku, F = "", i = 0
    end
  end
```


Slovníková metoda LZ78

- Dekodér je jednodušší, než kodér (asymetrická kompresní metoda).
 - Čte tokeny vstupní délky a s každým načtením vygeneruje výstup a aktualizuje slovník.

inicializace slovníku $D = [\text{null}]$

while (nepřečten celý vstupní tok) **do**

begin

načti značku $\langle i, X \rangle$

přidej do slovníku řetězec $F = D[i] + X$ a zapiš do výstupního toku řetězec F

end

Efektivní vyhledávání položek ve slovníku

■ Implementace slovníku

- Velký slovník může obsahovat víc řetězců a tak dovoluje delší shody, avšak režií jsou delší ukazatele (tzn. delší značky) a pomalejší vyhledávání ve slovníku.
- Vhodná struktura pro slovník je **n-ární strom** ($n=256$ pro 8-bitové symboly).

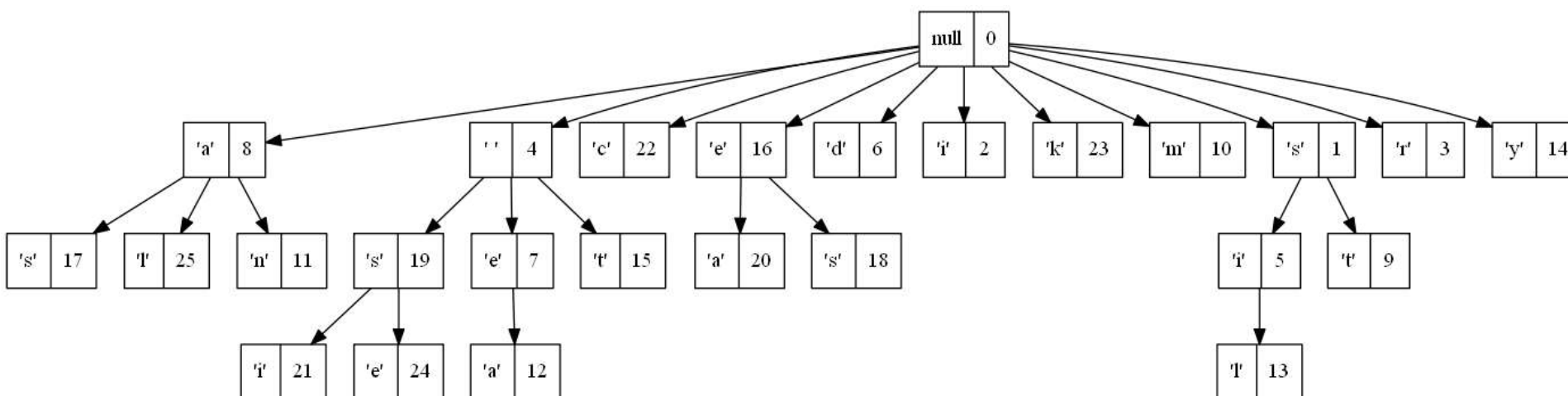
■ Vyhledávací n-ární strom (trie)

- Datová struktura pro uchovávání dvojic **klíč-hodnota**, kde klíči jsou obvykle řetězce (struktura větvení je určena pouze částí datové položky a ne celou položkou)
- Každý uzel stromu obsahuje **dvojici** (symbol, index). Vyhledávání se děje na základě znalosti symbolu. Je-li nalezena shoda, výstupem je index udávající pořadí řetězce ve slovníku.
- Kořen reprezentuje prázdný řetězec na indexu 0.
- Potomci uzlu odpovídají všem podřetězcům, kterými může pokračovat řetězec v dosud prohledané cestě.
 - Všechny řetězce, které začínají prázdným řetězcem se přidávají ke stromu jako potomci kořene.
 - V případě metody LZ78 se přidává k řetězci pouze jeden symbol, což se provede přidáním jedné větve.

Efektivní vyhledávání položek ve slovníku

- Příklad slovníku reprezentovaného pomocí stromu obsahujícího 26 položek, který postupně vznikl během komprese řetězce

s i r s i d e a s t m a n e a s i l y t e a s e s s e a s i c k s e a l s **s i l y**



Index	Řetězec	Index	Řetězec	Index	Řetězec	Index	Řetězec
0	null	8	"a"	16	"e"	24	" se"
1	"s"	9	"st"	17	"as"	25	"al"
2	"i"	10	"m"	18	"es"		
3	"r"	11	"an"	19	" s"		
4	" "	12	" ea"	20	"ea"		
5	"si"	13	"sil"	21	" si"		
6	"d"	14	"y"	22	"c"		
7	" e"	15	" t"	23	"k"		

Efektivní vyhledávání položek ve slovníku

- Celková výška stromu je typicky omezená a může dojít během komprese k zaplnění.
- Metoda LZ78 nespecifikuje, co v dělat v případě zaplnění stromu. Existuje několik možných řešení:
 1. Zmrazit slovník – žádné nové uzly se nesmějí přidat, strom se stane statickým slovníkem.
 2. Provést restart – jakmile se strom zaplní, strom se vyprázdní a začne se s novým stromem obsahujícím jedinou položku (index 0).
 3. Odstranit jednu či více nejdéle nepoužitých položek, čímž vznikne prostor pro nové (nejobtížnější).
- **Komentář k možnosti 2**
 - Řešení efektivně rozseká vstup na bloky, každý s vlastním slovníkem. Když se vstup blok od bloku mění, poskytne toto řešení dobrou kompresi, protože eliminuje slovník s řetězci, jejichž použití je v budoucnosti nepravděpodobné.
 - Implicitně se předpokládá, že budoucí symboly budou využívat více nových dat, než starých (stejný předpoklad, jako u LZ77).
- **Metoda compress kombinuje možnost 1 a 2.**
 - Jakmile se slovník stane statickým, monitoruje se kompresní poměr a pokud poklesne pod určitou mez, provede se restart.

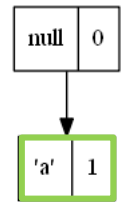
Komprese pomocí LZ78 - krok 1

- Řetězec "a" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 1.
- Kodér zapíše do výstupního toku značku <0,'a'> a posune se o jeden znak doprava.



Výstup
<0,'a'>

Slovník
0: null
1 : 'a'



000000000 01100001
L_{OUT}=17 bits

Uvažujeme slovník s max. 512 položkami.

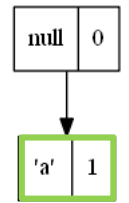
Komprese pomocí LZ78 - krok 2

- Řetězec "a" se nachází ve slovníku na pozici 1.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup
<0,'a'>

Slovník
0: null
1: 'a'



000000000001100001
L_{OUT}=17 bits

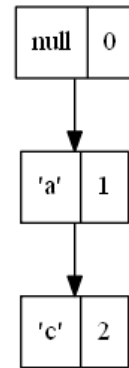
Komprese pomocí LZ78 - krok 3

- Řetězec "ac" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 2.
- Kodér zapíše do výstupního toku značku <1,'c'> a posune se o jeden znak doprava.



Výstup
<0,'a'>
<1,'c'>

Slovník
0: null
1 : 'a'
2 : 'ac'



00000000000110000100000000101100011
 $L_{OUT}=34$ bits

PŘÍKLAD

-


$$L(x)=40, H(x)=1.56$$

Výstup

$\langle 0, 'a' \rangle$

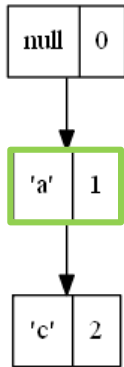
<1, 'c'>

Slovník

0: null

1: 'a'

2 : 'ac'



000000000110000100000000101100011

$$L_{OUT}=34 \text{ bits}$$

Komprese pomocí LZ78 - krok 5

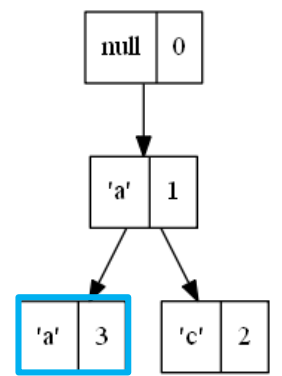
- Řetězec "aa" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 3.
- Kodér zapíše do výstupního toku značku <1,'a'> a posune se o jeden znak doprava.



$L(x)=40, H(x)=1.56$

Výstup
<0,'a'>
<1,'c'>
<1,'a'>

Slovník
0: null
1 : 'a'
2 : 'ac'
3 : 'aa'



0000000000011000010000000010110001100000000101100001
 $L_{out}=51$ bits

PŘÍKLAD

Komprese pomocí LZ78 - krok 6

- Řetězec "c" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 4.
- Kodér zapíše do výstupního toku značku <0,'c'> a posune se o jeden znak doprava.



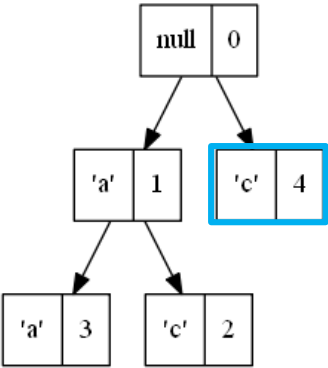
L(x)=40, H(x)=1.56

Výstup

<0,'a'>
<1,'c'>
<1,'a'>
<0,'c'>

Slovník

0: null
1 : 'a'
2 : 'ac'
3 : 'aa'
4 : 'c'



00000000000110000100000000101100011000000001011000010000000001100011
L_{OUT}=68 bits

PŘÍKLAD

Komprese pomocí LZ78 - krok 7

- Řetězec "a" se nachází ve slovníku na pozici 1.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.

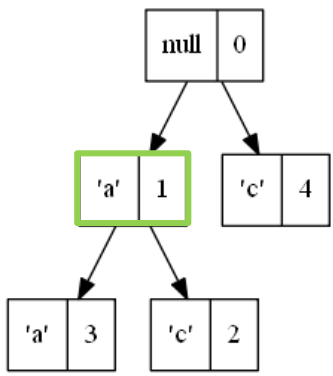


Výstup

<0,'a'>
<1,'c'>
<1,'a'>
<0,'c'>

Slovník

0: null
1: 'a'
2: 'ac'
3: 'aa'
4: 'c'



000000000001100001000000001011000110000000010110000100000000001100011
 $L_{out}=68$ bits

PŘÍKLAD

Kompresse pomocí LZ78 - krok 8

- Řetězec "aa" se nachází ve slovníku na pozici 3.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.

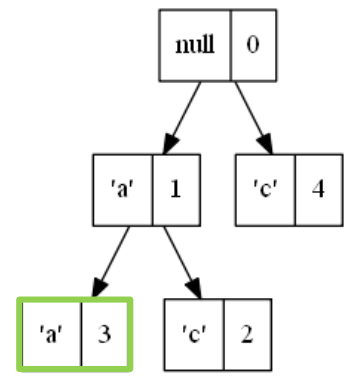


Výstup

- <0,'a'>
- <1,'c'>
- <1,'a'>
- <0,'c'>

Slovník

- 0: null
- 1 : 'a'
- 2 : 'ac'
- 3 : 'aa'
- 4 : 'c'



000000000001100001000000001011000110000000010110000100000000001100011
L_{OUT}=68 bits

PŘÍKLAD

Komprese pomocí LZ78 - krok 9

- Řetězec "aac" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 5.
- Kodér zapíše do výstupního toku značku <3,'c'> a posune se o jeden znak doprava.



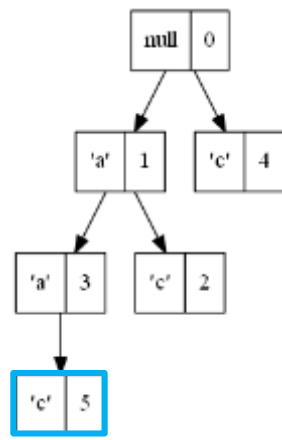
L(x)=40, H(x)=1.56

Výstup

- <0,'a'>
- <1,'c'>
- <1,'a'>
- <0,'c'>
- <3,'c'>

Slovník

- 0: null
- 1 : 'a'
- 2 : 'ac'
- 3 : 'aa'
- 4 : 'c'
- 5 : 'aac'



0000000000011000010000000010110001100000000101100001000000000110001100000001101100011
L_{OUT}=85 bits

Komprese pomocí LZ78 - krok 10

- Řetězec "a" se nachází ve slovníku na pozici 1.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



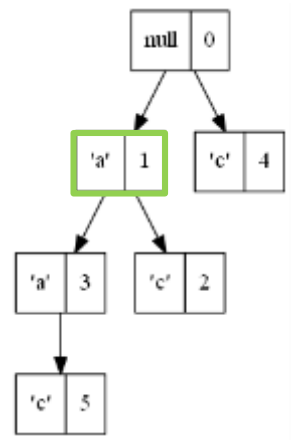
Výstup

<0,'a'>
<1,'c'>
<1,'a'>
<0,'c'>
<3,'c'>

Slovník

0: null
1: 'a'
2: 'ac'
3: 'aa'
4: 'c'
5: 'aac'

000000000011000010000000010110001100000000101100001000000000110001100000001101100011
L_{OUT}=85 bits



Komprese pomocí LZ78 - krok 11

- Řetězec "aa" se nachází ve slovníku na pozici 3.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.

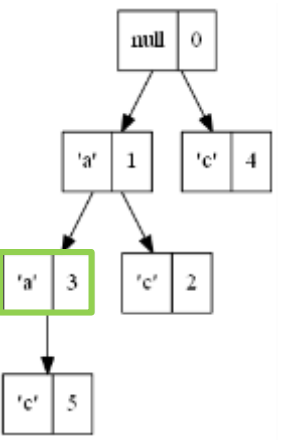


Výstup

<0,'a'>
<1,'c'>
<1,'a'>
<0,'c'>
<3,'c'>

Slovník

0: null
1: 'a'
2: 'ac'
3: 'aa'
4: 'c'
5: 'aac'



0000000000011000010000000001011000110000000001011000010000000000110001100000001101100011
L_{OUT}=85 bits

Komprese pomocí LZ78 - krok 12

- Řetězec "aac" se nachází ve slovníku na pozici 5.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.

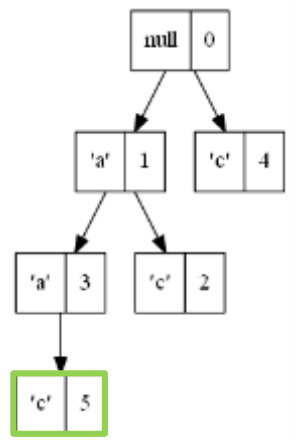


Výstup

<0,'a'>
<1,'c'>
<1,'a'>
<0,'c'>
<3,'c'>

Slovník

0: null
1 : 'a'
2 : 'ac'
3 : 'aa'
4 : 'c'
5 : 'aac'



0000000000011000010000000001011000110000000001011000010000000000110001100000001101100011
L_{OUT}=85 bits

PŘÍKLAD

Komprese pomocí LZ78 - krok 13

- Řetězec "aaca" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 6.
- Kodér zapíše do výstupního toku značku <5,'a'> a posune se o jeden znak doprava.

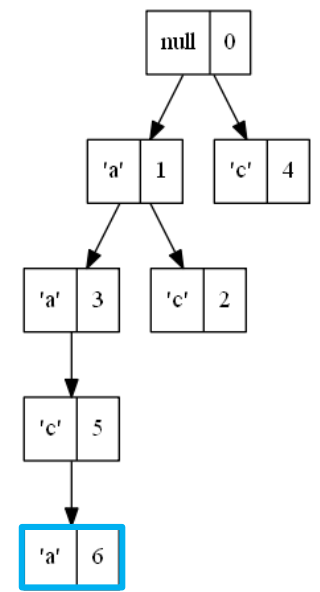


Výstup

- <0,'a'>
- <1,'c'>
- <1,'a'>
- <0,'c'>
- <3,'c'>
- <5,'a'>

Slovník

- 0: null
- 1 : 'a'
- 2 : 'ac'
- 3 : 'aa'
- 4 : 'c'
- 5 : 'aac'
- 6 : 'aaca'



000000000001100001000000000101100011000000000101100001000000000011000110000000110110001100000010101100001
L_{OUT}=102 bits

Komprese pomocí LZ78 - krok 14

- Řetězec "a" se nachází ve slovníku na pozici 1.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.

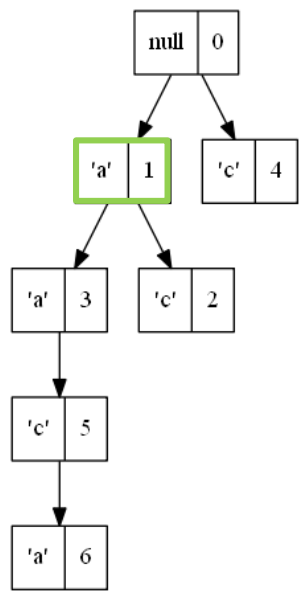


Výstup

- <0,'a'>
- <1,'c'>
- <1,'a'>
- <0,'c'>
- <3,'c'>
- <5,'a'>

Slovník

- 0: null
- 1: 'a'
- 2: 'ac'
- 3: 'aa'
- 4: 'c'
- 5: 'aac'
- 6: 'aaca'



0000000000011000010000000001011000110000000001011000010000000000110001100000001101100011000000010101100001
L_{OUT}=102 bits

Komprese pomocí LZ78 - krok 15

- Řetězec "ac" se nachází ve slovníku na pozici 2.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.

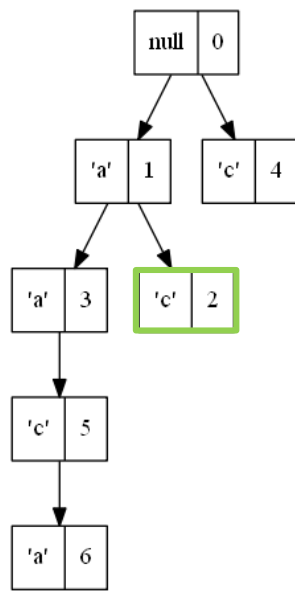


Výstup

<0,'a'>
<1,'c'>
<1,'a'>
<0,'c'>
<3,'c'>
<5,'a'>

Slovník

0: null
1 : 'a'
2 : 'ac'
3 : 'aa'
4 : 'c'
5 : 'aac'
6 : 'aaca'



000000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
L_{OUT}=102 bits

PŘÍKLAD

Komprese pomocí LZ78 - krok 16

- Řetězec "aca" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 7.
- Kodér zapíše do výstupního toku značku <2,'a'> a posune se o jeden znak doprava.

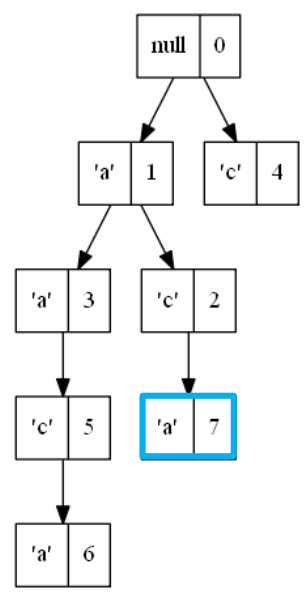


Výstup

<0,'a'>
<1,'c'>
<1,'a'>
<0,'c'>
<3,'c'>
<5,'a'>
<2,'a'>

Slovník

0: null
1: 'a'
2: 'ac'
3: 'aa'
4: 'c'
5: 'aac'
6: 'aaca'
7: 'aca'



00000000001100001000000000101100011000000000101100001000000000011000110000000110110001100000010101100001
00000001001100001
L_{OUT}=119 bits

Komprese pomocí LZ78 - krok 17

- Řetězec "a" se nachází ve slovníku na pozici 1.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<0,'a'>
<1,'c'>
<1,'a'>
<0,'c'>
<3,'c'>
<5,'a'>
<2,'a'>

Slovník

0: null
1 : 'a'
2 : 'ac'
3 : 'aa'
4 : 'c'
5 : 'aac'
6 : 'aaca'
7 : 'aca'

00000000001100001000000000101100011000000000101100001000000000011000110000000110110001100000010101100001
00000001001100001
L_{OUT}=119 bits

Komprese pomocí LZ78 - krok 18

- Řetězec "ac" se nachází ve slovníku na pozici 2.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

- <0,'a'>
- <1,'c'>
- <1,'a'>
- <0,'c'>
- <3,'c'>
- <5,'a'>
- <2,'a'>

Slovník

- 0: null
- 1 : 'a'
- 2 : 'ac'
- 3 : 'aa'
- 4 : 'c'
- 5 : 'aac'
- 6 : 'aaca'
- 7 : 'aca'

00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
00000001001100001
L_{OUT}=119 bits

Komprese pomocí LZ78 - krok 19

- Řetězec "aca" se nachází ve slovníku na pozici 7.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

- <0,'a'>
- <1,'c'>
- <1,'a'>
- <0,'c'>
- <3,'c'>
- <5,'a'>
- <2,'a'>

Slovník

- 0: null
- 1 : 'a'
- 2 : 'ac'
- 3 : 'aa'
- 4 : 'c'
- 5 : 'aac'
- 6 : 'aaca'
- 7 : 'aca'

00000000001100001000000000101100011000000000101100001000000000011000110000000110110001100000010101100001
00000001001100001
L_{OUT}=119 bits

PŘÍKLAD

Kompresa pomocí LZ78 - krok 21

- Řetězec "a" se nachází ve slovníku na pozici 1.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<0,'a'>

<1,'c'>

<1,'a'>

 $\langle 0, 'c' \rangle$

<3,'c'>

<5,'a'>

<2,'a'>

<7,'a'>

Slovník

0: null

8 : 'acaa'

1: 'a'

2 : 'ac'

3 : 'aa'

4 : 'c'

5 : 'aac'

6 : 'aaca'

7 : 'aca'

000000000011000010000000010110001100000000101100001000000000011000110000000110110001100000010101100001

0000000100110000100000011101100001

$$L_{OUT}=136 \text{ bits}$$

Komprese pomocí LZ78 - krok 23

- Řetězec "aca" se nachází ve slovníku na pozici 7.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

- <0,'a'>
- <1,'c'>
- <1,'a'>
- <0,'c'>
- <3,'c'>
- <5,'a'>
- <2,'a'>
- <7,'a'>

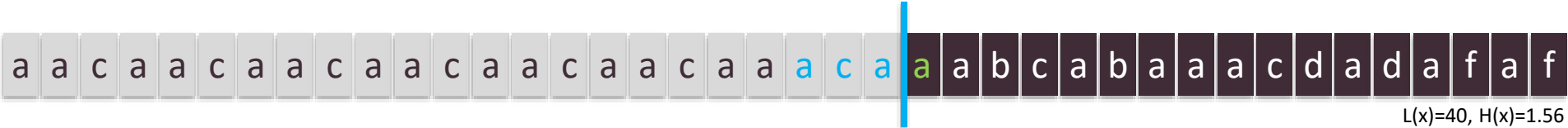
Slovník

- 0: null
- 1: 'a'
- 2: 'ac'
- 3: 'aa'
- 4: 'c'
- 5: 'aac'
- 6: 'aaca'
- 7: 'aca'
- 8: 'acaa'

00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
0000000100110000100000011101100001
L_{OUT}=136 bits

Komprese pomocí LZ78 - krok 24

- Řetězec "acaa" se nachází ve slovníku na pozici 8.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

- <0,'a'>
- <1,'c'>
- <1,'a'>
- <0,'c'>
- <3,'c'>
- <5,'a'>
- <2,'a'>
- <7,'a'>

Slovník

- 0: null
- 1 : 'a'
- 2 : 'ac'
- 3 : 'aa'
- 4 : 'c'
- 5 : 'aac'
- 6 : 'aaca'
- 7 : 'aca'
- 8 : 'acaa'

00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
0000000100110000100000011101100001
L_{OUT}=136 bits

PŘÍKLAD

-


$$L(x)=40, H(x)=1.56$$

Výstup

$\langle 0, 'a' \rangle$

<1,'c'>

<1,'a'>

 $\langle 0, 'c' \rangle$

$\langle 3, 'c' \rangle$

<5,'a'>

<2,'a'>

<7,'a'>

<8, 'a'>

Slovník

0: null

1: 'a'

2 : 'ac'

3 : 'aa'

4 : 'c'

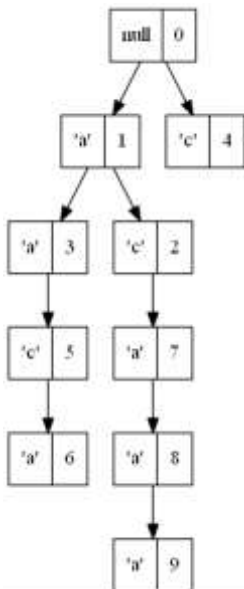
5 : 'aac'

6 : 'aaca'

7 : 'aca'

8 : 'acaa'

9 : 'acaaa'



000000000011000010000000010110001100000000101100001000000000011000110000000110110001100000010101100001

000000010011000010000001110110000100000100001100001

$$L_{OUT}=153 \text{ bits}$$

Komprese pomocí LZ78 - krok 26

- Řetězec "b" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 10.
- Kodér zapíše do výstupního toku značku <0,'b'> a posune se o jeden znak doprava.



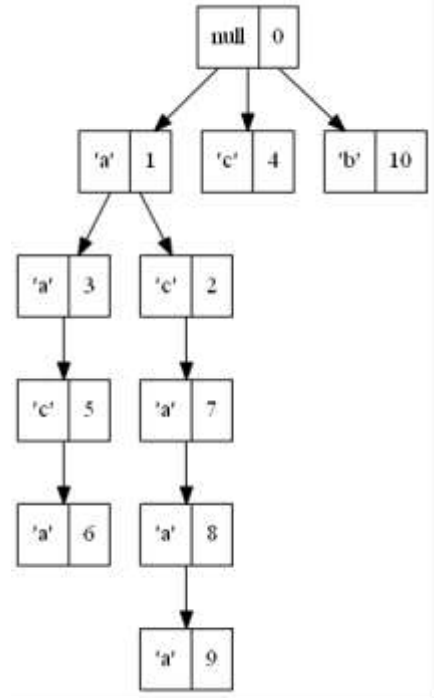
Výstup

- <0,'a'>
 - <1,'c'>
 - <1,'a'>
 - <0,'c'>
 - <3,'c'>
 - <5,'a'>
 - <2,'a'>
 - <7,'a'>
- <8,'a'>
- <0,'b'>

Slovník

- 0: null
- 1: 'a'
- 2: 'ac'
- 3: 'aa'
- 4: 'c'
- 5: 'aac'
- 6: 'aaca'
- 7: 'aca'
- 8: 'acaa'
- 9: 'acaaa'
- 10: 'b'

00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
0000000100110000100000011101100001000001000011000010000000001100010
 $L_{OUT}=170$ bits



Komprese pomocí LZ78 - krok 27

- Řetězec "c" se nachází ve slovníku na pozici 4.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	
<0,'c'>	
<3,'c'>	
<5,'a'>	
<2,'a'>	
<7,'a'>	

Slovník

0: null	8: 'acaa'
1: 'a'	9: 'acaaa'
2: 'ac'	10: 'b'
3: 'aa'	
4: 'c'	
5: 'aac'	
6: 'aaca'	
7: 'aca'	

00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
00000001001100001000000111011000010000010000110000100000000001100010
L_{OUT}=170 bits

Komprese pomocí LZ78 - krok 29

- Řetězec "b" se nachází ve slovníku na pozici 10.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	
<3,'c'>	
<5,'a'>	
<2,'a'>	
<7,'a'>	

Slovník

0: null	8: 'acaa'
1: 'a'	9: 'acaaa'
2: 'ac'	10: 'b'
3: 'aa'	11: 'ca'
4: 'c'	
5: 'aac'	
6: 'aaca'	
7: 'aca'	

000000000011000010000000010110001100000000101100001000000000110001100000001101100011000000010101100001
000000010011000010000001110110000100000100001100001000000000110001000000010001100001
L_{OUT}=187 bits

Komprese pomocí LZ78 - krok 30

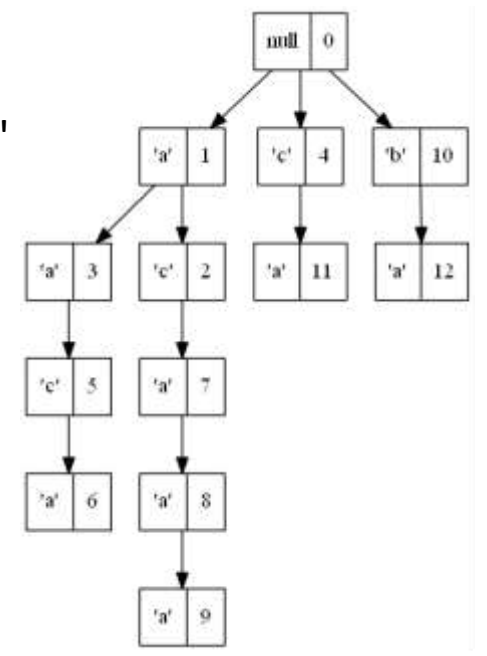
- Řetězec "ba" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 12.
- Kodér zapíše do výstupního toku značku <10,'a'> a posune se o jeden znak doprava.



Výstup	
<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	
<5,'a'>	
<2,'a'>	
<7,'a'>	

Slovník	
0: null	8: 'acaa'
1: 'a'	9: 'acaaa'
2: 'ac'	10: 'b'
3: 'aa'	11: 'ca'
4: 'c'	12: 'ba'
5: 'aac'	
6: 'aaca'	
7: 'aca'	

00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
000000010011000010000001110110000100000100001100001000000000110001000000010001100001000001000001000001
L_{OUT}=204 bits



Komprese pomocí LZ78 - krok 31

- Řetězec "a" se nachází ve slovníku na pozici 1.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



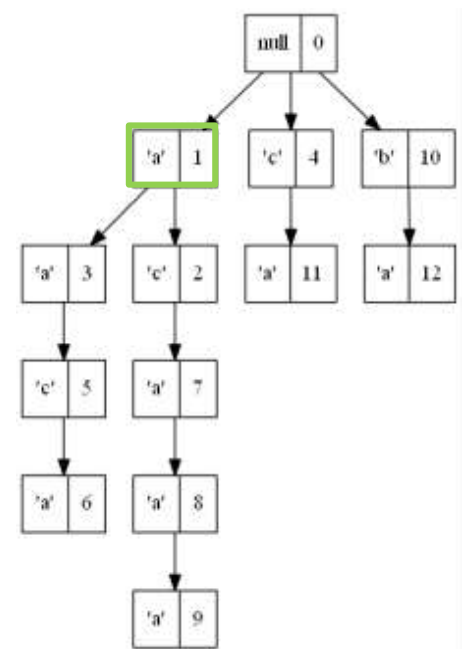
Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	
<5,'a'>	
<2,'a'>	
<7,'a'>	

Slovník

0: null	8: 'acaa'
1: 'a'	9: 'acaaa'
2: 'ac'	10: 'b'
3: 'aa'	11: 'ca'
4: 'c'	12: 'ba'
5: 'aac'	
6: 'aaca'	
7: 'aca'	

000000000011000010000000010110001100000000101100001000000000110001100000001101100011000000010101100001
00000001001100001000000111011000010000010000110000100000000011000100000001000110000100000101001100001
L_{OUT}=204 bits



PŘÍKLAD

-


$$L(x)=40, H(x)=1.56$$

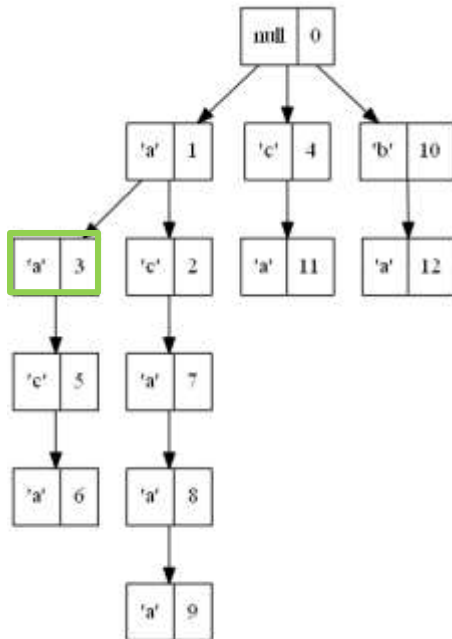
Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	
<5,'a'>	
<2,'a'>	
<7,'a'>	

Slovník

```
0: null
1: 'a'
2: 'ac'
3: 'aa'
4: 'c'
5: 'aac'
6: 'aaca'
7: 'aca'
```

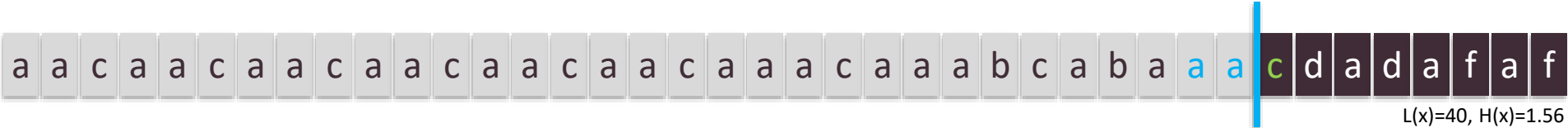
```
8 : 'acaa'
9 : 'acaaa'
10 : 'b'
11 : 'ca'
12 : 'ba'
```



0000000000011000010000000010110001100000000101100001000000000011000110000000110110001100000010101100001
000000010011000010000001110110000100000100001100001000000000011000100000001000110000100000101001100001
L_{OUT}=204 bits

Komprese pomocí LZ78 - krok 33

- Řetězec "aac" se nachází ve slovníku na pozici 5.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.

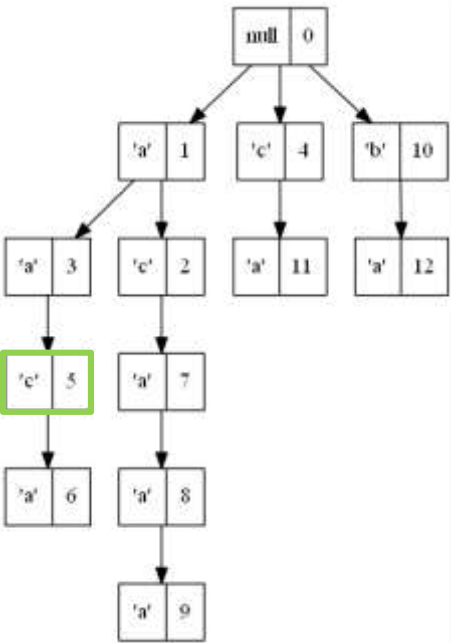


Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	
<5,'a'>	
<2,'a'>	
<7,'a'>	

Slovník

0: null	8: 'acaa'
1: 'a'	9: 'acaaa'
2: 'ac'	10: 'b'
3: 'aa'	11: 'ca'
4: 'c'	12: 'ba'
5: 'aac'	
6: 'aaca'	
7: 'aca'	



000000000011000010000000010110001100000000101100001000000000110001100000001101100011000000010101100001
00000001001100001000000111011000010000010000110000100000000011000100000001000110000100000101001100001
L_{OUT}=204 bits

Komprese pomocí LZ78 - krok 34

- Řetězec "aacd" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 13.
- Kodér zapíše do výstupního toku značku <5,'d'> a posune se o jeden znak doprava.

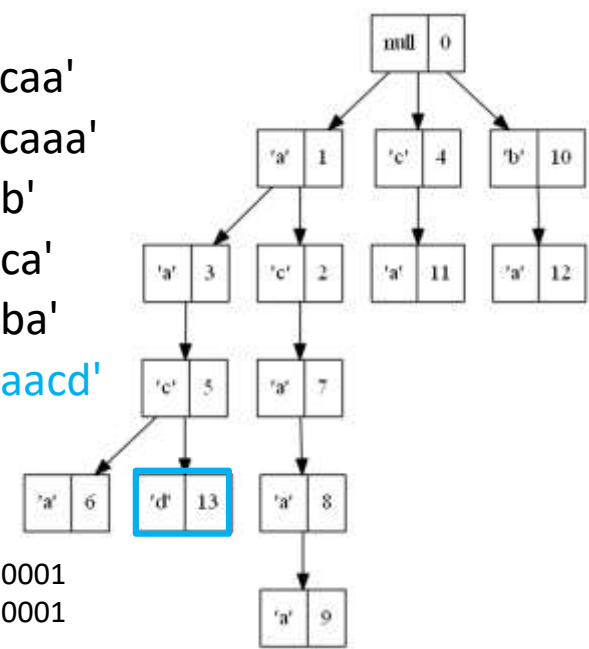


Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	<5,'d'>
<5,'a'>	
<2,'a'>	
<7,'a'>	

Slovník

0: null	8: 'acaa'
1: 'a'	9: 'acaaa'
2: 'ac'	10: 'b'
3: 'aa'	11: 'ca'
4: 'c'	12: 'ba'
5: 'aac'	13: 'aacd'
6: 'aaca'	
7: 'aca'	



00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
00000001001100001000000111011000010000010000110000100000000011000100000001000110000100000101001100001

00000010101100100

L_{out}=221 bits

Komprese pomocí LZ78 - krok 35

- Řetězec "a" se nachází ve slovníku na pozici 1.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



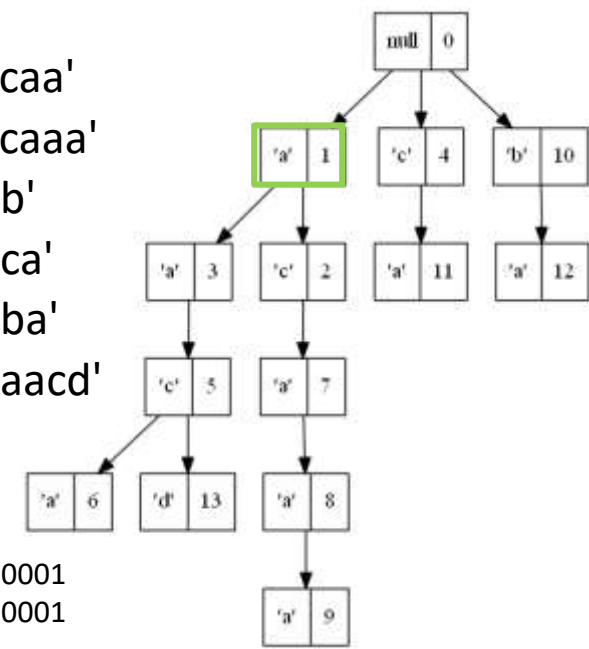
Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	<5,'d'>
<5,'a'>	
<2,'a'>	
<7,'a'>	

Slovník

0: null
1: 'a'
2: 'ac'
3: 'aa'
4: 'c'
5: 'aac'
6: 'aaca'
7: 'aca'

8: 'acaa'
9: 'acaaa'
10: 'b'
11: 'ca'
12: 'ba'
13: 'aacd'



00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
00000001001100001000000111011000010000010000110000100000000011000100000001000110000100000101001100001
00000010101100100
 $L_{out}=221$ bits

Komprese pomocí LZ78 - krok 36

- Řetězec "ad" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 14.
- Kodér zapíše do výstupního toku značku <1,'d'> a posune se o jeden znak doprava.



Výstup

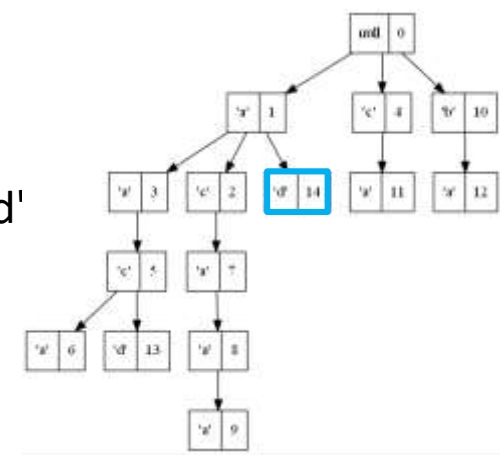
<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	<5,'d'>
<5,'a'>	<1,'d'>
<2,'a'>	
<7,'a'>	

Slovník

0: null	8: 'acaa'
1: 'a'	9: 'acaaa'
2: 'ac'	10: 'b'
3: 'aa'	11: 'ca'
4: 'c'	12: 'ba'
5: 'aac'	13: 'aacd'
6: 'aaca'	14: 'ad'
7: 'aca'	

000000000011000010000000010110001100000000101100001000000000110001100000001101100011000000010101100001
00000001001100001000000111011000010000010000110000100000000011000100000001000110000100000101001100001
000000101011001000000000101100100

L_{OUT}=238 bits



Komprese pomocí LZ78 - krok 37

- Řetězec "a" se nachází ve slovníku na pozici 1.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



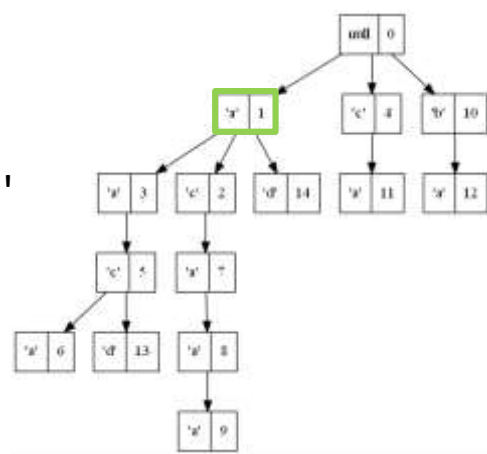
Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	<5,'d'>
<5,'a'>	<1,'d'>
<2,'a'>	
<7,'a'>	

Slovník

0: null	8: 'acaa'
1: 'a'	9: 'acaaa'
2: 'ac'	10: 'b'
3: 'aa'	11: 'ca'
4: 'c'	12: 'ba'
5: 'aac'	13: 'aacd'
6: 'aaca'	14: 'ad'
7: 'aca'	

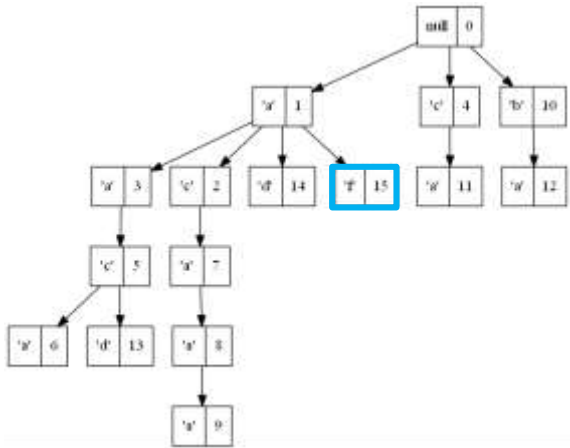
00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
00000001001100001000000111011000010000010000110000100000000011000100000001000110000100000101001100001
00000010101100100000000000101100100
L_{OUT}=238 bits



PŘÍKLAD

Kompresse pomocí LZ78 - krok 38

- Řetězec "af" se ve slovníku nenachází a je proto kodérem do slovníku přidán a sice na pozici 15.
- Kodér zapíše do výstupního toku značku <1,'f'> a posune se o jeden znak doprava.



a a c a a c a a c a a c a a c a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=1.56

Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	<5,'d'>
<5,'a'>	<1,'d'>
<2,'a'>	<1,'f'>
<7,'a'>	

Slovník

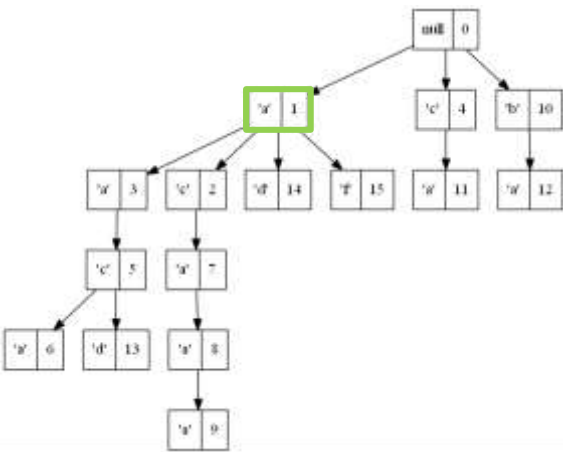
0 : null	8 : 'acaa'
1 : 'a'	9 : 'acaaa'
2 : 'ac'	10 : 'b'
3 : 'aa'	11 : 'ca'
4 : 'c'	12 : 'ba'
5 : 'aac'	13 : 'aacd'
6 : 'aaca'	14 : 'ad'
7 : 'aca'	15 : 'af'

00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
00000001001100001000000111011000010000010000110000100000000011000100000001000110000100000101001100001
00000010101100100000000001011001000000000101100110000000101100110

L_{out}=255 bits

Komprese pomocí LZ78 - krok 39

- Řetězec "a" se nachází ve slovníku na pozici 1.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=1.56

Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	<5,'d'>
<5,'a'>	<1,'d'>
<2,'a'>	<1,'f'>
<7,'a'>	

Slovník

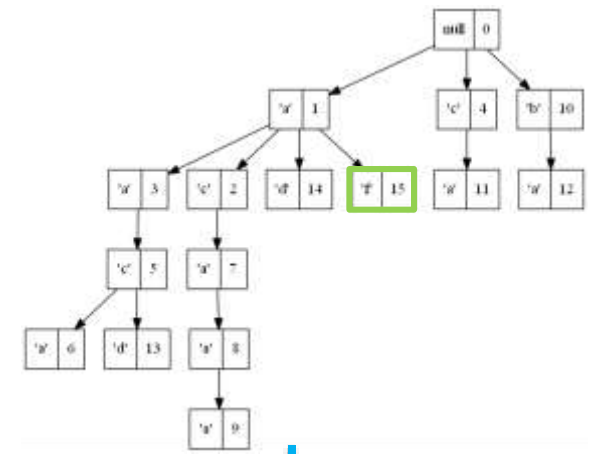
0: null	8: 'acaa'
1: 'a'	9: 'acaaa'
2: 'ac'	10: 'b'
3: 'aa'	11: 'ca'
4: 'c'	12: 'ba'
5: 'aac'	13: 'aacd'
6: 'aaca'	14: 'ad'
7: 'aca'	15: 'af'

00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000010101100001
00000001001100001000000111011000010000010000110000100000000011000100000001000110000100000101001100001
000000101011001000000000010110010000000000101100110

L_{out}=255 bits

Komprese pomocí LZ78 - krok 40

- Řetězec "af" se nachází ve slovníku na pozici 15.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=.56

Výstup

<0,'a'>	<8,'a'>
<1,'c'>	<0,'b'>
<1,'a'>	<4,'a'>
<0,'c'>	<10,'a'>
<3,'c'>	<5,'d'>
<5,'a'>	<1,'d'>
<2,'a'>	<1,'f'>
<7,'a'>	

Slovník

0: null	8: 'acaa'
1: 'a'	9: 'acaaa'
2: 'ac'	10: 'b'
3: 'aa'	11: 'ca'
4: 'c'	12: 'ba'
5: 'aac'	13: 'aacd'
6: 'aaca'	14: 'ad'
7: 'aca'	15: 'af'

00000000001100001000000001011000110000000010110000100000000011000110000000110110001100000001010110000100000001001
1000010000001110110000100000100001100001000000000110001000000010001100001000001010011000010000001010110010000000
00010110010000000000101100110

L_{OUT}=255 bits

Komprese pomocí LZ78 - výstup

- Vstupní řetězec byl zakódován pomocí 264 bitů (6,6 bitů na symbol). Dosažen kompresní faktor 0,825 (142 / 40*8), tj. horší než v případě LZSS (0,443) i Huffmanova kódu (0,459).
- Poznámka: výstup neobsahuje žádnou hlavičku, parametry kodéru (maximální počet položek slovníku) jsou předány externě.
- Token má fixní délku: 9b offset + 8b znak

a a c a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=1.56



data

```
00000000001100001000000001011000110000000010110000100000
00000110001100000001101100011000000101011000010000000100
11000010000001110110000100000100001100001000000000011000
10000000100011000010000010100110000100000010101100100000
0000010110010000000000101100110000001111
```

Další varianty LZ77 a LZ78

■ LZFG (Fiala a Greene, 1989)

- Pojmem LZFG se označuje několik příbuzných komprimačních algoritmů, které jsou v podstatě **kříženci mezi metodami LZ 77 a LZ 78** (LZFG používá posuvné okno, ale uchovává si stav v modifikovaném trie)
- Všechny metody třídy LZFG generují komprimovaný soubor obsahující pouze tokeny dvou druhů:
 - **Literal token** v komprimovaném souboru znamená, že teď následuje řetězec literálů (ASCII kódů) - tedy objeví-li se v komprimovaném souboru <literal x> znamená to příkaz pro dekompressor - načti x znaků z komprimovaného souboru a zapiš je přímo na výstup (dekomprimovaný soubor).
 - **Copy token** ukazuje na řetězec, který už se objevil ve zpracovávaných datech - přesněji: objeví-li se v komprimovaném souboru <copy x, y> říká to dekompresoru, aby se vrátil o y znaků ve výstupním (dekomprimovaném) souboru a zkopíroval odtud x znaků na konec výstupního souboru.
- Dosažená komprese závisí mimo jiné na tom, jakým způsobem budeme zapisovat (tedy kolik místa zaberou) literal a copy tokeny.

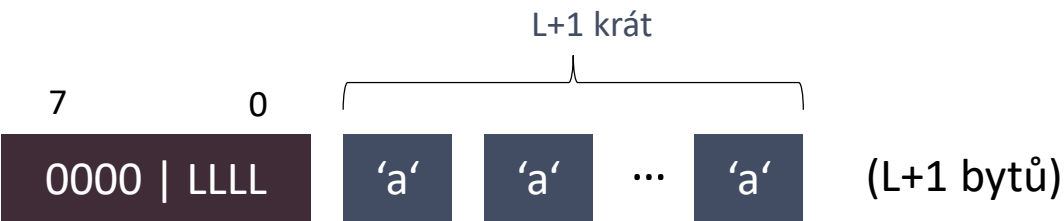
■ Příklad:

- Z řetězce "the_boy_on_my_right_is_the_right_boy" může po zakódování vzniknout např. řetězec: „<literal 23>the_boy_on_my_right_is_<copy 4, 23><copy 6, 13><copy 3, 29>”

LZFG – varianta A1

- Metoda A1 je nejjednodušší metoda třídy LZFG.
 - Používá 8 bitů pro literal token a 16 bitů pro copy token a byla vytvořena pro kompresi 8 bitového textu.

- Rozlišení a formát tokenů:
 - Pokud jsou první 4 bity nulové, pak tento kód je kódem **literal tokenu** a další 4 bity kódují délku řetězce x v rozmezí [1..16]. Za tímto kódem tedy musí následovat x 8-bitových hodnot ASCII znaků.



- V opačném případě (alespoň jeden z prvních 4 bitů nenulový) se jedná o kód **copy tokenu** a první 4 bity udávají délku x v rozmezí [2..16] a dalších 12 bitů je vzdálenost y (odkud kopírovat) v rozmezí [1..4096].



LZFG – varianta A1

- Kodér i dekodér používají posuvné okno (podobně jako LZ 77), ovšem toto je jinak reprezentováno pro urychlení komprese.
- V každém kroku se kodér rozhoduje, zda zapíše literal token či copy token tímto způsobem:
 - Pokud je **kodér v "počátečním" stavu** (právě dokončil copy token nebo ukončil literal token kvůli limitu na 16 znaků či jde o úplný začátek komprese):
 - Pokud nejdelší shoda (s předponou slova v aktuálním okně), kterou se kodéru podaří najít (v prohlížečím okně) je délky alespoň 2, bude zapsán **copy token**.
 - Jinak začneme zapisovat **literal token**.
 - V opačném případě (zahájeno zapisování literal tokenu)
 - Budeme literal token prodlužovat tak dlouho dokud neobjevíme **shodu délky alespoň 3**, nebo dokud nedosáhneme limitu na délku literal tokenu (tj. 16).
- Pro rychlé vyhledání nejdelší shody se používá modifikovaná varianta McCreightova suffix tree (varianta trie bez nadbytečných uzlů majících jediného potomka).

LZFG – varianta A1

- Metoda A1 zakóduje text "the_boy_on_my_right_is_the_right_boy" takto:

```
"(literal 16)the_boy_on_my_ri(literal 7)ght_is_(copy 4 23)(copy 6 23)(copy 3 29),,
```

- v prvních 16 krocích jsme nenašli shodu a narazili na limit 16-ti znaků, zapíšeme literal token a pokračujeme.
 - Po sedmi krocích se nám podaří nalézt shodu délky 4, zapíšeme proto literal token následovaný copy tokenem.
 - Poté se nám podaří nalézt shodu délky 6 a následně 3 znaky.
- Shrnutí:
 - Metoda A1 je podobná metodě LZ 77, ale liší se použitím speciálního literal tokenu, tzn. vypůjčuje si myšlenku pohyblivého bufferu z LZ77, ale chová se také jako LZ78, protože vytváří značky se dvěma poli.
 - Metoda A1 byla vytvořena pro kompresi 8 bitového textu, ale ukazuje se jako celkem vhodná i na ostatní druhy dat.
 - Empirické studie autorů algoritmu ukázaly, že pro anglický text a zdrojové soubory lze mírně zlepšit kompresní poměr zmenšením maximální délky v literal tokenu o 1 bit. Tímto ovšem přijdeme o možnost práce se souborem po bytech (literal token by byl 7 bitů).

LZFG – varianta A2

- Metoda A2 používá **delší vyhledávací buffer** (dlouhý až 21KB), čímž se zlepšuje komprese, protože se dají najít delší kopie.
- Nastává však problém s velikostí značky.
 - Velký vyhledávací buffer implikuje velké posuny ve značkách copy; dlouhé kopie implikují v těchto značkách velká pole „délky“. Současně očekáváme, že obě pole (posunu i délky) typické značky copy budou krátká, protože většina shod se nalézá blízko k začátku vyhledávacího bufferu.
 - Metoda A2 používá pro zakódování polí kód s proměnnou délkou, tzv. start-step-stop kód založený na obecném unárním kódu
- **Start-step-stop kód (*start, step, stop*)**
 - Slovo začíná sledem n jedničkových bitů (unárním kódem) následovaným nulovým bitem a pokračuje binárním číslem délky $start + n * step$ bitů.
 - Binární číslo je v rozsahu $(stop - start) / step$

n	kód	rozsah
0	0xxx	0-7
1	10xxxxx	8-39
2	110xxxxxxxx	40-167
3	111xxxxxxxxxx	168-679

Příklad (3,2,9)-kódu

Tedy příslušný start-step-stop kód pro 7 bude 0111 a pro 10 bude 1000010.

LZFG – varianta A2 (informativní)

■ Reprezentace tokenů ve výstupním toku:

- Pokud první 3 bity tokenu jsou 000, jde o **literal** token. Pro kódování délky následujícího řetězce literálů (ASCII kódů) je použit **(0, 1, 5) kód**, tedy maximální délka tohoto řetězce je 63 znaků.
- Pokud první 3 bity tokenu nejsou 000, jedná se o copy token a následuje odkaz na příslušnou **pozici** v okně zapsaný **(10,2,14)** kódem, což nám umožní velikost okna až 21504 bytů. K ukládání **délky** shodujícího se řetězce v **copy** tokenu používá **(2, 1, 10)** start-step-stop kód, který umožňuje, aby délka tohoto řetězce byla až 2044 znaků (kódy jsou o 1 posunuty, protože nepotřebujeme kód pro délku 1; tedy 001 znamená 2 atd.).

■ Kodér:

- Pro rozhodování mezi copy tokenem a literal tokenem je zvolena stejná strategie jako u algoritmu A1.
- Aby se dosáhlo lepší komprese, používá metoda A2 další tři vylepšení (viz další slide).
- Algoritmus A2 má velmi rychlou dekompresi s minimálními nároky na paměť, ale komprese ačkoli probíhá v lineárním čase (vzhledem k délce souboru) je asi 5 krát pomalejší než dekomprese. Algoritmy A1 a A2 jsou tedy vhodné pro použití v případech, kdy není nutná maximální rychlost.

LZFG – další varianty (B1,B2 a C2) - informativní

■ Metoda B1 a B2

- vyvinuty pro archivaci souborů, kde metody A1 a A2 jsou příliš pomalé.
- Metody B1 a B2 vychází po řadě z metod A1 a A2 ale počítají odkazy na pozice do prohlížečího okna (pro copy tokeny) odlišně a blíží se tak více metodě LZ 78.
- Odkazy na pozice mohou ukazovat jen na místo, odpovídající začátku řetězce, který byl zakódován nějakým předchozím copy tokenem nebo na místa, která byla zapsána pomocí literal tokenu (klidně i doprostřed řetězce literálů). To znamená, že zde již **nelze adresovat všechny pozice v prohlížečím okně**.
- Rychlost komprimace zhruba ztrojnásobí, ovšem rychlost dekomprese a rychlost, jakou se algoritmus adaptuje na příchozí data se o něco sníží.

■ Metoda C2

- Metoda C2 byla vytvořena za účelem zlepšení kompresního poměru na úkor rychlosti algoritmu.
- Pokud se nějaký řetězec objevuje v souboru často, žádná z výše uvedených metod nefunguje optimálně
- Založíme-li tedy kódová slova pro kopírování řetězce na datové struktuře slovníku, znatelně zlepšíme kompresní poměr (dva typy copy tokenů **NodeCopy** a **LeafCopy**).
- Kódovými slovy založenými na pozicích v explicitním slovníku se tato metoda blíží metodě LZ 78, prohlížečím oknem a explicitním udáváním délky kopírovaného řetězce připomíná metodu LZ 77.



Další varianty LZ77 a LZ78

■ LZRW (Ross Williams, 1991)

- Pojmem LZRW se označuje několik příbuzných komprimačních algoritmů, které jsou se snaží vylepšit rychlost komprese LZ77 **využitím hashovacích tabulek**.
- V dnešní době existuje několik variant LZRW1 až LZRW5

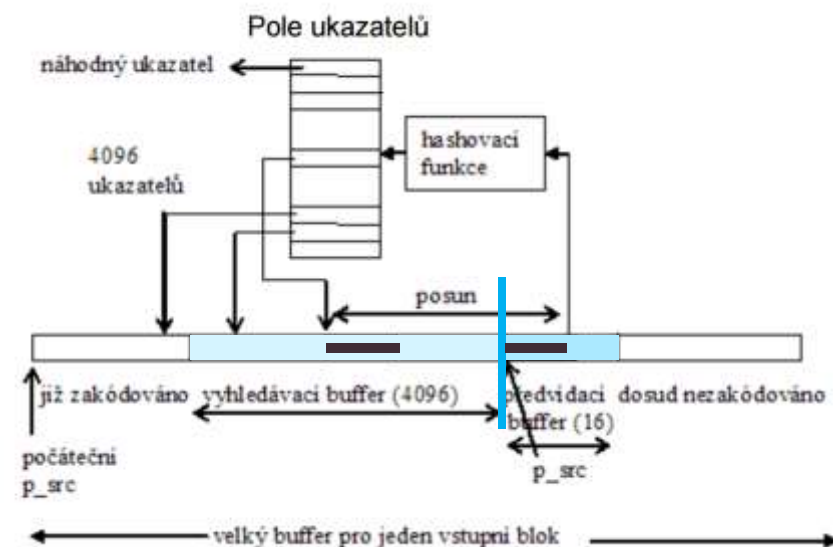
■ Metoda LZRW1

- Navazuje na LZFG A1, základní myšlenka je **najít s použitím hashovací tabulky shodu** “v jednom kroku”.
- Je to rychlé, ale neefektivní, protože nalezená shoda není vždy ta nejdelší.
- Metoda rozděljuje vstupní tok do bloků a **využívá celou přidělenou paměť jako buffer**. Délka vyhledávacího bufferu je 4K a u předvídacího bufferu je to 16 bytů. Buffery se posouvají po vstupním bloku v paměti zleva doprava.
- **První tři znaky předvídacího bufferu se hashují** do 12-bitového čísla i , které se používá k indexaci pole $2^{12} = 4096$ ukazatelů do vyhledávacího bufferu.
- LZRW1 dosahuje asi o 10% horšího kompresního poměru než LZC (funkce compress v UNIXu), ale je čtyřikrát rychlejší. Oproti LZFG-A1 dosahuje 4% zhoršení ale pracuje desetkrát rychleji a hodí se tak pro případy, kdy je rychlost důležitější, než kompresní výkonnost (např. transparentní komprimace dat v souborovém systému).

Metoda LZRW1

■ Princip kódování

- První tři znaky předvídacího bufferu se **hashují** do 12-bitového čísla i , které se používá k indexaci pole $2^{12} = 4096$ ukazatelů do vyhledávacího bufferu.
- Z tabulky H se na indexu i zjistí ukazatel $P=H[i]$ do bufferu.
- Pokud P ukazuje mimo vyhledávací buffer, neexistuje shoda a tudíž první znak v předvídacím bufferu je chápán jako literál a kodér posune okno o jednu pozici doprava. Stejná situace nastane i tehdy, ukazuje-li P do vyhledávacího bufferu, ale obsah se neshoduje s předvídacím bufferem.
- Pokud se shodují alespoň tři symboly od P , kodér najde nejdelší shodu (maximálně 16 znaků), vyšle položku shody a posune okno o délku shody.



Metoda LZRW1

■ Výstup kodéru

- skládá se ze skupin, kdy každá začíná 16-bitovým řídicím slovem (flagy), následovaným 16 položkami.
- Šestnáct bitů řídicího slova označuje typ každé z následujících 16 položek
 - Značka 0 označuje literal a značka 1 copy položku
 - Poslední skupina může obsahovat méně, než 16 položek.
- Každá položka je buď **8-bitový literal**, nebo **16-bitová položka copy** (shoda), tvořená 4-bitovým **polem délky** b (kdy délka je b+1) a 12-bitovým **posunem** (pole a a c). Pole délky kóduje hodnoty mezi 3 a 16.

■ Dekodér

- Dekodér je jednodušší, než kodér, protože nepotřebuje pole ukazatelů.
 - S využitím jednoho ukazatele p_src udržuje velký buffer stejným způsobem, jako kodér.
 - Dekodér přečte z komprimovaného toku řídicí slovo, a použije jeho 16 bitů ke čtení 16 položek.
 - Položka literal se dekóduje jeho připojením k bufferu a inkrementací p_src o 1.
 - Položka copy se dekóduje odečtením posunu od p_src, načtením řetězce z vyhledávacího bufferu s délkou určenou polem délky, a jeho připojením k bufferu.
 - Následně se p_src inkrementuje o délku.

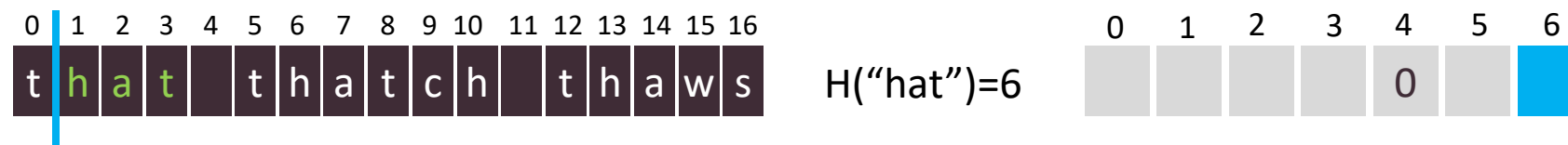


LZRW1 – příklad hledání shody

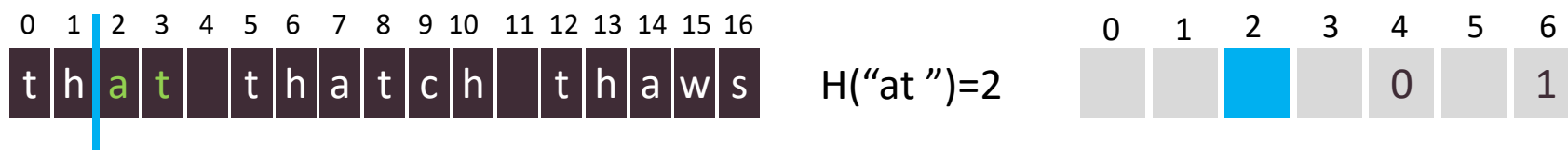
- Pro jednoduchost uvažujme prvních 7 položek tabulky, náhodně inicializovaných.
- V prvním kroku se vezmou první tři znaky a hashují se do 12-bit čísla, např. $H(\text{"tha"})=4$. Tabulka obsahuje jiný ukazatel a tak se na výstup dá znak "t", do tabulky se uloží ukazatel a kodér se posune o znak doprava.



- V druhém kroku se vypočte $H(\text{"hat"})=6$. Tabulka obsahuje jiný ukazatel a tak se na výstup dá znak "h", do tabulky se uloží ukazatel a kodér se posune o znak doprava.

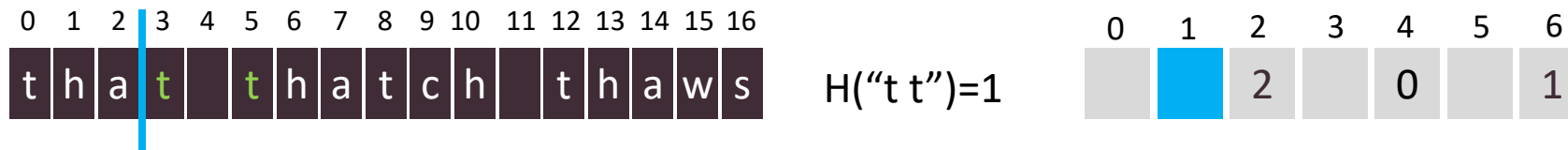


- V třetím kroku se vypočte $H(\text{"at "})=2$. Na výstup se dá "a" a pokračuje jako v předchozím případě.

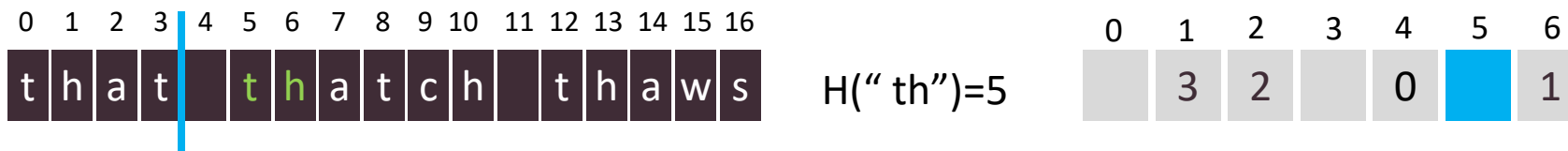


LZRW1 – příklad hledání shody

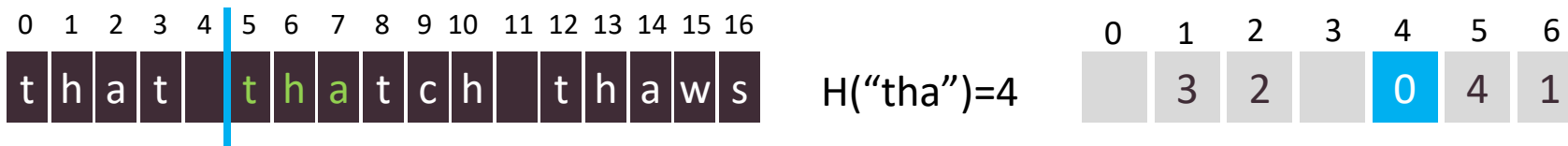
- V dalším kroku se vypočte $H(\text{"t t"})=1$. Na výstup se dá znak "t" a pokračuje jako v předchozím případě.



- V dalším kroku se vypočte $H(\text{"th"})=5$. Na výstup se dá znak " " a pokračuje jako v předchozím případě.



- V dalším kroku se vypočte $H(\text{"tha"})=4$. Položka tabulky na indexu 4 odkazuje do bufferu na index 0, který se nachází v okně a je nalezena shoda délky 4. Na výstup se dá token $\langle 5, 4 \rangle$ (aktuální index – 5 = index 0) a kódér se posune o 4 znaky doprava.



Metoda LZRW4 – informativní

■ Princip metody LZRW4

- LZRW4 kombinuje slovníkovou metodu s predikcí a používá některé techniky z LZRW1
- Metoda využívá dlouhý 1 MB buffer, ve kterém se posouvá zleva doprava vyhledávací i předvídací buffer.
- V každém okamžiku kódovacího procesu se **používá k predikci aktuálního symbolu kontext 2-řádu** (dva nejposlednější symboly ve vyhledávacím bufferu).
- Dva symboly, tvořící kontext se hashují do 12-bitového čísla, které se používá jako ukazatel do **pole úseků** s $2^{12} = 4096$ položkami. Každý úsek obsahuje 32 ukazatelů na vstupní data v 1 MB bufferu (každý ukazatel je takto dlouhý 20 bitů). Všechny 32 ukazatelů v úseku A[I] se testuje, aby se našla nejdelší shoda mezi předvídacím bufferem a dosud načtenými vstupními daty. Vybere se nejdelší shoda a zakóduje se do 8 bitů.

■ Výstup kodéru

- Na výstupu mohou být dva typy značek – literál a copy. K odlišení se používá flag.
- **Literál** se kóduje pomocí 8 bitů. Položka **copy** má 8 bitů a tvoří ji **dvě pole**. První 3 bity kódují délku shody, zbývajících 5 bitů identifikuje ukazatel v úseku.
- Flagy se shromažďují do skupin po 16 a každá skupina se vysílá, stejně jako v LZRW1, a předchází 16 položek, kterých se týká.



Metoda LZW

- LZW (Terry Welch, 1984) je varianta LZ78, která výrazně ovlivnila oblast komprese dat a inspirovala mnoho lidí k implementaci a modifikacím této metody. Metoda byla dokonce patentována a licencována.
- Hlavním rysem je, že metoda LZW začíná **inicializací slovníku na všechny symboly abecedy**.
 - V případě 8-bitových symbolů je již před načtením dat prvních 256 položek slovníku obsazeno (položky 0 až 255).
- Tento jednoduchý krok dovoluje odstranit druhé pole ze značky (tokenu) a **značka LZW** tedy udává **pouze index do slovníku**. Důvodem je fakt, že vstupní znak se ve slovníku vždy najde.
- Princip činnosti kodéru:
 - kodér načítá vstupní symboly jeden za druhým, a střežá je v řetězci F.
 - Po každém připojení jednoho symbolu se řetězec hledá ve slovníku a když se najde, proces pokračuje.
 - V určitém okamžiku způsobí přidání symbolu x, že vyhledávání selže; řetězec F ve slovníku je, ale řetězec F+x (symbol x připojený k F) nikoliv. V tomto okamžiku kodér uloží token <I>, kde I je index na F ve slovníku, do slovníku přidá řetězec F+x a inicializuje F na x.



Metoda LZW

■ Pseudokód kodéru LZW:

```
inicializace slovníku D = [0,...,255], F = ""  
while (nepřečten celý vstupní tok) do  
  begin  
    načti symbol X  
    if F+X ∈ D then  
      přidej X k řetězci se shodou F = F + X a  
      poznač si pozici i = pozice F v D  
    else  
      ulož značku <i>  
      přidej F+X do slovníku, F = X, i = ord(X)  
    end  
  end  
  ulož značku <i>
```

■ Pseudokód dekodéru LZW:

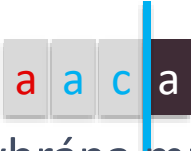
```
inicializace slovníku D = [0,...,255], F = ""  
while (nepřečten celý vstupní tok) do  
  begin  
    načti značku <i>  
  
    pokud i je ve slovníku, pak S = D[i],  
    jinak S = F + F[0]  
  
    zapiš do výstupního toku řetězec S  
    if nejedná se o první značku then  
      přidej do slovníku položku F+S[0]  
    end  
  
    nastav F = S  
  end
```

ošetření situace, kdy položka
ještě nebyla přidána do slovníku

Dekomprese pomocí LZW

- Ukázka situace, kdy při dekompresi položka ve slovníku ještě není, protože se právě zpracovává

načtený token	zapsáno na výstup	přidáno do slovníku	poznámka
<97>	'a'		první token, do slovníku se nic nepřidává
<97>	'a'	256: 'aa'	první znak aktuálního řetězce + předchozí shoda
<99>	'c'	257: 'ac'	
<256>	'aa'	258: 'ca'	
<258>	'ca'	259: 'aac'	
<257>	'ac'	260: 'caa'	
<259>	'aac'	261: 'aca'	
<262>	????		položka slovníku se právě zpracovává, vezmeme proto předchozí shodu, tj. 'aac' a připojíme zprava první znak
	'aaca'	262: 'aaca'	

Stav okna při zapisování tokenu <259>:  dochází k vložení položky <262>. Aby byla položka 262 ihned v následujícím kroku vybrána musí další shoda začínat na 

Struktura slovníku LZW

- Z principu činnosti kodéru vyplývá, že pro každý řetězec slovníku s více než jedním symbolem existuje ve slovníku řetězec, který je o jeden symbol kratší.
 - Kodér načítá symboly a spojuje je do proměnné F tak dlouho, dokud kodér nenačte symbol x , který způsobí neúspěšné vyhledání ve slovníku. V tento okamžik přidá do slovníku řetězec $F+x$.
- Slovník lze uchovávat pomocí stromové struktury, kterou jsme používali u LZ78. Přidání řetězce $F+x$ vede na přidání jediného uzlu.
- Jak však datovou strukturu pro strom navrhnout tak, aby uzel mohl mít libovolný počet potomků, ale aby se pro ně předem nemusela v každém uzlu vyhrazovat paměť?
 - Jedna z možností je reprezentovat trie strom pomocí pole uzlů, přičemž pohyb stromem dolů se provede hashovacím procesem, při kterém se ukazatel na uzel (index) a symbol potomka ' x ' hashují pomocí vhodné HASH funkce, která vygeneruje index do pole uzlů odkazující na potomka.

Implementace trie pomocí pole uzlů

■ Příklad:

- Předpokládejme, že máme k dispozici pole 512 uzlů a že kodér již zpracoval řetězec "abcd" a tím v poli vytvořil čtyři uzly s následujícím obsahem

	1	...	97	98	...	266	...	283	284	285	...	299	...	...	512
symbol	c		a			b			d						
index rodiče	266		0			97			1						

- Kodér si poznačil ukazatel na pozici 284 (poslední znak zpracovaného řetězce).
- Předpokládejme, že kodér má nyní na vstupu symbol 'e' a hledá tedy řetězec abcde.
- Hledání se redukuje na výpočet $H(284, 101)$, kde H je vhodná hash funkce a 101 je ASCII hodnota pro symbol 'e'.
- Nastat může jedna ze tří situací:
 - $H(284, 101)$ ukazuje na uzel, který je prázdný (např. 299). Kodér do uzlu zapíše hodnotu 101 a index 284
 - $H(284, 101)$ ukazuje na uzel, který není prázdný a který obsahuje znak 'e'. Kodér si zapamatuje index a pokračuje načtením dalšího symbolu (hledáním delší shody).
 - $H(284, 101)$ ukazuje na uzel, který není prázdný a který obsahuje odlišný znak. Nastala kolize a je zapotřebí nalézt jiný vhodný index.

Implementace trie pomocí pole uzlů

- V praxi tvoříme uzly se třemi částmi (poli):
 - ukazatel na rodičovský uzel,
 - ukazatel vytvořený hashováním (index), a
 - kód symbolu v uzlu obsaženého (standardně ASCII).
- Druhé pole je nutné kvůli kolizím

	1	...	97	98	...	108	...	266	267	268	...	278	...	...	512
symbol			a			l									
index rodiče			0			0									
index			97			108									

```
dict[l].parent:=...;  
dict[l].index:=...;  
dict[l].symbol:=...;
```

Příklad pro řetězec “alf eats alfalfa”:

1. H(‘a’,0)=97. Pozice 97 již obsahuje znak ‘a’, pokračujeme hledáním delší shody.
2. H(‘l’,97)=278. Pozice pole 278 záznam neobsahuje, nastaví se na (‘l’,97,278).
3. H(‘f’,108)=266. Pozice pole 266 záznam neobsahuje, nastaví se na (‘f’,108,266).



Implementace trie pomocí pole uzlů

Příklad pro řetězec “alf eats alfalfa”:

- 4. $H(' ', 102) = 269$. Pozice pole 269 se nastaví na $(' ', 102, 269)$
- 5. $H('e', 32) = 267$. Pozice pole 267 se nastaví na $('e', 32, 267)$.
- 6. $H('a', 101) = 265$. Pozice pole 265 se nastaví na $('a', 101, 265)$.
- 7. $H('t', 97) = 272$. Pozice pole 272 se nastaví na $('t', 97, 272)$.
- 8. $H('s', 116) = 265$. **Kolize!** Skočí se na následující volnou pozici, 268, a nastaví se na $('s', 116, 268)$. Proto se musí index uložit.
- 9. $H(' ', 115) = 270$. Pozice pole 270 se nastaví na $(' ', 115, 270)$.

	1	...	97	98	...	265	266	267	268	269	270	271	272	...	278	...
symbol			a			a	f	e	s	_	_		t		l	
index rodiče			0			101	108	32	116	102	115		97		97	
index			97			265	266	267	265	269	270		272		278	



Implementace trie pomocí pole uzlů

Příklad pro řetězec “alf eats alfalfa”:

- 10. $H('a',32)=268$. **Kolize!** Skočí se na následující volnou pozici, 271, a nastaví se na $(‘a’,32,268)$.
- 11. $H('l',97)=278$. Pozice pole 278 již obsahuje index 278 a symbol ‘l’ z kroku 2, takže nic jiného se nemusí ukládat, nebo přidat novou položku trie.
- 12. $H('f',278)=276$. Pozice pole 276 se nastaví na $(‘f’,278,276)$.
- 13. $H('a',102)=274$. Pozice pole 274 se nastaví na $(‘a’,102,274)$.
- 14. $H('l',97)=278$. Pozice pole 278 již obsahuje index 278 a symbol ‘l’ z kroku 11, takže se nic nemusí dělat.
- 15. $H('f',278)=276$. Pozice pole 276 již obsahuje index 276 a symbol ‘f’ z kroku 12, takže se nic nemusí dělat.
- 16. $H('a',276) \rightarrow 274$. **Kolize!** Skočí se na následující volnou pozici, 275, a nastaví se na $(‘a’,276,274)$.

	1	...	97	98	...	265	266	267	268	269	270	271	272	...	278	...
symbol			a			a	f	e	s	—	—		t		l	
index rodiče			0			101	108	32	116	102	115		97		97	
index			97			265	266	267	265	269	270		272		278	



LZW - krok 1

- Kodér zahajuje svou činnost tím, že inicializuje slovník (naplní prvních 255 položek ASCII symboly) a přečte první symbol ze vstupního toku.
- Řetězec "a" se nachází ve slovníku na pozici 97.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Slovník
0...255: ASCII

- Řetězec "aa" se ve slovníku nenachází a je proto kóděrem do slovníku přidán a sice na pozici 256.
- Kodér zapíše do výstupního toku značku <97> a posune se o jeden znak doprava.



Výstup

<97> ('a')

Slovník

0...255: ASCII

256 : 'aa'

001100001

L_{OUT}=9 bits

- Řetězec "ca" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 258.
- Kodér zapíše do výstupního toku značku <99> a posune se o jeden znak doprava.



Výstup

<97> ('a')
<97> ('a')
<99> ('c')

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'

- Řetězec "aa" se nachází ve slovníku na pozici 256.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



$L(x)=40, H(x)=1.56$

Výstup

<97> ('a')
<97> ('a')
<99> ('c')

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'

- Řetězec "ca" se nachází ve slovníku na pozici 258.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'
259 : 'aac'

- Řetězec "caa" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 260.
- Kodér zapíše do výstupního toku značku <258> a posune se o jeden znak doprava.



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'
259 : 'aac'
260 : 'caa'

- Řetězec "ac" se nachází ve slovníku na pozici 257.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



L(x)=40, H(x)=1.56

Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'
259 : 'aac'
260 : 'caa'

- Řetězec "aca" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 261.
- Kodér zapíše do výstupního toku značku <257> a posune se o jeden znak doprava.



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<257>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'
259 : 'aac'
260 : 'caa'
261 : 'aca'

LZW - krok 11

- Řetězec "aa" se nachází ve slovníku na pozici 256.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



$L(x)=40$, $H(x)=1.56$

Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<257>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'
259 : 'aac'
260 : 'caa'
261 : 'aca'

LZW - krok 13

- Řetězec "aaca" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 262.
- Kodér zapíše do výstupního toku značku <259> a posune se o jeden znak doprava.



<u>Výstup</u>	<u>Slovník</u>
<97> ('a')	0...255: ASCII
<97> ('a')	256 : 'aa'
<99> ('c')	257 : 'ac'
<256>	258 : 'ca'
<258>	259 : 'aac'
<257>	260 : 'caa'
<259>	261 : 'aca'
	262 : 'aaca'

- Řetězec "aac" se nachází ve slovníku na pozici 259.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



<u>Výstup</u>	<u>Slovník</u>
<97> ('a')	0...255: ASCII
<97> ('a')	256 : 'aa'
<99> ('c')	257 : 'ac'
<256>	258 : 'ca'
<258>	259 : 'aac'
<257>	260 : 'caa'
<259>	261 : 'aca'
	262 : 'aaca'

- Řetězec "aaca" se ve slovníku nenachází a je proto kóděrem do slovníku přidán a sice na pozici 263.
- Kodér zapíše do výstupního toku značku <262> a posune se o jeden znak doprava.



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<257>
<259>
<262>

Slovník

0...255: ASCII 263 : 'aaca'
256 : 'aa'
257 : 'ac'
258 : 'ca'
259 : 'aac'
260 : 'caa'
261 : 'aca'
262 : 'aaca'

- Řetězec "acaa" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 264.
- Kódér zapíše do výstupního toku značku <261> a posune se o jeden znak doprava.



Výstup

<97> ('a') <261>
<97> ('a')
<99> ('c')
<256>
<258>
<257>
<259>
<262>

Slovník

0...255: ASCII	263 : 'aacaa'
256 : 'aa'	264 : 'acaa'
257 : 'ac'	
258 : 'ca'	
259 : 'aac'	
260 : 'caa'	
261 : 'aca'	
262 : 'aaca'	

- Řetězec "aac" se nachází ve slovníku na pozici 259.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<97> ('a') <261>
<97> ('a')
<99> ('c')
<256>
<258>
<257>
<259>
<262>

Slovník

0...255: ASCII	263 : 'aaca'
256 : 'aa'	264 : 'acaa'
257 : 'ac'	
258 : 'ca'	
259 : 'aac'	
260 : 'caa'	
261 : 'aca'	
262 : 'aaca'	

- Řetězec "aaca" se nachází ve slovníku na pozici 262.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



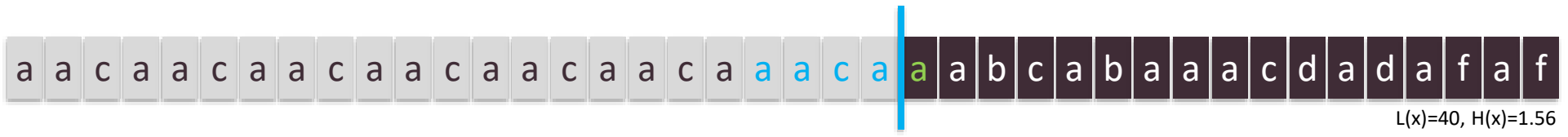
Výstup

<97> ('a') <261>
<97> ('a')
<99> ('c')
<256>
<258>
<257>
<259>
<262>

Slovník

0...255: ASCII 263 : 'aaca'
256 : 'aa' 264 : 'acaa'
257 : 'ac'
258 : 'ca'
259 : 'aac'
260 : 'caa'
261 : 'aca'
262 : 'aaca'

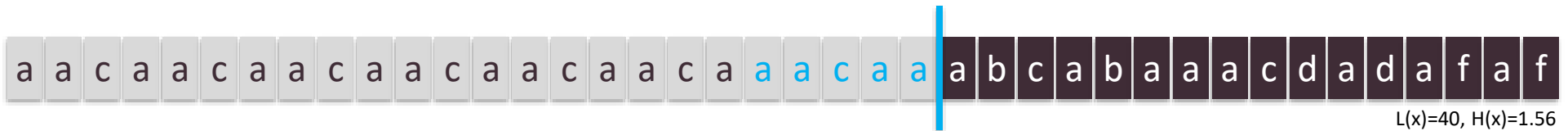
- Řetězec "aaca" se nachází ve slovníku na pozici 263.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup		Slovník	
<97> ('a')	<261>	0...255: ASCII	263 : 'aaca'
<97> ('a')		256 : 'aa'	264 : 'acaa'
<99> ('c')		257 : 'ac'	
<256>		258 : 'ca'	
<258>		259 : 'aac'	
<257>		260 : 'caa'	
<259>		261 : 'aca'	
<262>		262 : 'aaca'	

LZW - krok 25

- Řetězec "aacaaa" se ve slovníku nenachází a je proto kóděrem do slovníku přidán a sice na pozici 265.
- Kodér zapíše do výstupního toku značku <263> a posune se o jeden znak doprava.



Výstup		Slovník	
<97> ('a')	<261>	0...255: ASCII	263 : 'aaciaa'
<97> ('a')	<263>	256 : 'aa'	264 : 'aacia'
<99> ('c')		257 : 'ac'	265 : 'aacaaa'
<256>		258 : 'ca'	
<258>		259 : 'aac'	
<257>		260 : 'caa'	
<259>		261 : 'aca'	
<262>		262 : 'aaca'	

- Řetězec "bc" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 267.
- Kodér zapíše do výstupního toku značku <98> a posune se o jeden znak doprava.



Výstup		Slovník	
<97> ('a')	<261>	0...255: ASCII	263 : 'aaca'
<97> ('a')	<263>	256 : 'aa'	264 : 'acaa'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'aaca'
<256>	<98> ('b')	258 : 'ca'	266 : 'ab'
<258>		259 : 'aac'	267 : 'bc'
<257>		260 : 'caa'	
<259>		261 : 'aca'	
<262>		262 : 'aaca'	

- Řetězec "ca" se nachází ve slovníku na pozici 258.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<97> ('a') <261>
<97> ('a') <263>
<99> ('c') <97> ('a')
<256> <98> ('b')
<258>
<257>
<259>
<262>

Slovník

0...255: ASCII	263 : 'aaca'
256 : 'aa'	264 : 'aca'
257 : 'ac'	265 : 'aaca'
258 : 'ca'	266 : 'ab'
259 : 'aac'	267 : 'bc'
260 : 'caa'	
261 : 'aca'	
262 : 'aaca'	

- Řetězec "cab" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 268.
- Kodér zapíše do výstupního toku značku <258> a posune se o jeden znak doprava.



Výstup

<97> ('a')	<261>
<97> ('a')	<263>
<99> ('c')	<97> ('a')
<256>	<98> ('b')
<258>	<258>
<257>	
<259>	
<262>	

Slovník

0...255: ASCII	263 : 'aaca'
256 : 'aa'	264 : 'acaa'
257 : 'ac'	265 : 'aaca'
258 : 'ca'	266 : 'ab'
259 : 'aac'	267 : 'bc'
260 : 'caa'	268 : 'cab'
261 : 'aca'	
262 : 'aaca'	

00110000100110000100110001110000000010000001010000000110000001110000011010000010110000011100110000100110001010000

L_{OUT}=117 bits

- Řetězec "ba" se ve slovníku nenachází a je proto kóděrem do slovníku přidán a sice na pozici 269.
- Kodér zapíše do výstupního toku značku <98> a posune se o jeden znak doprava.



Výstup		Slovník	
<97> ('a')	<261>	0...255: ASCII	263 : 'aaca'
<97> ('a')	<263>	256 : 'aa'	264 : 'acaa'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'aaca'
<256>	<98> ('b')	258 : 'ca'	266 : 'ab'
<258>	<258>	259 : 'aac'	267 : 'bc'
<257>	<98> ('b')	260 : 'caa'	268 : 'cab'
<259>		261 : 'aca'	269 : 'ba'
<262>		262 : 'aaca'	

- Řetězec "aa" se nachází ve slovníku na pozici 256.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<97> ('a')	<261>
<97> ('a')	<263>
<99> ('c')	<97> ('a')
<256>	<98> ('b')
<258>	<258>
<257>	<98> ('b')
<259>	
<262>	

Slovník

0...255: ASCII	263 : 'aacao'
256 : 'aa'	264 : 'acao'
257 : 'ac'	265 : 'aacaaa'
258 : 'ca'	266 : 'ab'
259 : 'aac'	267 : 'bc'
260 : 'caa'	268 : 'cab'
261 : 'aca'	269 : 'ba'
262 : 'aaca'	

00110000100110000100110001110000000010000001010000000110000001110000011010000010110000011100110000100110001010000

0010001100010

L_{OUT}=126 bits

- Řetězec "aaa" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 270.
- Kodér zapíše do výstupního toku značku <256> a posune se o jeden znak doprava.



Výstup		Slovník	
<97> ('a')	<261>	0...255: ASCII	263 : 'aacaa'
<97> ('a')	<263>	256 : 'aa'	264 : 'acaa'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'aacaaa'
<256>	<98> ('b')	258 : 'ca'	266 : 'ab'
<258>	<258>	259 : 'aac'	267 : 'bc'
<257>	<98> ('b')	260 : 'caa'	268 : 'cab'
<259>	<256>	261 : 'aca'	269 : 'ba'
<262>		262 : 'aaca'	270 : 'aaa'

00110000100110000100110001110000000010000001010000000110000001110000011010000010110000011100110000100110001010000

0010001100010100000000

L_{OUT}=135 bits

- Řetězec "ac" se nachází ve slovníku na pozici 257.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<97> ('a')	<261>
<97> ('a')	<263>
<99> ('c')	<97> ('a')
<256>	<98> ('b')
<258>	<258>
<257>	<98> ('b')
<259>	<256>
<262>	

Slovník

0...255: ASCII	263 : 'aacao'
256 : 'aa'	264 : 'acao'
257 : 'ac'	265 : 'aacaaa'
258 : 'ca'	266 : 'ab'
259 : 'aac'	267 : 'bc'
260 : 'caa'	268 : 'cab'
261 : 'aca'	269 : 'ba'
262 : 'aaca'	270 : 'aaa'

00110000100110000100110001110000000010000001010000000110000001110000011010000010110000011100110000100110001010000
0010001100010100000000

L_{OUT}=135 bits

- Řetězec "acd" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 271.
- Kodér zapíše do výstupního toku značku <257> a posune se o jeden znak doprava.



Výstup		Slovník		
<97> ('a')	<261>	0...255: ASCII	263 : 'aaca'	271 : 'acd'
<97> ('a')	<263>	256 : 'aa'	264 : 'acaa'	
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'aaca'	
<256>	<98> ('b')	258 : 'ca'	266 : 'ab'	
<258>	<258>	259 : 'aac'	267 : 'bc'	
<257>	<98> ('b')	260 : 'caa'	268 : 'cab'	
<259>	<256>	261 : 'aca'	269 : 'ba'	
<262>	<257>	262 : 'aaca'	270 : 'aaa'	

- Řetězec "da" se ve slovníku nenachází a je proto kóděrem do slovníku přidán a sice na pozici 272.
- Kodér zapíše do výstupního toku značku <100> a posune se o jeden znak doprava.



Výstup

<97> ('a')	<261>	<100> ('d')
<97> ('a')	<263>	
<99> ('c')	<97> ('a')	
<256>	<98> ('b')	
<258>	<258>	
<257>	<98> ('b')	
<259>	<256>	
<262>	<257>	

Slovník

0...255: ASCII	263 : 'aacaa'	271 : 'acd'
256 : 'aa'	264 : 'acaa'	272 : 'da'
257 : 'ac'	265 : 'aacaaa'	
258 : 'ca'	266 : 'ab'	
259 : 'aac'	267 : 'bc'	
260 : 'caa'	268 : 'cab'	
261 : 'aca'	269 : 'ba'	
262 : 'aaca'	270 : 'aaa'	

00110000100110000100110001110000000010000001010000000110000001110000011010000010110000011100110000100110001010000

0010001100010100000000100000001001100100

L_{OUT}=153 bits

- Řetězec "ad" se ve slovníku nenachází a je proto kóděrem do slovníku přidán a sice na pozici 273.
- Kodér zapíše do výstupního toku značku <97> a posune se o jeden znak doprava.



Výstup			Slovník		
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'aacaa'	271 : 'acd'
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'acaa'	272 : 'da'
<99> ('c')	<97> ('a')		257 : 'ac'	265 : 'aacaaa'	273 : 'ad'
<256>	<98> ('b')		258 : 'ca'	266 : 'ab'	
<258>	<258>		259 : 'aac'	267 : 'bc'	
<257>	<98> ('b')		260 : 'caa'	268 : 'cab'	
<259>	<256>		261 : 'aca'	269 : 'ba'	
<262>	<257>		262 : 'aaca'	270 : 'aaa'	

- Řetězec "da" se nachází ve slovníku na pozici 272.
- Kodér si řetězec zapamatuje (postačí si pamatovat ukazatel do trie) a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup			Slovník		
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'aacia'	271 : 'acd'
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'acaa'	272 : 'da'
<99> ('c')	<97> ('a')		257 : 'ac'	265 : 'aaciaa'	273 : 'ad'
<256>	<98> ('b')		258 : 'ca'	266 : 'ab'	
<258>	<258>		259 : 'aac'	267 : 'bc'	
<257>	<98> ('b')		260 : 'caa'	268 : 'cab'	
<259>	<256>		261 : 'aca'	269 : 'ba'	
<262>	<257>		262 : 'aaca'	270 : 'aaa'	

00110000100110000100110001110000000010000001010000000110000001110000011010000010110000011100110000100110001010000
0010001100010100000000100000001001100100001100001

L_{OUT}=162 bits

- Řetězec "daf" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 274.
- Kodér zapíše do výstupního toku značku <272> a posune se o jeden znak doprava.



Výstup			Slovník		
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'aaca'	271 : 'acd'
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'acaa'	272 : 'da'
<99> ('c')	<97> ('a')	<272>	257 : 'ac'	265 : 'aaca'	273 : 'ad'
<256>	<98> ('b')		258 : 'ca'	266 : 'ab'	274 : 'daf'
<258>	<258>		259 : 'aac'	267 : 'bc'	
<257>	<98> ('b')		260 : 'caa'	268 : 'cab'	
<259>	<256>		261 : 'aca'	269 : 'ba'	
<262>	<257>		262 : 'aaca'	270 : 'aaa'	

- Řetězec "fa" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 275.
- Kodér zapíše do výstupního toku značku <102> a posune se o jeden znak doprava.



Výstup			Slovník		
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'aaca'	271 : 'acd'
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'acaa'	272 : 'da'
<99> ('c')	<97> ('a')	<272>	257 : 'ac'	265 : 'aaca'	273 : 'ad'
<256>	<98> ('b')	<102> ('f')	258 : 'ca'	266 : 'ab'	274 : 'daf'
<258>	<258>		259 : 'aac'	267 : 'bc'	275 : 'fa'
<257>	<98> ('b')		260 : 'caa'	268 : 'cab'	
<259>	<256>		261 : 'aca'	269 : 'ba'	
<262>	<257>		262 : 'aaca'	270 : 'aaa'	

00110000100110000100110001110000000010000001010000000110000001110000011010000010110000011100110000100110001010000
00100011000101000000001000000010011001000011000011000100000011001100

- Řetězec "af" se ve slovníku nenachází a je proto kódérem do slovníku přidán a sice na pozici 276.
- Kodér zapíše do výstupního toku značku <97> a posune se o jeden znak doprava.



Výstup			Slovník		
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'aaca'	271 : 'acd'
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'acaa'	272 : 'da'
<99> ('c')	<97> ('a')	<272>	257 : 'ac'	265 : 'aaca'	273 : 'ad'
<256>	<98> ('b')	<102> ('f')	258 : 'ca'	266 : 'ab'	274 : 'daf'
<258>	<258>	<97> ('a')	259 : 'aac'	267 : 'bc'	275 : 'fa'
<257>	<98> ('b')		260 : 'caa'	268 : 'cab'	276 : 'af'
<259>	<256>		261 : 'aca'	269 : 'ba'	
<262>	<257>		262 : 'aaca'	270 : 'aaa'	

00110000100110000100110001110000000010000001010000000110000001110000011010000010110000011100110000100110001010000
001000110001010000000010000000100110010000110000110001000000110011000110001

- Kodér končí svou činnost tím, že zapíše na výstup token <102>

a a c a a c a a c a a c a a c a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=1.56

Výstup			Slovník		
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'aacia'	271 : 'acd'
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'acaa'	272 : 'da'
<99> ('c')	<97> ('a')	<272>	257 : 'ac'	265 : 'aacaaa'	273 : 'ad'
<256>	<98> ('b')	<102> ('f')	258 : 'ca'	266 : 'ab'	274 : 'daf'
<258>	<258>	<97> ('a')	259 : 'aac'	267 : 'bc'	275 : 'fa'
<257>	<98> ('b')	<102> ('f')	260 : 'caa'	268 : 'cab'	276 : 'af'
<259>	<256>		261 : 'aca'	269 : 'ba'	
<262>	<257>		262 : 'aaca'	270 : 'aaa'	

00110000100110000100110001110000000010000001010000000110000001110000011010000010110000011100110000100110001010000
001000110001010000000010000000100110010000110000110001000000011001100011000010011001100001001100110000

Komprese pomocí LZW - výstup

- Vstupní řetězec byl zakódován pomocí 198 bitů (4,95 bitů na symbol). Dosažen kompresní faktor 0,618 (198 / 40*8), tj. lepší než LZ78 (0,875) ale horší než v případě LZSS (0,443) i Huffmanova kódu (0,459).
- Poznámka: výstup neobsahuje žádnou hlavičku, parametry kodéru (maximální počet položek slovníku) jsou předány externě. Token má fixní délku: 9b

a a c a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=1.56



data

```
00110000100110000100110001110000000010000001010000000110
00000111000001101000001011000001110011000010011000101000
000100011000101000000000100000001001100100001100001100010
000001100110001100001001100110
```

Varianty LZW

- LZW je adaptivní metoda komprese dat, ale **vstupům se přizpůsobuje pomalu**, protože řetězce v každém okamžiku prodlužují pouze o jeden znak.

Inkrement řetězce o	Přidání řetězce do slovníku / struktura řetězce	
	na frázi	na vstupní symbol
jeden symbol	LZW 	LZY
několik symbolů	LZMW 	LZAP

- Existují tři varianty LZW
 - LZMW (Miller a Wegman, 1985)
 - LZY (Bernstein, 1981), rozšíření LZW
 - LZAP (Storer, 1988), rozšíření LZMW
- Mimo to existuje i metoda LZWL (Lánský, 2005) pracující nikoliv na úrovni symbolů, ale na úrovni slov (přesněji na úrovni slabik, na které je text rozložen).

Metoda LZMW

■ Modifikace zavedené v LZMW

1. Nový řetězec = **spojení předchozí shody F' s aktuální shodou F** . Místo přidání F plus jednoho znaku (LZW) z následující fráze do slovníku, se tím pádem přidá do slovníku spojení dvou frází.
2. Když se slovník zaplní, nejdéle nepoužitá fráze slovníku se vymaže. Prvních 256 frází slovníku se nesmí nikdy vymazat.

■ LZMW se adaptuje na svůj vstup rychleji, než LZW, ale má tyto nevýhody:

- Datová struktura slovníku nemůže být jednoduchý LZW trie, protože všechny prefixy slovníkové fráze ve slovníku nejsou. To znamená, že vyhledávací metoda „jeden symbol v jednom kroku“, používaná v LZW, nebude fungovat. Naopak, když se přidá do slovníku LZMW fráze F , do datové struktury se musí přidat všechny prefixy F a každý uzel v datové struktuře musí mít příznak, zda uzel ve slovníku je, anebo není.
- Vyhledávání nejdelšího řetězce může potřebovat backtracking. Když slovník obsahuje „aaaa“ a „aaaaaaaa“, musíme dosáhnout osmého symbolu ve frázi „aaaaaaab“ abychom si uvědomili, že musíme vybrat kratší frázi.
- Stejný řetězec se může přidat do slovníku dvakrát.

Metoda LZMW

■ Pseudokód kodéru LZMW:

```
inicializace slovníku D = [0,...,255], F' = "", i=0
while (i < délka vstupu) do
begin
    F = nejdelší shoda na vstupu nacházející se v D
    ulož značku <i>, kde i je pozice F v D
    if F'+F ∉ D then
        přidej řetězec F'+F do slovníku
    end
    posuň okno o délku shody, tj. i += len(F)
    nastav F' = F
end
```

■ Pseudokód dekodéru LZMW:

```
inicializace slovníku D = [0,...,255], F' = ""
while (nepřečten celý vstupní tok) do
begin
    načti značku <i> a jí odpovídající řetězec ze slovníku F = D[i]
    zapiš do výstupního toku řetězec F
    if F'+F ∉ D then
        přidej řetězec F'+F do slovníku
    end
    nastav F' = F
end
```



- Nejdelší shoda, která se nachází ve slovníku je řetězec "a" uložený na pozici 97.
- Kodér zapíše na výstup token <97>, posune okno o 1 znak doprava .



Výstup
<97> ('a')

Slovník
0...255: ASCII

- Nejdelší shoda, která se nachází ve slovníku je řetězec "a" uložený na pozici 97.
- Kodér zapíše na výstup token <97>, posune okno o 1 znak doprava a přidá do slovníku řetězec "aa" na pozici 256.



Výstup

<97> ('a')

<97> ('a')

Slovník

0...255: ASCII

256 : 'aa'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "aa" uložený na pozici 256.
- Kodér zapíše na výstup token <256>, posune okno o 2 znaky doprava a přidá do slovníku řetězec "caa" na pozici 258.



Výstup

<97> ('a')
 <97> ('a')
 <99> ('c')
 <256>

Slovník

0...255: ASCII
 256 : 'aa'
 257 : 'ac'
 258 : 'caa'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "caacaa" uložený na pozici 260.
- Kodér zapíše na výstup token <260>, posune okno o 6 znaků doprava a přidá do slovníku řetězec "caacaacaa" na pozici 261.



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<258>
<260>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'caa'
259 : 'aacaa'
260 : 'caacaa'
261 : 'caacaacaa'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "caa" uložený na pozici 258.
- Kodér zapíše na výstup token <258>, posune okno o 3 znaky doprava ale řetězec "caacaacaa" do slovníku nepřidává neboť se již nachází na pozici 261.



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<258>
<260>
<258>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'caa'
259 : 'aaca'
260 : 'caaca'
261 : 'caacaacaa'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "ac" uložený na pozici 257.
- Kodér zapíše na výstup token <257>, posune okno o 2 znaky doprava a přidá do slovníku řetězec "caaac" na pozici 262.



Výstup

<97> ('a') <257>
<97> ('a')
<99> ('c')
<256>
<258>
<258>
<260>
<258>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'caa'
259 : 'aaca'
260 : 'caaca'
261 : 'caacaaca'
262 : 'caaac'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "aa" uložený na pozici 256.
- Kodér zapíše na výstup token <256>, posune okno o 2 znaky doprava a přidá do slovníku řetězec "acaa" na pozici 263.



Výstup		Slovník	
<97> ('a')	<257>	0...255: ASCII	263 : 'acaa'
<97> ('a')	<256>	256 : 'aa'	
<99> ('c')		257 : 'ac'	
<256>		258 : 'caa'	
<258>		259 : 'aacia'	
<258>		260 : 'caacia'	
<260>		261 : 'caaciaa'	
<258>		262 : 'caaac'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "a" uložený na pozici 97.
- Kodér zapíše na výstup token <97>, posune okno o 1 znak doprava a přidá do slovníku řetězec "aaa" na pozici 264.



Výstup		Slovník	
<97> ('a')	<257>	0...255: ASCII	263 : 'acaa'
<97> ('a')	<256>	256 : 'aa'	264 : 'aaa'
<99> ('c')	<97> ('a')	257 : 'ac'	
<256>		258 : 'caa'	
<258>		259 : 'aaciaa'	
<258>		260 : 'caaciaa'	
<260>		261 : 'caaciaaciaa'	
<258>		262 : 'caaac'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "b" uložený na pozici 98.
- Kodér zapíše na výstup token <98>, posune okno o 1 znak doprava a přidá do slovníku řetězec "ab" na pozici 265.



Výstup		Slovník	
<97> ('a')	<257>	0...255: ASCII	263 : 'acaa'
<97> ('a')	<256>	256 : 'aa'	264 : 'aaa'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'ab'
<256>	<98> ('b')	258 : 'caa'	
<258>		259 : 'aaciaa'	
<258>		260 : 'caaciaa'	
<260>		261 : 'caaciaaia'	
<258>		262 : 'caaac'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "c" uložený na pozici 99.
- Kodér zapíše na výstup token <99>, posune okno o 1 znak doprava a přidá do slovníku řetězec "bc" na pozici 266.



Výstup		Slovník	
<97> ('a')	<257>	0...255: ASCII	263 : 'acaa'
<97> ('a')	<256>	256 : 'aa'	264 : 'aaa'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'ab'
<256>	<98> ('b')	258 : 'caa'	266 : 'bc'
<258>	<99> ('c')	259 : 'aaciaa'	
<258>		260 : 'caaciaa'	
<260>		261 : 'caaciaaaa'	
<258>		262 : 'caaac'	

00110000100110000100110001110000000010000001010000001010000010010000001010000000110000000000110000100110001000110

- Nejdelší shoda, která se nachází ve slovníku je řetězec "ab" uložený na pozici 265.
- Kodér zapíše na výstup token <265>, posune okno o 2 znaky doprava a přidá do slovníku řetězec "cab" na pozici 267.



Výstup		Slovník	
<97> ('a')	<257>	0...255: ASCII	263 : 'acaa'
<97> ('a')	<256>	256 : 'aa'	264 : 'aaa'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'ab'
<256>	<98> ('b')	258 : 'caa'	266 : 'bc'
<258>	<99> ('c')	259 : 'aacao'	267 : 'cab'
<258>	<265>	260 : 'caacao'	
<260>		261 : 'caacaacao'	
<258>		262 : 'caaac'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "aaa" uložený na pozici 264.
- Kodér zapíše na výstup token <264>, posune okno o 3 znaky doprava a přidá do slovníku řetězec "abaaa" na pozici 268.



Výstup

<97> ('a')	<257>
<97> ('a')	<256>
<99> ('c')	<97> ('a')
<256>	<98> ('b')
<258>	<99> ('c')
<258>	<265>
<260>	<264>
<258>	

Slovník

0...255: ASCII	263 : 'acaa'
256 : 'aa'	264 : 'aaa'
257 : 'ac'	265 : 'ab'
258 : 'caa'	266 : 'bc'
259 : 'aacao'	267 : 'cab'
260 : 'caacao'	268 : 'abaaa'
261 : 'caacaacao'	
262 : 'caaac'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "c" uložený na pozici 99.
- Kodér zapíše na výstup token <99>, posune okno o 1 znak doprava a přidá do slovníku řetězec "aaac" na pozici 269.



Výstup		Slovník	
<97> ('a')	<257>	0...255: ASCII	263 : 'acaa'
<97> ('a')	<256>	256 : 'aa'	264 : 'aaa'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'ab'
<256>	<98> ('b')	258 : 'caa'	266 : 'bc'
<258>	<99> ('c')	259 : 'aacao'	267 : 'cab'
<258>	<265>	260 : 'caacao'	268 : 'abaaa'
<260>	<264>	261 : 'caacaacao'	269 : 'aaac'
<258>	<99> ('c')	262 : 'caaac'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "d" uložený na pozici 100.
- Kodér zapíše na výstup token <100>, posune okno o 1 znak doprava a přidá do slovníku řetězec "cd" na pozici 270.



Výstup			Slovník	
<97> ('a')	<257>	<100> ('d')	0...255: ASCII	263 : 'acaa'
<97> ('a')	<256>		256 : 'aa'	264 : 'aaa'
<99> ('c')	<97> ('a')		257 : 'ac'	265 : 'ab'
<256>	<98> ('b')		258 : 'caa'	266 : 'bc'
<258>	<99> ('c')		259 : 'aaciaa'	267 : 'cab'
<258>	<265>		260 : 'caaciaa'	268 : 'abaaa'
<260>	<264>		261 : 'caaciaa'	269 : 'aac'
<258>	<99> ('c')		262 : 'caaac'	270 : 'cd'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "a" uložený na pozici 97.
- Kodér zapíše na výstup token <97>, posune okno o 1 znak doprava a přidá do slovníku řetězec "da" na pozici 271.



Výstup			Slovník		
<97> ('a')	<257>	<100> ('d')	0...255: ASCII	263 : 'acaa'	271 : 'da'
<97> ('a')	<256>	<97> ('a')	256 : 'aa'	264 : 'aaa'	
<99> ('c')	<97> ('a')		257 : 'ac'	265 : 'ab'	
<256>	<98> ('b')		258 : 'caa'	266 : 'bc'	
<258>	<99> ('c')		259 : 'aaciaa'	267 : 'cab'	
<258>	<265>		260 : 'caaciaa'	268 : 'abaaa'	
<260>	<264>		261 : 'caaciaaaa'	269 : 'aaac'	
<258>	<99> ('c')		262 : 'caaac'	270 : 'cd'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "da" uložený na pozici 271.
- Kodér zapíše na výstup token <271>, posune okno o 2 znaky doprava a přidá do slovníku řetězec "ada" na pozici 272.



Výstup			Slovník		
<97> ('a')	<257>	<100> ('d')	0...255: ASCII	263 : 'acaa'	271 : 'da'
<97> ('a')	<256>	<97> ('a')	256 : 'aa'	264 : 'aaa'	272 : 'ada'
<99> ('c')	<97> ('a')	<271>	257 : 'ac'	265 : 'ab'	
<256>	<98> ('b')		258 : 'caa'	266 : 'bc'	
<258>	<99> ('c')		259 : 'aaciaa'	267 : 'cab'	
<258>	<265>		260 : 'caaciaa'	268 : 'abaaa'	
<260>	<264>		261 : 'caacaaciaa'	269 : 'aaac'	
<258>	<99> ('c')		262 : 'caaac'	270 : 'cd'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "f" uložený na pozici 102.
- Kodér zapíše na výstup token <102>, posune okno o 1 znak doprava a přidá do slovníku řetězec "daf" na pozici 273.



Výstup			Slovník		
<97> ('a')	<257>	<100> ('d')	0...255: ASCII	263 : 'acaa'	271 : 'da'
<97> ('a')	<256>	<97> ('a')	256 : 'aa'	264 : 'aaa'	272 : 'ada'
<99> ('c')	<97> ('a')	<271>	257 : 'ac'	265 : 'ab'	273 : 'daf'
<256>	<98> ('b')	<102> ('f')	258 : 'caa'	266 : 'bc'	
<258>	<99> ('c')		259 : 'aacao'	267 : 'cab'	
<258>	<265>		260 : 'caacao'	268 : 'abaaa'	
<260>	<264>		261 : 'caacaacao'	269 : 'aac'	
<258>	<99> ('c')		262 : 'caaac'	270 : 'cd'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "a" uložený na pozici 97.
- Kodér zapíše na výstup token <97>, posune okno o 1 znak doprava a přidá do slovníku řetězec "fa" na pozici 274.



Výstup			Slovník		
<97> ('a')	<257>	<100> ('d')	0...255: ASCII	263 : 'acaa'	271 : 'da'
<97> ('a')	<256>	<97> ('a')	256 : 'aa'	264 : 'aaa'	272 : 'ada'
<99> ('c')	<97> ('a')	<271>	257 : 'ac'	265 : 'ab'	273 : 'daf'
<256>	<98> ('b')	<102> ('f')	258 : 'caa'	266 : 'bc'	274 : 'fa'
<258>	<99> ('c')	<97> ('a')	259 : 'aaciaa'	267 : 'cab'	
<258>	<265>		260 : 'caaciaa'	268 : 'abaaa'	
<260>	<264>		261 : 'caaciaaaa'	269 : 'aaac'	
<258>	<99> ('c')		262 : 'caaac'	270 : 'cd'	

00110000100110000100110001110000000010000001010000001010000010010000001010000000110000000000110000100110001000110
001110000100110000100000110001100110010000110000110000111100110011000110001

- V posledním kroku kodér nemůže nalézt shodu delší než-li jeden symbol, neboť na vstupu se více znaků nenachází. Na výstup proto zapíše token <102> a svou činnost končí.



Výstup

<97> ('a')	<257>	<100> ('d')
<97> ('a')	<256>	<97> ('a')
<99> ('c')	<97> ('a')	<271>
<256>	<98> ('b')	<102> ('f')
<258>	<99> ('c')	<97> ('a')
<258>	<265>	<102> ('f')
<260>	<264>	
<258>	<99> ('c')	

Slovník

0...255: ASCII	263 : 'acaa'	271 : 'da'
256 : 'aa'	264 : 'aaa'	272 : 'ada'
257 : 'ac'	265 : 'ab'	273 : 'daf'
258 : 'caa'	266 : 'bc'	274 : 'fa'
259 : 'aaciaa'	267 : 'cab'	275 : 'af'
260 : 'caaciaa'	268 : 'abaaa'	
261 : 'caaciaaaa'	269 : 'aaac'	
262 : 'caaac'	270 : 'cd'	

00110000100110000100110001110000000010000001010000001010000010010000001010000000110000000000110000100110001000110
0011100001001100001000001100011001100100001100001100001111001100110001100001001100110

L_{OUT}=198 bits

Komprese pomocí LZMW - výstup

- Vstupní řetězec byl zakódován pomocí 198 bitů (4,95 bitů na symbol). Dosažen kompresní faktor 0,618 (198 / 40*8), tj. stejný jako LZW, lepší než LZ78 (0,875) ale horší než v případě LZSS (0,443) i Huffmanova kódu (0,459).
- Poznámka: výstup neobsahuje žádnou hlavičku, parametry kodéru (maximální počet položek slovníku) jsou předány externě. Token má fixní délku: 9b

a a c a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=1.56



data

```
00110000100110000100110001110000000010000001010000001010
00001001000000101000000011000000000011000010011000100011
00011100001001100001000001100011001100100001100001100001
111001100110001100001001100110
```

Metoda LZAP

- LZAP (James Storer, 1988) je modifikací LZMW spočívající v tom, že namísto přidávání jednoho řetězce $F'+F$ se **do slovníku přidávají postupně všechny řetězce $F'+P$** , přičemž $P \in \text{Prefix}(F)$ (odtud označení metody – All Prefixes).
- **Příklad:**
 - předpokládejme předchozí shodu $F'="com"$ a aktuální shodu $F="press"$
 - kodér přidá do slovníku postupně 5 nových řetězců: "comp", "compr", "compre", "compres" a "compress"
- **Vlastnosti:**
 - LZAP přidává do slovníku více frází, než LZMW, takže pro reprezentaci pozice fráze je zapotřebí více bitů.
 - LZAP také poskytuje pro shodu se vstupním řetězcem větší výběr slovníkových frází, takže končí kompresi mírně lépe, než LZMW.
 - LZAP je paradoxně rychlejší než LZMW díky použití odlišné datové struktury slovníku, která eliminuje backtracking.

Metoda LZAP

■ Pseudokód kodéru LZAP:

```
inicializace slovníku D =[0,...,255], F' = "", i=0
while (i < délka vstupu) do
  begin
    F = nejdelší shoda na vstupu nacházející se v D
    ulož značku <i>, kde i je pozice F v D
    for (P ∈ Prefix(F)) begin
      if F'+P ∉ D then
        přidej řetězec F'+P do slovníku
      end
    end
    posuň okno o délku shody, tj. i += len(F)
    nastav F' = F
  end
```

LZAP přidává do slovníku všechny prefixy na rozdíl od LZMW, které zde přidávalo pouze F' + F

LZAP - krok 1

- Nejdelší shoda, která se nachází ve slovníku je řetězec "a" uložený na pozici 97.
- Kodér zapíše na výstup token <97>, posune okno o 1 znak doprava .



Výstup
<97>

Slovník
0...255: ASCII

001100001
 $L_{OUT}=9$ bits

- Nejdelší shoda, která se nachází ve slovníku je řetězec "a" uložený na pozici 97.
- Kodér zapíše na výstup token <97>, posune okno o 1 znak doprava a přidá do slovníku řetězec "aa".



Výstup

<97>

<97>

Slovník

0...255: ASCII

256 : 'aa'

001100001001100001

L_{OUT}=18 bits

- Nejdelší shoda, která se nachází ve slovníku je řetězec "c" uložený na pozici 99.
- Kodér zapíše na výstup token <99>, posune okno o 1 znak doprava a přidá do slovníku řetězec "ac".



Výstup

<97>

<97>

<99>

Slovník

0...255: ASCII

256 : 'aa'

257 : 'ac'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "aa" uložený na pozici 256.
- Kodér zapíše na výstup token <256>, posune okno o 2 znaky doprava a přidá do slovníku řetězce "ca","caa".



Výstup

<97>
<97>
<99>
<256>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'
259 : 'caa'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "caa" uložený na pozici 259.
- Kodér zapíše na výstup token <259>, posune okno o 3 znaky doprava a přidá do slovníku řetězce "aac","aaca","aacao".



$L(x)=40$, $H(x)=1.56$

Výstup

<97>
<97>
<99>
<256>
<259>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'
259 : 'caa'
260 : 'aac'
261 : 'aaca'
262 : 'aacao'

LZAP - krok 6

- Nejdelší shoda, která se nachází ve slovníku je řetězec "caa" uložený na pozici 259.
- Kodér zapíše na výstup token <259>, posune okno o 3 znaky doprava a přidá do slovníku řetězce "caac", "caaca", "caacaa".



Výstup

<97>

<97>

<99>

<256>

<259>

<259>

Slovník

0...255: ASCII 263 : 'caac'

256 : 'aa' 264 : 'caaca'

257 : 'ac' 265 : 'caacaa'

258 : 'ca'

259 : 'caa'

260 : 'aac'

261 : 'aaca'

262 : 'aacaa'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "caacaa" uložený na pozici 265.
- Kodér zapíše na výstup token <265>, posune okno o 6 znaků doprava a přidá do slovníku řetězce "caacaac", "caacaaca", "caacaacaa".



<u>Výstup</u>	<u>Slovník</u>
<97>	0...255: ASCII 263 : 'caac'
<97>	256 : 'aa' 264 : 'caaca'
<99>	257 : 'ac' 265 : 'caacaa'
<256>	258 : 'ca' 266 : 'caacaac'
<259>	259 : 'caa' 267 : 'caacaaca'
<259>	260 : 'aac' 268 : 'caacaacaa'
<265>	261 : 'aaca'
	262 : 'aacaa'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "ac" uložený na pozici 257.
- Kodér zapíše na výstup token <257>, posune okno o 2 znaky doprava a přidá do slovníku řetězce "caaa", "caaac".



Výstup		Slovník	
<97>	<257>	0...255: ASCII	263 : 'caac'
<97>		256 : 'aa'	264 : 'caaca'
<99>		257 : 'ac'	265 : 'caacaa'
<256>		258 : 'ca'	266 : 'caacaac'
<259>		259 : 'caa'	267 : 'caacaaca'
<259>		260 : 'aac'	268 : 'caacaacaa'
<265>		261 : 'aaca'	269 : 'caaa'
<259>		262 : 'aacaa'	270 : 'caaac'

- Nejdelší shoda, která se nachází ve slovníku je řetězec "aa" uložený na pozici 256.
- Kodér zapíše na výstup token <256>, posune okno o 2 znaky doprava a přidá do slovníku řetězce "aca","acaa".



L(x)=40, H(x)=1.56

Výstup

<97> <257>
<97> <256>
<99>
<256>
<259>
<259>
<265>
<259>

Slovník

0...255: ASCII	263 : 'caac'	271 : 'aca'
256 : 'aa'	264 : 'caaca'	272 : 'acaa'
257 : 'ac'	265 : 'caacaa'	
258 : 'ca'	266 : 'caacaac'	
259 : 'caa'	267 : 'caacaaca'	
260 : 'aac'	268 : 'caacaacaa'	
261 : 'aaca'	269 : 'caaaa'	
262 : 'aacaa'	270 : 'caaac'	

LZAP - krok 11

- Nejdelší shoda, která se nachází ve slovníku je řetězec "a" uložený na pozici 97.
- Kodér zapíše na výstup token <97>, posune okno o 1 znak doprava a přidá do slovníku řetězec "aaa".



Výstup		Slovník	
<97>	<257>	0...255: ASCII	263 : 'caac' 271 : 'aca'
<97>	<256>	256 : 'aa'	264 : 'caaca' 272 : 'acaa'
<99>	<97>	257 : 'ac'	265 : 'caacaa' 273 : 'aaa'
<256>		258 : 'ca'	266 : 'caacaac'
<259>		259 : 'caa'	267 : 'caacaaca'
<259>		260 : 'aac'	268 : 'caacaacaa'
<265>		261 : 'aaca'	269 : 'caaaa'
<259>		262 : 'aacaa'	270 : 'caaac'

0011000010011000010011000111000000001000000111000000111000010011000000111000000011000000000010000000000010000001
L_{OUT}=99 bits

- Nejdelší shoda, která se nachází ve slovníku je řetězec "b" uložený na pozici 98.
- Kodér zapíše na výstup token <98>, posune okno o 1 znak doprava a přidá do slovníku řetězec "ab".



Výstup		Slovník		
<97>	<257>	0...255: ASCII	263 : 'caac'	271 : 'aca'
<97>	<256>	256 : 'aa'	264 : 'caaca'	272 : 'acaa'
<99>	<97>	257 : 'ac'	265 : 'caacaa'	273 : 'aaa'
<256>	<98>	258 : 'ca'	266 : 'caacaac'	274 : 'ab'
<259>		259 : 'caa'	267 : 'caacaaca'	
<259>		260 : 'aac'	268 : 'caacaacaa'	
<265>		261 : 'aaca'	269 : 'caaa'	
<259>		262 : 'aacaa'	270 : 'caaac'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "ca" uložený na pozici 258.
- Kodér zapíše na výstup token <258>, posune okno o 2 znaky doprava a přidá do slovníku řetězce "bc","bca".



Výstup

<97>	<257>
<97>	<256>
<99>	<97>
<256>	<98>
<259>	<258>
<259>	
<265>	
<259>	

Slovník

0...255: ASCII	263 : 'caac'	271 : 'aca'
256 : 'aa'	264 : 'caaca'	272 : 'acaa'
257 : 'ac'	265 : 'caacaa'	273 : 'aaa'
258 : 'ca'	266 : 'caacaac'	274 : 'ab'
259 : 'caa'	267 : 'caacaaca'	275 : 'bc'
260 : 'aac'	268 : 'caacaacaa'	276 : 'bca'
261 : 'aaca'	269 : 'caaaa'	
262 : 'aacaa'	270 : 'caaac'	

0011000010011000010011000111000000001000000111000000111000010011000000111000000011000000000110000100110001010000

0010

L_{OUT}=117 bits

- Nejdelší shoda, která se nachází ve slovníku je řetězec "b" uložený na pozici 98.
- Kodér zapíše na výstup token <98>, posune okno o 1 znak doprava a přidá do slovníku řetězec "cab".



Výstup		Slovník		
<97>	<257>	0...255: ASCII	263 : 'caac'	271 : 'aca'
<97>	<256>	256 : 'aa'	264 : 'caaca'	272 : 'acaa'
<99>	<97>	257 : 'ac'	265 : 'caacaa'	273 : 'aaa'
<256>	<98>	258 : 'ca'	266 : 'caacaac'	274 : 'ab'
<259>	<258>	259 : 'caa'	267 : 'caacaaca'	275 : 'bc'
<259>	<98>	260 : 'aac'	268 : 'caacaacaa'	276 : 'bca'
<265>		261 : 'aaca'	269 : 'caaa'	277 : 'cab'
<259>		262 : 'aacaa'	270 : 'caaac'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "aaa" uložený na pozici 273.
- Kodér zapíše na výstup token <273>, posune okno o 3 znaky doprava a přidá do slovníku řetězce "ba", "baa", "baaa".



Výstup		Slovník			
<97>	<257>	0...255: ASCII	263 : 'caac'	271 : 'aca'	279 : 'baa'
<97>	<256>	256 : 'aa'	264 : 'caaca'	272 : 'acaa'	280 : 'baaa'
<99>	<97>	257 : 'ac'	265 : 'caacaa'	273 : 'aaa'	
<256>	<98>	258 : 'ca'	266 : 'caacaac'	274 : 'ab'	
<259>	<258>	259 : 'caa'	267 : 'caacaaca'	275 : 'bc'	
<259>	<98>	260 : 'aac'	268 : 'caacaacaa'	276 : 'bca'	
<265>	<273>	261 : 'aaca'	269 : 'caaa'	277 : 'cab'	
<259>		262 : 'aacaa'	270 : 'caaac'	278 : 'ba'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "c" uložený na pozici 99.
- Kodér zapíše na výstup token <99>, posune okno o 1 znak doprava a přidá do slovníku řetězec "aaac".



Výstup

<97>	<257>
<97>	<256>
<99>	<97>
<256>	<98>
<259>	<258>
<259>	<98>
<265>	<273>
<259>	<99>

Slovník

0...255: ASCII	263 : 'caac'	271 : 'aca'	279 : 'baa'
256 : 'aa'	264 : 'caaca'	272 : 'acaa'	280 : 'baaa'
257 : 'ac'	265 : 'caacaa'	273 : 'aaa'	281 : 'aaac'
258 : 'ca'	266 : 'caacaac'	274 : 'ab'	
259 : 'caa'	267 : 'caacaaca'	275 : 'bc'	
260 : 'aac'	268 : 'caacaacaa'	276 : 'bca'	
261 : 'aaca'	269 : 'caaaa'	277 : 'cab'	
262 : 'aacaa'	270 : 'caaac'	278 : 'ba'	

00110000100110000100110001110000000010000001110000001110000100110000001110000000110000000000110000100110001010000

00100011000101000100010011001100011

L_{OUT}=144 bits

-
- a a c a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f
- $L(x)=40, H(x)=1.56$

<97>	<257>	<100>
<97>	<256>	
<99>	<97>	
<256>	<98>	
<259>	<258>	
<259>	<98>	
<265>	<273>	
<259>	<99>	

0...255: ASCII	263 : 'caac'	271 : 'aca'	279 : 'baa'
256 : 'aa'	264 : 'caaca'	272 : 'acaa'	280 : 'baaa'
257 : 'ac'	265 : 'caacaa'	273 : 'aaa'	281 : 'aaac'
258 : 'ca'	266 : 'caacaac'	274 : 'ab'	282 : 'cd'
259 : 'caa'	267 : 'caacaaca'	275 : 'bc'	
260 : 'aac'	268 : 'caacaacaa'	276 : 'bca'	
261 : 'aaca'	269 : 'caaa'	277 : 'cab'	
262 : 'aacaa'	270 : 'caaac'	278 : 'ba'	

$$L_{OUT}=153 \text{ bits}$$

- Nejdelší shoda, která se nachází ve slovníku je řetězec "a" uložený na pozici 97.
- Kodér zapíše na výstup token <97>, posune okno o 1 znak doprava a přidá do slovníku řetězec "da".



Výstup			Slovník			
<97>	<257>	<100>	0...255: ASCII	263 : 'caac'	271 : 'aca'	279 : 'baa'
<97>	<256>	<97>	256 : 'aa'	264 : 'caaca'	272 : 'acaa'	280 : 'baaa'
<99>	<97>		257 : 'ac'	265 : 'caacaa'	273 : 'aaa'	281 : 'aaac'
<256>	<98>		258 : 'ca'	266 : 'caacaac'	274 : 'ab'	282 : 'cd'
<259>	<258>		259 : 'caa'	267 : 'caacaaca'	275 : 'bc'	283 : 'da'
<259>	<98>		260 : 'aac'	268 : 'caacaacaa'	276 : 'bca'	
<265>	<273>		261 : 'aaca'	269 : 'caaa'	277 : 'cab'	
<259>	<99>		262 : 'aacaa'	270 : 'caaac'	278 : 'ba'	

00110000100110000100110001110000000010000001110000001110000100110000001110000000110000000000110000100110001010000
0010001100010100010001001100011001100100001001100001

- Nejdelší shoda, která se nachází ve slovníku je řetězec "da" uložený na pozici 283.
- Kodér zapíše na výstup token <283>, posune okno o 2 znaky doprava a přidá do slovníku řetězce "ad","ada".



Výstup			Slovník			
<97>	<257>	<100>	0...255: ASCII	263 : 'caac'	271 : 'aca'	279 : 'baa'
<97>	<256>	<97>	256 : 'aa'	264 : 'caaca'	272 : 'acaa'	280 : 'baaa'
<99>	<97>	<283>	257 : 'ac'	265 : 'caacaa'	273 : 'aaa'	281 : 'aaac'
<256>	<98>		258 : 'ca'	266 : 'caacaac'	274 : 'ab'	282 : 'cd'
<259>	<258>		259 : 'caa'	267 : 'caacaaca'	275 : 'bc'	283 : 'da'
<259>	<98>		260 : 'aac'	268 : 'caacaacaa'	276 : 'bca'	284 : 'ad'
<265>	<273>		261 : 'aaca'	269 : 'caaa'	277 : 'cab'	285 : 'ada'
<259>	<99>		262 : 'aacaa'	270 : 'caaac'	278 : 'ba'	

- Nejdelší shoda, která se nachází ve slovníku je řetězec "f" uložený na pozici 102.
- Kodér zapíše na výstup token <102>, posune okno o 1 znak doprava a přidá do slovníku řetězec "daf".



Výstup			Slovník			
<97>	<257>	<100>	0...255: ASCII	263 : 'caac'	271 : 'aca'	279 : 'baa'
<97>	<256>	<97>	256 : 'aa'	264 : 'caaca'	272 : 'acaa'	280 : 'baaa'
<99>	<97>	<283>	257 : 'ac'	265 : 'caacaa'	273 : 'aaa'	281 : 'aaac'
<256>	<98>	<102>	258 : 'ca'	266 : 'caacaac'	274 : 'ab'	282 : 'cd'
<259>	<258>		259 : 'caa'	267 : 'caacaaca'	275 : 'bc'	283 : 'da'
<259>	<98>		260 : 'aac'	268 : 'caacaacaa'	276 : 'bca'	284 : 'ad'
<265>	<273>		261 : 'aaca'	269 : 'caaa'	277 : 'cab'	285 : 'ada'
<259>	<99>		262 : 'aacaa'	270 : 'caaac'	278 : 'ba'	286 : 'daf'

00110000100110000100110001110000000010000001110000001110000100110000001110000000110000000000110000100110001010000
0010001100010100010001001100011001100100001100001100011011001100110

- Nejdelší shoda, která se nachází ve slovníku je řetězec "a" uložený na pozici 97.
- Kodér zapíše na výstup token <97>, posune okno o 1 znak doprava a přidá do slovníku řetězec "fa".



Výstup			Slovník				
<97>	<257>	<100>	0...255: ASCII	263 : 'caac'	271 : 'aca'	279 : 'baa'	287 : 'fa'
<97>	<256>	<97>	256 : 'aa'	264 : 'caaca'	272 : 'acaa'	280 : 'baaa'	
<99>	<97>	<283>	257 : 'ac'	265 : 'caacaa'	273 : 'aaa'	281 : 'aaac'	
<256>	<98>	<102>	258 : 'ca'	266 : 'caacaac'	274 : 'ab'	282 : 'cd'	
<259>	<258>	<97>	259 : 'caa'	267 : 'caacaaca'	275 : 'bc'	283 : 'da'	
<259>	<98>		260 : 'aac'	268 : 'caacaacaa'	276 : 'bca'	284 : 'ad'	
<265>	<273>		261 : 'aaca'	269 : 'caaa'	277 : 'cab'	285 : 'ada'	
<259>	<99>		262 : 'aacaa'	270 : 'caaac'	278 : 'ba'	286 : 'daf'	

00110000100110000100110001110000000010000001110000001110000100110000001110000000110000000000110000100110001010000
00100011000101000100010011000110011001000011000011000110110011001100011000110001

- V posledním kroku kodér nemůže nalézt shodu delší než-li jeden symbol, neboť na vstupu se více znaků nenachází. Na výstup proto zapíše token <102> a svou činnost končí.



Výstup			Slovník				
<97>	<257>	<100>	0...255: ASCII	263 : 'caac'	271 : 'aca'	279 : 'baa'	287 : 'fa'
<97>	<256>	<97>	256 : 'aa'	264 : 'caaca'	272 : 'acaa'	280 : 'baaa'	288 : 'af'
<99>	<97>	<283>	257 : 'ac'	265 : 'caacaa'	273 : 'aaa'	281 : 'aaac'	
<256>	<98>	<102>	258 : 'ca'	266 : 'caacaac'	274 : 'ab'	282 : 'cd'	
<259>	<258>	<97>	259 : 'caa'	267 : 'caacaaca'	275 : 'bc'	283 : 'da'	
<259>	<98>	<102>	260 : 'aac'	268 : 'caacaacaa'	276 : 'bca'	284 : 'ad'	
<265>	<273>		261 : 'aaca'	269 : 'caaa'	277 : 'cab'	285 : 'ada'	
<259>	<99>		262 : 'aacaa'	270 : 'caaac'	278 : 'ba'	286 : 'daf'	

00110000100110000100110001110000000010000001110000001110000100110000001110000000110000000000110000100110001010000
0010001100010100010001001100011001100100001100001100011011001100110001100001001100110

L_{OUT}=198 bits

Komprese pomocí LZAP - výstup

- Vstupní řetězec byl zakódován pomocí 198 bitů (4,95 bitů na symbol). Dosažen kompresní faktor 0,618 (198/ 40*8), tj. stejný jako LZW, ale lepší než LZ78 (0,875) ale horší než v případě LZSS (0,443) i Huffmanova kódu (0,459).
- Poznámka: výstup neobsahuje žádnou hlavičku, parametry kodéru (maximální počet položek slovníku) jsou předány externě. Token má fixní délku: 9b

a a c a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=1.56



data

```
00110000100110000100110001110000000010000001110000001110
00010011000000111000000011000000000011000010011000101000
00010001100010100010001001100011001100100001100001100011
011001100110001100001001100110
```


Metoda LZY – informativní

- LZY (Bernstein, 1991) je modifikací LZW (Y je odvozeno od „Yabba“ pocházejícího ze vstupního řetězce, který se používal k demonstraci činnosti algoritmu).
- Princip (přesněji viz pseudokód):
 - Pro každý symbol X vstupního toku kodér hledá nejdelší řetězec P, který předchází před X a který se nachází ve slovníku (P může být prázdný řetězec).
 - Následně přidá do slovníku novou frázi P+X.
- Protože slovník se aktualizuje rychleji (při každém přečteném symbolu) než-li se produkují tokeny na výstup, mohlo by dojít k chybě díky předčasnému použití položky slovníku. Je proto nutné dočasně držet nové položky slovníku v odděleném seznamu a do slovníku je přidat až při zápisu tokenu na výstup.

Metoda LZY – pseudokód kodéru LZY

inicializace slovníku $D = [0, \dots, 255]$, pomocného slovníku $D' = []$ a řetězců shod $F = ""$, $F' = ""$

while (nepřečten celý vstupní tok) do

begin

načti symbol X

if $F+X \in D$ **then**

přidej X k řetězci se shodou $F = F + X$ a poznač si pozici $i = \text{pozice } F \text{ v } D$

else

ulož značku $\langle i \rangle$

přidej obsah D' do slovníku D

odstraň vše z D' , tj. $D' = []$

$F = X$, $i = \text{ord}(X)$

end

while ($F' + X \notin D$) and ($F' + X \notin D'$) **do**

begin

přidej $F' + X$ do pomocného slovníku D'

odstraň první znak z F' , tj. $F' = F'[1:]$

end

$F' = F' + X$

end

ulož značku $\langle i \rangle$

LZW zde přidávalo $F+X$ do D

Aktualizace slovníku dle LZY

LZY - krok 1

- Kodér zahajuje svou činnost tím, že inicializuje slovník (naplní prvních 255 položek ASCII symboly) a přečte první symbol ze vstupního toku.
- Řetězec "a" se nachází ve slovníku na pozici 97.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



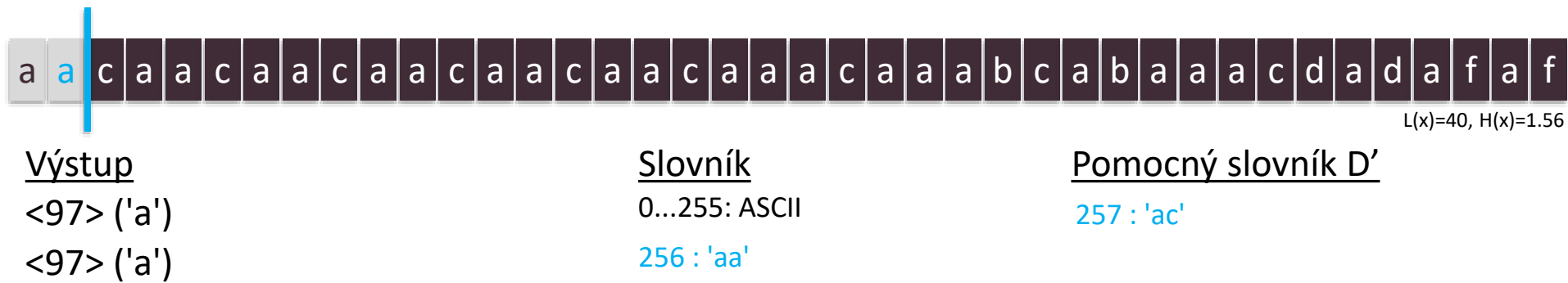
$L(x)=40, H(x)=1.56$

Slovník
0...255: ASCII

- Řetězec "aa" se ve slovníku nenachází.
- Kodér zapíše do výstupního toku značku <97>, zapamatuje si řetězec "a" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "aa"



- Řetězec "ac" se ve slovníku nenachází. Z pomocného slovníku je zkopírována 1 položka.
- Kodér запиše do výstupního toku značku <97>, zapamatuje si řetězec "c" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "ac"



- Řetězec "ca" se ve slovníku nenachází. Z pomocného slovníku je zkopírována 1 položka.
- Kodér zapíše do výstupního toku značku <99>, zapamatuje si řetězec "a" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "ca"



Výstup

<97> ('a')
<97> ('a')
<99> ('c')

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'

Pomocný slovník D'

258 : 'ca'

- Řetězec "aa" se nachází ve slovníku na pozici 256.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<97> ('a')
<97> ('a')
<99> ('c')

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'

Pomocný slovník D'

258 : 'ca'

- Řetězec "aac" se ve slovníku nenachází. Z pomocného slovníku je zkopírována 1 položka.
- Kodér zapíše do výstupního toku značku <256>, zapamatuje si řetězec "c" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "aac"



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'

Pomocný slovník D'

259 : 'aac'

- Řetězec "ca" se nachází ve slovníku na pozici 258.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.
- Do pomocného slovníku přidáme další položku, a sice "aca"



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'

Pomocný slovník D'

259 : 'aac'
260 : 'aca'

- Řetězec "ac" se nachází ve slovníku na pozici 257.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.
- Do pomocného slovníku přidáme další položku, a sice "caac"



<u>Výstup</u>	<u>Slovník</u>	<u>Pomocný slovník D'</u>
<97> ('a')	0...255: ASCII	263 : 'acaa'
<97> ('a')	256 : 'aa'	264 : 'caac'
<99> ('c')	257 : 'ac'	
<256>	258 : 'ca'	
<258>	259 : 'aac'	
<260>	260 : 'aca'	
	261 : 'caa'	
	262 : 'aaca'	

- Řetězec "aca" se nachází ve slovníku na pozici 260.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.
- Do pomocného slovníku aaca nepřidáváme, protože již je na pozici 262



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<260>

Slovník

0...255: ASCII
256 : 'aa'
257 : 'ac'
258 : 'ca'
259 : 'aac'
260 : 'aca'
261 : 'caa'
262 : 'aaca'

Pomocný slovník D'

263 : 'acaa'
264 : 'caac'

- Řetězec "ac" se nachází ve slovníku na pozici 257.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.
- Do pomocného slovníku přidáme další položku, a sice "acaac"



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<260>
<260>

Slovník

0...255: ASCII 263 : 'acaa'
256 : 'aa' 264 : 'caac'
257 : 'ac'
258 : 'ca'
259 : 'aac'
260 : 'aca'
261 : 'caa'
262 : 'aaca'

- Řetězec "aca" se nachází ve slovníku na pozici 260.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.
- Do pomocného slovníku přidáme další položku, a sice "caaca"



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<260>
<260>

Slovník

0...255: ASCII 263 : 'acaa'
256 : 'aa' 264 : 'caac'
257 : 'ac'
258 : 'ca'
259 : 'aac'
260 : 'aca'
261 : 'caa'
262 : 'aaca'

- Řetězec "acaac" se ve slovníku nenachází. Z pomocného slovníku jsou zkopírovány 3 položky.
- Kodér запиše do výstupního toku značku <263>, zapamatuje si řetězec "c" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "aacaac"



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<260>
<260>
<263>

Slovník

0...255: ASCII	263 : 'acaa'
256 : 'aa'	264 : 'caac'
257 : 'ac'	265 : 'aaca'
258 : 'ca'	266 : 'acaac'
259 : 'aac'	267 : 'caaca'
260 : 'aca'	
261 : 'caa'	
262 : 'aaca'	

- Řetězec "ca" se nachází ve slovníku na pozici 258.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.
- Do pomocného slovníku přidáme další položku, a sice "acaaca"



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<260>
<260>
<263>

Slovník

0...255: ASCII	263 : 'acaa'
256 : 'aa'	264 : 'caac'
257 : 'ac'	265 : 'aaca'
258 : 'ca'	266 : 'acaac'
259 : 'aac'	267 : 'caaca'
260 : 'aca'	
261 : 'caa'	
262 : 'aaca'	

- Řetězec "caa" se nachází ve slovníku na pozici 261.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.
- Do pomocného slovníku přidáme další položku, a sice "caacaa"



Výstup

<97> ('a')
<97> ('a')
<99> ('c')
<256>
<258>
<260>
<260>
<263>

Slovník

0...255: ASCII	263 : 'acaa'
256 : 'aa'	264 : 'caac'
257 : 'ac'	265 : 'aacaa'
258 : 'ca'	266 : 'acaac'
259 : 'aac'	267 : 'caaca'
260 : 'aca'	
261 : 'caa'	
262 : 'aaca'	

- Řetězec "caaa" se ve slovníku nenachází. Z pomocného slovníku jsou zkopírovány 3 položky.
- Kodér zapíše do výstupního toku značku <261>, zapamatuje si řetězec "a" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme další položku, a sice "aaa"



Výstup

<97> ('a') <261>
<97> ('a')
<99> ('c')
<256>
<258>
<260>
<260>
<263>

Slovník

0...255: ASCII	263 : 'acaa'
256 : 'aa'	264 : 'caac'
257 : 'ac'	265 : 'aacao'
258 : 'ca'	266 : 'acaac'
259 : 'aac'	267 : 'caaca'
260 : 'aca'	268 : 'aacaac'
261 : 'caa'	269 : 'acaaca'
262 : 'aaca'	270 : 'caacaa'

- Řetězec "ac" se nachází ve slovníku na pozici 257.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<97> ('a') <261>
<97> ('a')
<99> ('c')
<256>
<258>
<260>
<260>
<263>

Slovník

0...255: ASCII	263 : 'acaa'
256 : 'aa'	264 : 'caac'
257 : 'ac'	265 : 'aacaa'
258 : 'ca'	266 : 'acaac'
259 : 'aac'	267 : 'caaca'
260 : 'aca'	268 : 'aacaac'
261 : 'caa'	269 : 'acaaca'
262 : 'aaca'	270 : 'caacaa'

- Řetězec "aca" se nachází ve slovníku na pozici 260.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup	Slovník
<97> ('a') <261>	0...255: ASCII 263 : 'acaa'
<97> ('a')	256 : 'aa' 264 : 'caac'
<99> ('c')	257 : 'ac' 265 : 'aacia'
<256>	258 : 'ca' 266 : 'acaac'
<258>	259 : 'aac' 267 : 'caaca'
<260>	260 : 'aca' 268 : 'aacaac'
<260>	261 : 'caa' 269 : 'acaaca'
<263>	262 : 'aaca' 270 : 'caacaa'

- Řetězec "acaa" se nachází ve slovníku na pozici 263.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<97> ('a') <261>
<97> ('a')
<99> ('c')
<256>
<258>
<260>
<260>
<263>

Slovník

0...255: ASCII	263 : 'acaa'
256 : 'aa'	264 : 'caac'
257 : 'ac'	265 : 'aacia'
258 : 'ca'	266 : 'acaac'
259 : 'aac'	267 : 'caaca'
260 : 'aca'	268 : 'aacaac'
261 : 'caa'	269 : 'acaaca'
262 : 'aaca'	270 : 'caacaa'

- Řetězec "acaaa" se ve slovníku nenachází. Z pomocného slovníku jsou zkopírovány 4 položky.
- Kodér zapíše do výstupního toku značku <263>, zapamatuje si řetězec "a" a posune se o jeden znak doprava.



Výstup

<97> ('a') <261>
<97> ('a') <263>
<99> ('c')
<256>
<258>
<260>
<260>
<263>

Slovník

0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'
256 : 'aa'	264 : 'caac'	272 : 'acaaa'
257 : 'ac'	265 : 'aacaa'	273 : 'caaa'
258 : 'ca'	266 : 'acaac'	274 : 'aaa'
259 : 'aac'	267 : 'caaca'	
260 : 'aca'	268 : 'aacaac'	
261 : 'caa'	269 : 'acaaca'	
262 : 'aaca'	270 : 'caacaa'	

-
- $L(x)=40, H(x)=1.56$

```
<97> ('a')    <261>
<97> ('a')    <263>
<99> ('c')    <97> ('a')
<256>
<258>
<260>
<260>
<263>
```

0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'
256 : 'aa'	264 : 'caac'	272 : 'acaaa'
257 : 'ac'	265 : 'aaciaa'	273 : 'caaaa'
258 : 'ca'	266 : 'acaac'	274 : 'aaa'
259 : 'aac'	267 : 'caaca'	
260 : 'aca'	268 : 'aacaac'	
261 : 'caa'	269 : 'acaaca'	
262 : 'aaca'	270 : 'caacaa'	

00110000100110000100110001110000000010000001010000010010000010010000011110000010110000011100100001

$L_{OUT}=99$ bits

- Řetězec "bc" se ve slovníku nenachází. Z pomocného slovníku je zkopírováno 6 položek.
- Kodér zapíše do výstupního toku značku <98>, zapamatuje si řetězec "c" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "bc"



Výstup

<97> ('a') <261>
<97> ('a') <263>
<99> ('c') <97> ('a')
<256> <98> ('b')
<258>
<260>
<260>
<263>

Slovník

0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'
256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'
257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	
258 : 'ca'	266 : 'acaac'	274 : 'aaa'	
259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	
260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	
261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	
262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	

- Řetězec "ca" se nachází ve slovníku na pozici 258.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup		Slovník			
<97> ('a')	<261>	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'
<97> ('a')	<263>	256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	
<256>	<98> ('b')	258 : 'ca'	266 : 'acaac'	274 : 'aaa'	
<258>		259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	
<260>		260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	
<260>		261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	
<263>		262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	

- Řetězec "cab" se ve slovníku nenachází. Z pomocného slovníku je zkopírována 1 položka.
- Kodér запиše do výstupního toku značku <258>, zapamatuje si řetězec "b" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "cab"



Výstup		Slovník			
<97> ('a')	<261>	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'
<97> ('a')	<263>	256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'
<256>	<98> ('b')	258 : 'ca'	266 : 'acaac'	274 : 'aaa'	
<258>	<258>	259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	
<260>		260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	
<260>		261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	
<263>		262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	

00110000100110000100110001110000000010000001010000010010000010010000011110000010110000011100110000100110001010000

0010

L_{OUT}=117 bits

- Řetězec "ba" se ve slovníku nenachází. Z pomocného slovníku je zkopírována 1 položka.
- Kodér zapíše do výstupního toku značku <98>, zapamatuje si řetězec "a" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme další položku, a sice "ba"



Výstup		Slovník			
<97> ('a')	<261>	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'
<97> ('a')	<263>	256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'
<256>	<98> ('b')	258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'
<258>	<258>	259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	
<260>	<98> ('b')	260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	
<260>		261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	
<263>		262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	

- Řetězec "aa" se nachází ve slovníku na pozici 256.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<97> ('a') <261>
<97> ('a') <263>
<99> ('c') <97> ('a')
<256> <98> ('b')
<258> <258>
<260> <98> ('b')
<260>
<263>

Slovník

0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'
256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'
257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'
258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'
259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	
260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	
261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	
262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	

- Řetězec "aaa" se nachází ve slovníku na pozici 274.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup

<97> ('a') <261>
<97> ('a') <263>
<99> ('c') <97> ('a')
<256> <98> ('b')
<258> <258>
<260> <98> ('b')
<260>
<263>

Slovník

0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'
256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'
257 : 'ac'	265 : 'aacia'	273 : 'caaa'	281 : 'bc'
258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'
259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	
260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	
261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	
262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	

00110000100110000100110001110000000010000001010000010010000010010000011110000010110000011100110000100110001010000

0010001100010

L_{OUT}=126 bits

- Řetězec "aaac" se ve slovníku nenachází. Z pomocného slovníku jsou zkopírovány 2 položky.
- Kodér zapíše do výstupního toku značku <274>, zapamatuje si řetězec "c" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "aaac"



Výstup		Slovník			
<97> ('a')	<261>	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'
<97> ('a')	<263>	256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'
<256>	<98> ('b')	258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'
<258>	<258>	259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	283 : 'aba'
<260>	<98> ('b')	260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	284 : 'ba'
<260>	<274>	261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	
<263>		262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	

- Řetězec "cd" se ve slovníku nenachází. Z pomocného slovníku je zkopírována 1 položka.
- Kodér zapíše do výstupního toku značku <99>, zapamatuje si řetězec "d" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme další položku, a sice "cd"



Výstup		Slovník			
<97> ('a')	<261>	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'
<97> ('a')	<263>	256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'
<99> ('c')	<97> ('a')	257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'
<256>	<98> ('b')	258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'
<258>	<258>	259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	283 : 'aba'
<260>	<98> ('b')	260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	284 : 'ba'
<260>	<274>	261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	285 : 'aac'
<263>	<99> ('c')	262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	

- Řetězec "da" se ve slovníku nenachází. Z pomocného slovníku jsou zkopírovány 3 položky.
- Kodér запиše do výstupního toku značku <100>, zapamatuje si řetězec "a" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "da"



Výstup			Slovník				
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'	287 : 'acd'
<97> ('a')	<263>		256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'	288 : 'cd'
<99> ('c')	<97> ('a')		257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'	
<256>	<98> ('b')		258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'	
<258>	<258>		259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	283 : 'aba'	
<260>	<98> ('b')		260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	284 : 'ba'	
<260>	<274>		261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	285 : 'aaac'	
<263>	<99> ('c')		262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	286 : 'aacd'	

- Řetězec "ad" se ve slovníku nenachází. Z pomocného slovníku je zkopírována 1 položka.
- Kodér zapíše do výstupního toku značku <97>, zapamatuje si řetězec "d" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "ad"



Výstup			Slovník				
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'	287 : 'acd'
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'	288 : 'cd'
<99> ('c')	<97> ('a')		257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'	289 : 'da'
<256>	<98> ('b')		258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'	
<258>	<258>		259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	283 : 'aba'	
<260>	<98> ('b')		260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	284 : 'ba'	
<260>	<274>		261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	285 : 'aaac'	
<263>	<99> ('c')		262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	286 : 'aacd'	

- Řetězec "da" se nachází ve slovníku na pozici 289.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup			Slovník				
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'	287 : 'acd'
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'	288 : 'cd'
<99> ('c')	<97> ('a')		257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'	289 : 'da'
<256>	<98> ('b')		258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'	
<258>	<258>		259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	283 : 'aba'	
<260>	<98> ('b')		260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	284 : 'ba'	
<260>	<274>		261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	285 : 'aaac'	
<263>	<99> ('c')		262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	286 : 'aacd'	

00110000100110000100110001110000000010000001010000010010000010010000011110000010110000011100110000100110001010000
0010001100010100010010001100011001100100001100001

- Řetězec "daf" se ve slovníku nenachází. Z pomocného slovníku je zkopírována 1 položka.
- Kodér zapíše do výstupního toku značku <289>, zapamatuje si řetězec "f" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme další položku, a sice "af"



Výstup			Slovník				
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'	287 : 'acd'
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'	288 : 'cd'
<99> ('c')	<97> ('a')	<289>	257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'	289 : 'da'
<256>	<98> ('b')		258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'	290 : 'ad'
<258>	<258>		259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	283 : 'aba'	
<260>	<98> ('b')		260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	284 : 'ba'	
<260>	<274>		261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	285 : 'aaac'	
<263>	<99> ('c')		262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	286 : 'aacd'	

00110000100110000100110001110000000010000001010000010010000010010000011110000010110000011100110000100110001010000
0010001100010100010010001100011001100100001100001100100001

- Řetězec "fa" se ve slovníku nenachází. Z pomocného slovníku jsou zkopírovány 2 položky.
- Kodér zapíše do výstupního toku značku <102>, zapamatuje si řetězec "a" a posune se o jeden znak doprava.
- Do pomocného slovníku přidáme první položku, a sice "fa"



Výstup

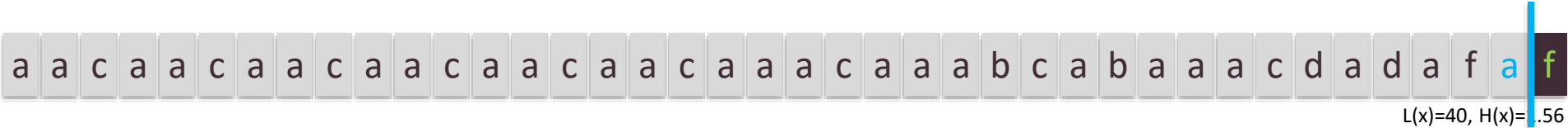
<97> ('a')	<261>	<100> ('d')
<97> ('a')	<263>	<97> ('a')
<99> ('c')	<97> ('a')	<289>
<256>	<98> ('b')	<102> ('f')
<258>	<258>	
<260>	<98> ('b')	
<260>	<274>	
<263>	<99> ('c')	

Slovník

0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'	287 : 'acd'
256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'	288 : 'cd'
257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'	289 : 'da'
258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'	290 : 'ad'
259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	283 : 'aba'	291 : 'daf'
260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	284 : 'ba'	292 : 'af'
261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	285 : 'aaac'	
262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	286 : 'aacd'	

00110000100110000100110001110000000010000001010000010010000010010000011110000010110000011100110000100110001010000
0010001100010100010010001100011001100100001100001100100001001100110

- Řetězec "af" se nachází ve slovníku na pozici 292.
- Kodér si řetězec zapamatuje a posune se o jeden znak doprava, aby mohl otestovat delší shodu.



Výstup			Slovník				
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'	287 : 'acd'
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'	288 : 'cd'
<99> ('c')	<97> ('a')	<289>	257 : 'ac'	265 : 'aacia'	273 : 'caaa'	281 : 'bc'	289 : 'da'
<256>	<98> ('b')	<102> ('f')	258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'	290 : 'ad'
<258>	<258>		259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	283 : 'aba'	291 : 'daf'
<260>	<98> ('b')		260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	284 : 'ba'	292 : 'af'
<260>	<274>		261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	285 : 'aaac'	
<263>	<99> ('c')		262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	286 : 'aacd'	

00110000100110000100110001110000000010000001010000010010000010010000011110000010110000011100110000100110001010000
0010001100010100010010001100011001100100001100001100100001001100110

- Kodér přečetl celý vstupní tok a končí nalezením shody délky dva symboly. Na výstup proto zapíše token <292> a svou činnost končí.



Výstup			Slovník					
<97> ('a')	<261>	<100> ('d')	0...255: ASCII	263 : 'acaa'	271 : 'aacaaa'	279 : 'aab'	287 : 'acd'	
<97> ('a')	<263>	<97> ('a')	256 : 'aa'	264 : 'caac'	272 : 'acaaa'	280 : 'ab'	288 : 'cd'	
<99> ('c')	<97> ('a')	<289>	257 : 'ac'	265 : 'aacaa'	273 : 'caaa'	281 : 'bc'	289 : 'da'	
<256>	<98> ('b')	<102> ('f')	258 : 'ca'	266 : 'acaac'	274 : 'aaa'	282 : 'cab'	290 : 'ad'	
<258>	<258>	<292>	259 : 'aac'	267 : 'caaca'	275 : 'aacaaab'	283 : 'aba'	291 : 'daf'	
<260>	<98> ('b')		260 : 'aca'	268 : 'aacaac'	276 : 'acaaab'	284 : 'ba'	292 : 'af'	
<260>	<274>		261 : 'caa'	269 : 'acaaca'	277 : 'caaab'	285 : 'aaac'		
<263>	<99> ('c')		262 : 'aaca'	270 : 'caacaa'	278 : 'aaab'	286 : 'aacd'		

00110000100110000100110001110000000010000001010000010010000010010000011110000010110000011100110000100110001010000
0010001100010100010010001100011001100100001100001100100001001100110100100100

Komprese pomocí LZY - výstup

- Vstupní řetězec byl zakódován pomocí 189 bitů (4,725 bitů na symbol). Dosažen kompresní faktor 0,590 (189 / 40*8), tj. lepší než LZW (0,618) i LZ78 (0,875) ale horší než v případě LZSS (0,443) i Huffmanova kódu (0,459).
- Poznámka: výstup neobsahuje žádnou hlavičku, parametry kodéru (maximální počet položek slovníku) jsou předány externě. Token má fixní délku: 9b

a a c a a c a a c a a c a a c a a a c a a a b c a b a a a c d a d a f a f

L(x)=40, H(x)=1.56



data

00110000100110000100110001110000000010000001010000010010
00001001000001111000001011000001110011000010011000101000
00010001100010100010010001100011001100100001100001100100
001001100110100100100

Výkonnost jednotlivých metod na testovacím řetězci

- Statistické metody
 - Entropie vstupu 1,556 bitů na symbol
 - Huffman, kanonická verze 145 bitů (3,625 bitů na symbol) z toho 65 bitů data (1,625 bitů na symbol)
 - Aritmetické kódování 103 bitů (2,575 bitů na symbol) z toho 69 bitů data (1,725 bitů na symbol)
- Slovníkové metody LZ77 (sliding window dictionary)
 - LZ77 145 bitů (3,625 bitů na symbol), token: 17 bitů
 - LZSS 142 bitů (3,55 bitů na symbol), token: 9 bitů
- Slovníkové metody LZ78 (dictionary of phrases)
 - LZ78 264 bitů (6,6 bitů na symbol), token: 9 bitů
 - LZW 198 bitů (4,95 bitů na symbol), token: 9 bitů
 - LZY 189 bitů (4,725 bitů na symbol), token: 9 bitů
 - LZAP 198 bitů (4,95 bitů na symbol), token: 9 bitů
 - LZMW 198 bitů (4,95 bitů na symbol), token: 9 bitů

Využití LZW v praxi – compress

- V Linuxu příkaz `compress/uncompress`, který používá LZW s rostoucím slovníkem.
 - Jedná se o jednu ze standardizovaných metod pro kompresi HTTP protokolu (Content-Encoding: `compress`)
- Princip metody (označované jako LZC):
 - Začíná se s **malým slovníkem**, s $2^9 = 512$ položkami. Do výstupního toku se zapisují 9-bitové ukazatele.
 - **Zaplnění způsobí zdvojnásobení velikosti** na 1024 položek, a od tohoto okamžiku se používají 10-bitové ukazatele. Tento proces pokračuje, až do velikosti maxima ukazatele, které nastavil uživatel (může se nastavit od 9 do 16 bitů, implicitní hodnota je 16)
 - Když se zaplní největší dovolený slovník, tak program pokračuje beze změny slovníku (ze kterého je tak **statický slovník**), avšak monitoruje se kompresní poměr. Když poměr klesne pod předem definovanou úroveň, **slovník se vymaže** a začne se s novým, s 512 položkami. Takto se slovník nikdy nestane „příliš zastaralý“.

Využití LZW v praxi – komprese obrázků formátu GIF

■ GIF – the graphics interchange format

- navržen firmou Compuserve Information Services v r. 1987 jako efektivní formát komprimovaných grafických souborů

■ Obrazová data jsou komprimována pomocí varianty LZW.

- Používá dynamický rostoucí slovník podobně jako compress. Začne s parametrem počtu bitů na pixel b ($b = 2$ pro monochromatický obrázek, $b = 8$ pro obrázek s 256 barvami nebo úrovněmi šedi).
- GIF skenuje obrázek řádek po řádku, takže vyhledává korelace v rámci řádku, ale ne mezi řádky. Můžeme říci, že GIF komprese je neefektivní, protože je jednorozměrná, zatímco obrázek je dvourozměrný.

■ Princip komprese:

- Slovník začíná s 2^{b+1} položkami, a pokaždé, když se zaplní, tak se jeho velikost **zdvojnásobí**, až dosáhne velikosti $2^{12} = 4096$ položek, kdy se stane statickým.
- Při dosažení maximální velikosti slovníku kodér **monitoruje kompresní poměr** a může se kdykoliv rozhodnout slovník zahodit, a začít s novým, prázdným. Když kodér učiní takové rozhodnutí, vyšle jako clear code hodnotu 2^b , což je pro dekodér pokyn, aby vynuloval svůj slovník.

■ Ukazatele, které se od jednoho slovníku ke druhému prodlužují o 1 bit, se střádají a vysílají v blocích 8-bitových bytů. Před každým blokem předchází záhlaví, které obsahuje velikost bloku (maximálně 256 bytů) a ukončuje se nulovým bytem. Poslední blok obsahuje těsně před ukončujícím znakem hodnotu eof, což je 2^{b+1} .



Využití LZ v praxi (informativní)

- ARC je kompresní/archivační/katalogizační program, který vyvinul uprostřed 80. let Robert A. Freed. Nabízí dobrou kompresi a schopnost kombinovat několik souborů do jednoho souboru, zvaného archiv. Využívá LZW a **kóduje značky pomocí Huffmanova kódu**.
- PKArc je vylepšená verze ARC. Vyvinul ji Philip Katz, který založil společnost PKWare, která prodávala software PKzip, PKunzip a PKlite. Programy PK jsou rychlejší a obecnější, než ARC a rovněž poskytují uživateli větší možnosti ovládání.
- LHArc od Haruyasu Yoshizaki a LHA od Haruhiko Okumury a Haruyasu Yoshizaki používají **adaptivní Huffmanovo kódování s rysy převzatými z LZSS**. Velikost okna LZ může být až 16 KB. ARJ je starší kompresní nástroj od Roberta K. Junga, který používá stejný algoritmus, ale zvětšuje okno na 26624 bytů.
- RAR/WinRAR a ACE/WinAce používají **LZSS** s velkým vyhledávacím bufferem (až 4 MB), kombinovaným s **Huffmanovým kódem** případně **kontextovou adaptivní statistickou kompresní metodou PPM**.
- 7-zip je opensource nástroj pro kompresi a podporuje předchozí formáty. Přichází s metodou **LZMA** využívající LZ77 s posuvným oknem o velikosti až 4GB a intervalové kódování (range encoding) .

Princip LZMA

- LZMA (tedy Lempel-Ziv-Markov chain-Algorithm) je výchozím algoritmem v 7z. Varianta LZ77 optimalizovaná pro vysoké kompresní poměry a rychlou dekompresi.
- Hlavní rysy LZMA:
 - Velikost slovníku je proměnlivá a může být až 4 GB (současná implementace má mez 1 GB).
 - Dekodér používá pouze operace typu integer a snadno se dá implementovat na 32-bitovém procesoru (implementací na 16-bitové CPU je víc).
 - Implementace využívá multi-threading. Současný kodér LZMA může používat jedno nebo dvě vlákna, a dekodér LZMA může použít pouze jedno vlákno.
- Princip komprese LZMA je obdobný metodě Deflate, avšak místo Huffmanova kódu používá *intervalové kódování* (range encoding)
 - To komplikuje konstrukci kodéru, ale vede na lepší kompresi (připomeňme, že intervalové kódování je celočíselná verze aritmetického kódování, a může komprimovat velmi blízko k entropii dat), přičemž se minimalizuje počet potřebných renormalizací. Intervalové kódování se realizuje binárně tak, že při dělení integerů se nepoužívá pomalé operace dělení, ale posuvy.
 - Lokalizace shod ve vyhledávacím bufferu se provádí pomocí hašování dvou bytů, to je *aktuálního bytu* v předvídacím bufferu a *bezprostředně pravého bytu*. Výstupem hašovací funkce je index do pole. Velikost pole se vybírá jako mocnina 2, která je nejbližší polovině velikosti slovníku. Pro velikost slovníku 256 MB (2^{28}) generuje 27 bitové hodnoty. Velké pole minimalizuje hašovací kolize.

Deflate

- Deflate je populární univerzální bezeztrátová kompresní metoda založená na variantě LZ77 kombinované s Huffmanovým kódem
- Metodu Deflate využívá řada aplikací
 - protokol HTTP (Content-Encoding: gzip, deflate, compress,...)
 - protokol pro řízení komprese PPP (CCP)
 - formáty grafických souborů PNG (Portable Network Graphics) a MNG (Multiple-Image Network Graphics)
 - nástroje pro kompresi (gzip)
- Nejznámější implementací je multiplatformní **knihovna zlib**
 - přenositelná a volná kompresní knihovna od Jean-Loup Gaillyho a Marka Adlera, kteří ji vyvinuli bez patentové a licenční zátěže.
 - knihovna implementuje formáty souborů ZLIB a GZIP (hlavička+checksum), které jsou jádrem většiny aplikací Deflate, včetně populárního software Gzip.
- Princip:
 - Deflate komprimuje **vstupní tok po blocích**, kdy se každý blok komprimuje odděleně.
 - Na rozdíl od LZ77 nezapisuje token obsahující tři pole, ale používá tři typy entit: **literály** (znaky beze shody), **offsety** (nazývané též vzdálenosti), a **délky**.
 - Do komprimovaného toku se zapisují **Huffmanovy kódy**, přičemž se používají **dvě kódovací tabulky** – jedna pro literály a délky, druhá pro vzdálenosti.



Deflate – zpracování vstupního souboru

■ Použití dvou tabulek Huffmanových kódů:

- Literály jsou typicky byty ležící v intervalu $[0, 255]$, délky jsou omezeny na 258
- Vzdálenosti mohou být velká čísla, Deflate dovoluje vyhledávací buffer až 32 KB

■ Zpracování vstupního souboru po blocích:

- Každý blok se komprimuje odděleně.
- Bloky mohou mít různé délky. Délku bloku určí kodér na základě odlišných použitých prefixových kódových kombinací (jejich délka je omezena na 15 bitů), a volné paměti pro dekodér (s výjimkou toho, že v režimu 1 jsou bloky omezeny na 65535 bytů nekomprimovaných dat).
- Dekodér Deflate musí být schopen dekódovat bloky jakékoliv délky.
- Deflate nabízí tři režimy komprese, a každý blok může být v kterémkoliv z režimů.

■ Kódování bloků

- Každý blok začíná 3-bitovým záhlavím
- První bit je flag určující zda-li se jedná o poslední blok (hodnota 1)
- Další dva bity mohou být 00, 01 a 10 a určují režim (po řadě 1, 2 nebo 3) ve kterém byl blok zpracován.
- Za záhlavím následují vlastní data, jejichž formát se liší dle zvoleného režimu.



Deflate - režimy komprese

■ Režimy komprese:

1. **Bez komprese.** Tento režim má význam pro soubory, nebo části souborů, které jsou nekomprimovatelné, tj. náhodné, nebo jsou již komprimované, nebo případy, kdy se po kompresním software chce segmentovat soubor bez komprese.
2. **Komprese s fixními kódovými tabulkami.** Do kodéru i dekodéru jsou zabudovány dvě kódové tabulky s předdefinovaným obsahem. To urychluje kompresi i dekompresi a má další výhodu v tom, že kódové tabulky se nemusí zapisovat do komprimovaného toku. Pokud se však komprimovaná data statisticky liší od dat, která byla použita pro konstrukci těchto tabulek, tak se účinnost komprese snižuje. Využívá se pro krátké textové zprávy.
3. **Komprese s individuálními kódovacími tabulkami.** Kodér během komprese bloků shromažďuje statistiky o datech, a může být schopen během postupu z bloku na blok konstruovat zdokonalené kódovací tabulky. Obě kódovací tabulky se však musí zapsat do komprimovaného toku (v komprimovaném formátu). Většina bloků je komprimována v tomto režimu.

Deflate – tabulky

- Zvýšení kompresního poměru se dosahuje nejen efektivnějším hledáním shody ve vyhledávacím bufferu (viz dále), ale také efektivním kódováním značek.
- Kódovací tabulka pro literály a délky:
 - Kanonický Huffmanův strom má 288 symbolů s následujícím významem
 - 0–255: reprezentuje literály/byty
 - 256: symbol **end-of-block** označující konec aktuálně zpracovávaného bloku
 - 257–264: symboly kódující shodu délky 3–10 bytů
 - 265–268: symboly kódující shodu délky 11–18 bytů, 1 extra bit; např. symbol 265 kóduje délku shody buď 11 nebo 12 bitů v závislosti na extra bitu
 - 269–272: symboly kódující shodu délky 19–34 bytů, využívají se 2 extra bity
 - 273–276: symboly kódující shodu délky 35–66 bytů, využívají se 3 extra bity
 - 277–280: symboly kódující shodu délky 67–130 bytů, využívají se 4 extra bity
 - 281–284: symboly kódující shodu délky 131–257 bytů, využívá se 5 extra bitů
 - 285: symbol kódující shodu délky 258 bytů
 - 286, 287: nevyužívané



Deflate – tabulky

■ Kódovací tabulka pro vzdálenosti:

– Kanonický Huffmanův strom má 32 symbolů s následujícím významem

- 0–3: vzdálenost 1–4
- 4–5: vzdálenost 5–8, 1 extra bit
- 6–7: vzdálenost 9–16, 2 extra bity
- 8–9: vzdálenost 17–32, 3 extra bity
- ...
- 26–27: vzdálenost 8193–16384, 12 extra bitů
- 28–29: vzdálenost 16385–32768, 13 extra bitů
- 30–31: nevyužívá se

■ V režimu 2 jsou Huffmanovy kódy fixní a neobjeví se ve výstupním toku.

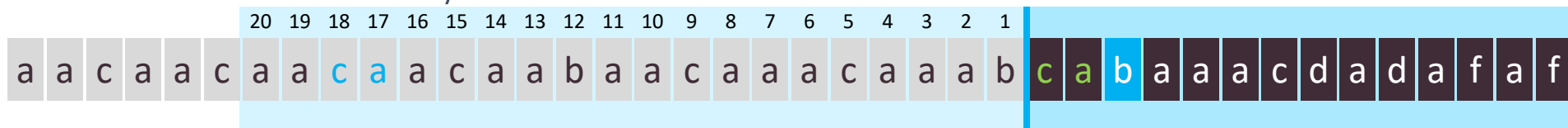
■ V režimu 3 jsou zapsány ihned za tříbitovou hlavičkou. Zapisují se pouze kódy nikoliv symboly, ty jsou fixní.

Deflate – implementace

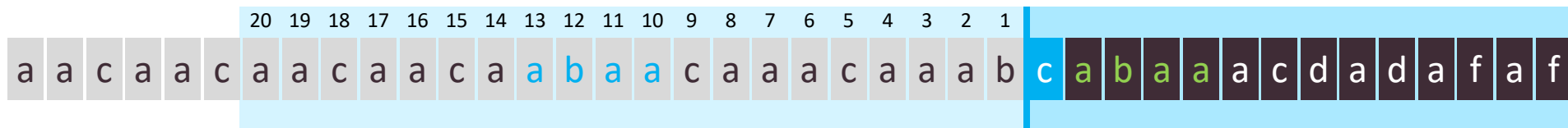
- Algoritmus dovoluje uživateli zvolit úroveň komprese od 0 do 9 přičemž
 - 0) No compression data se nekomprimují, tj. použije se režim 0 pro všechny bloky
 - 1) Best speed nejvyšší rychlost, avšak nejnižší kompresní poměr
 - 9) Best compression nejlepší kompresní poměr, avšak pomalé
- V režimu vysoké komprese se vždy provádí **odložený výběr shody**, jehož cílem je zefektivnit kompresi. Než kodér vybere shodu, zkusí se posunout o znak doprava a nalézt delší shodu.

■ Příklad

- dekodér našel shodu délky 2



- před vygenerováním výstupu se však zkusí posunout o znak doprava a nalézt shodu delší. V našem případě našel shodu délky 4 a proto запиše na výstup znak c a shodu.



GZIP

- Software pro archivaci dat, který umožňuje zkomprimovat jeden soubor.
- Využívá algoritmus DEFLATE.
- Zveřejněn roku 1992 jako sada příkazů gzip, gunzip a zcat.
- Formát GZIP se používá např. v protokolu HTTP pro kompresi přenášených dat.

```
HTTP/1.1 200 OK
Server: github.com
Content-Type: text/html; charset=utf-8
Content-Encoding: gzip
...
```

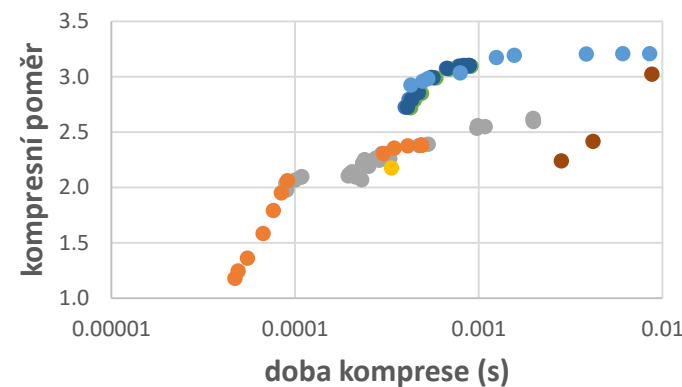
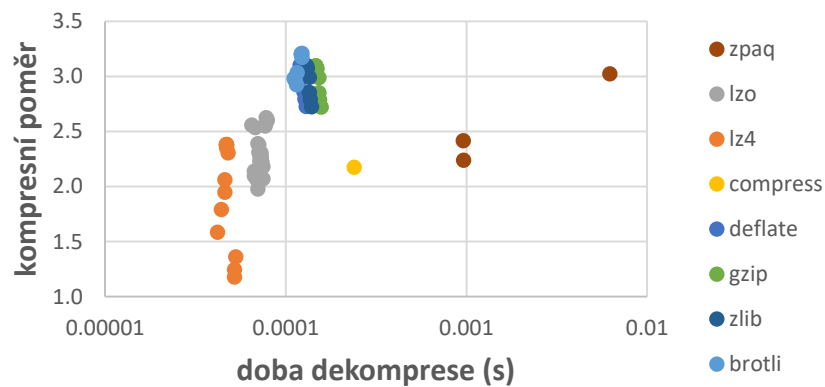
- Formát souboru GZIP (standardizován RFC 1952):
 - 10-bytová hlavička obsahující tzv. magic number (1F 8B), identifikátor komprese, atributy souboru, timestamp, parametry komprese and identifikátor OS
 - volitelná extra hlavička obsahující např. původní název souboru
 - tělo obsahující data zkomprimovaná pomocí algoritmu DEFLATE
 - 8-bytová patička obsahující kontrolní součet CRC-32, původní délku dat modulo 232

Brotli (informativní)

- Brotli (Google 2013-2016) je kompresní metoda kombinující LZ77 s Huffmanovým kódováním a modelem kontextu druhého řádu.
- Metoda byla původně navržena pro kompresi webových fontů WOFF2 a v roce 2015 zobecněna na kompresi HTTP (standardizováno RFC 7932).
- V porovnání s DEFLATE dosahuje o cca 12-16% lepšího kompresního poměru při zachování rychlosti dekomprese (Canterbury corpus, Web content corpora).

```
HTTP/1.1 200 OK
Server: google.com
Content-Type: text/html; charset=utf-8
Content-Encoding: br
...
```

quixdb.github.io/squash-benchmark / HTML source 24kB

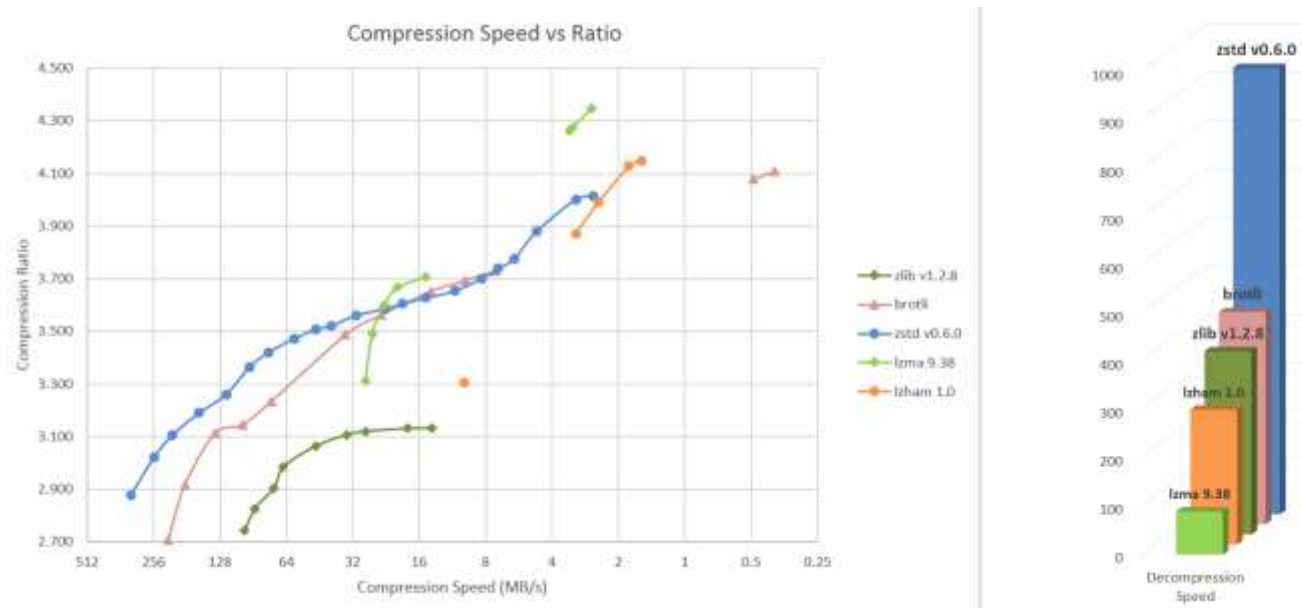


Brotli (informativní)

- Komprese je založena na LZ77 s posuvným oknem. Metoda používá posuvné okno odpovídající velikosti mocniny dvou v rozsahu 2^{10} až 2^{24} (1kB až 16MB).
- Pro kompresi malých souborů používá statický slovník.
- Soubor je zpracováván po 64kB blocích.
- Token má tvar
 - $\langle \text{length, literal, literal, ..., literal, distance} \rangle$, kde length určuje délku shody, literal nekomprimovatelný znak a distance vzdálenost shody. Dekodér vytvoří následující sekvenci: literal, literal, ..., literal, copy, copy, ..., copy, kde copy odpovídá shodě.
 - Pole length, distance a dokonce i jednotlivé literály jsou kódovány pomocí prefixového kódu, pro každou položku se používá vlastní kód.
 - Pokud hodnota distance je větší, než-li definuje okno, tak distance určuje jednak index do statického slovníku a jednak transformaci (přidání mezery před, za, přidání the, tvorba přídavného jména, atd., existuje celkem 121 možností), která se má nad slovem ze slovníku provést. Length určuje délku kopírovaného slova.

Zstandard (informativní)

- Zstandard (Facebook 2016) je kompresní metoda kombinující LZ77 s velkým oknem s entropickým kódováním (Finite state entropy nebo Huffmanův kód).
- Metoda je optimalizována s ohledem na zachování obdobného kompresního poměru jako DEFLATE avšak s výrazně vyšší rychlostí komprese a dekomprese.
- Stejně jako Brotli dovoluje uživateli použít vlastní slovník.



Očekávané znalosti

- **Princip slovníkových metod se statickým a dynamickým slovníkem**
 - výhody dynamického slovníku, tvary tokenů
- **Slovníkové metody na bázi posuvného okna**
 - Princip metody LZ77, určení parametrů a paměťových nároků, výpočetní náročnost
 - Princip metody LZSS, vlastnosti a rozdíly oproti LZ77
 - Efektivní vyhledávání pomocí BST
- **Slovníkové metody se slovníkem**
 - Princip LZ78 ,výpočetní náročnost, datová struktura trie
 - Využití hashovací funkce v LZRW1, LZW, princip, tvar hash funkcí
 - Princip LZW, vlastnosti a rozdíly oproti LZ78
 - Efektivní vyhledávání pomocí trie implementované jako pole uzlů
 - Varianty LZW a jejich princip (LZMW, LZAP)
- **Kombinované slovníkové metody LZFG varianty A1 a A2**
 - Princip, způsob využití start-step-stop kódování
- **Princip komprese Compress, Deflate, využití v gzip**